2025 동계 비교과 SKILL-UP 프로그램
-자율주행 시뮬레이션 실습(CARLA)-

January 5-6, 2025

Yongseok Kim

전체 강의 개요

- ROS2 기초 및 실습(**1H**, **26/01/05**)
  - ROS 기초 이해와 기본 구조
  - ROS2 설치 및 실습
- CARLA(**3H**, **26/01/05**)
  - CARLA 설치
  - CARLA 시뮬레이터 기능 실습
  - 자율주행 실습 환경 구축
  - ROS2-CARLA 연동 실습
    - **차선 인식 주행**
- CARLA 자율주행 실습(**2H**, **26/01/06**)
  - Perception
  - Planning
  - Control

Linux

▶ 기본 명령어

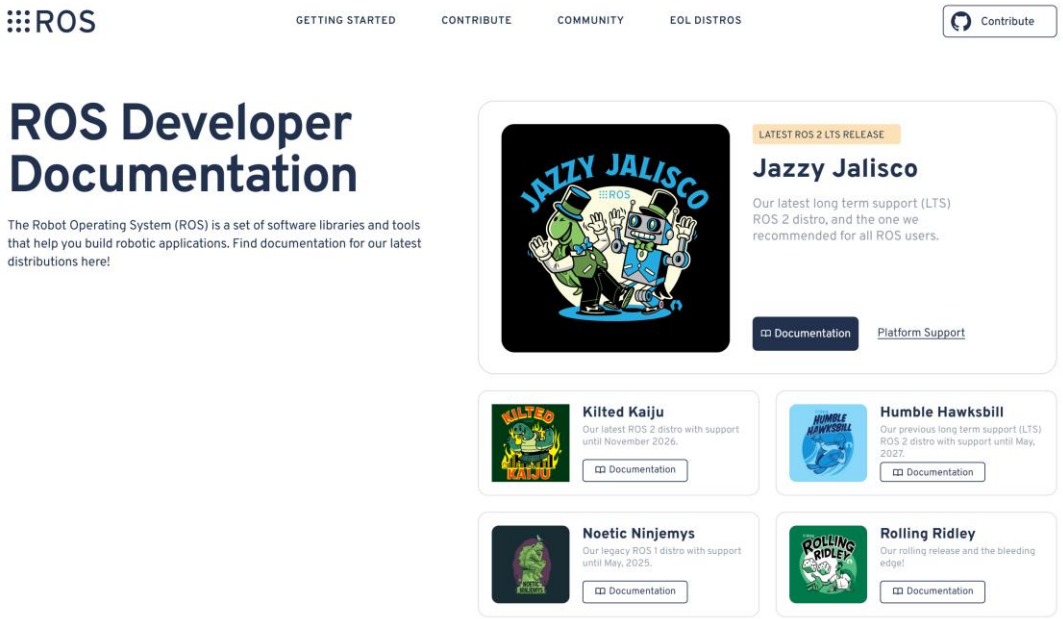
Linux command	Description	Linux command example
cd ★	Change directory with a specified path	cd */path/directory1*
clear	Clear the screen	clear
cp ★	Copy file(s)	cp */path1/file1 /path2/file1*
diff	Compare the contents of files	diff *file1 file2*
exit	Log out of Linux	exit
grep ★	Find a string of text in a file	grep "word or phrase" *file1*
head	Display beginning of a file	head *file1*
less	View a file	less *file1*
ls ★	List contents of a directory	ls */path/directory1*
mv ★	Move file(s) or rename file(s)	mv */path1/file1 /path2/file2*
mkdir ★	Create a directory	mkdir *directory*
rm ★	Delete file(s)	rm *file1*
rmdir	Remove a directory	rmdir *directory*
tail	Display end of a file	tail *file1*
tar	Store, list or extract files in an archive	tar *file1*
vi	Edit file(s) with simple text editor	vi *file1*

Robot Operating System(ROS)

Introduction

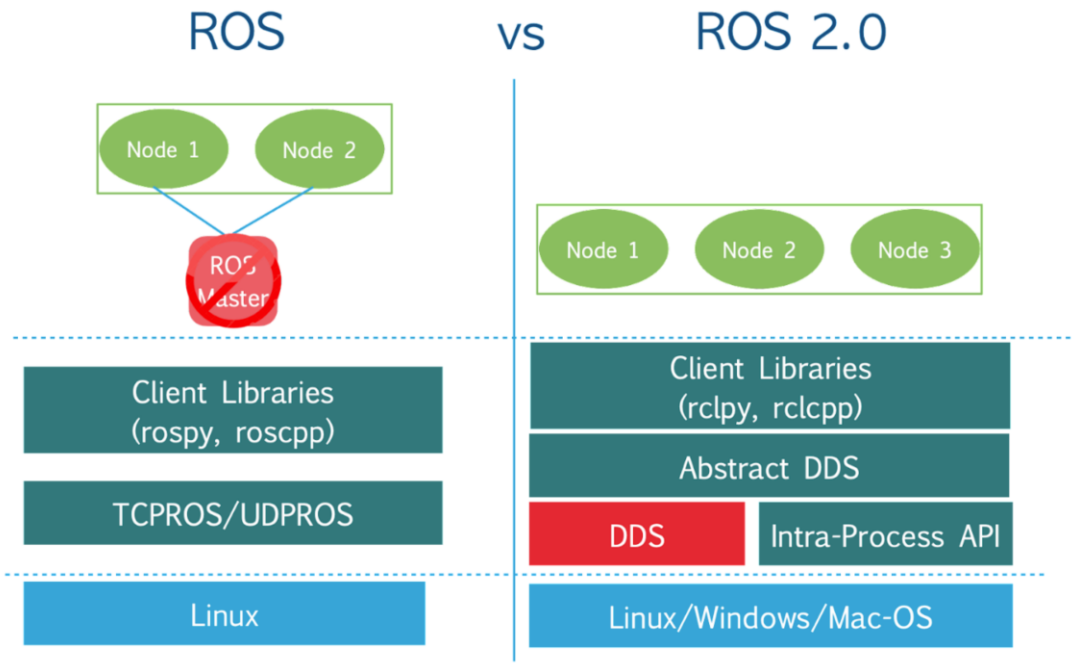
Robot Operating System

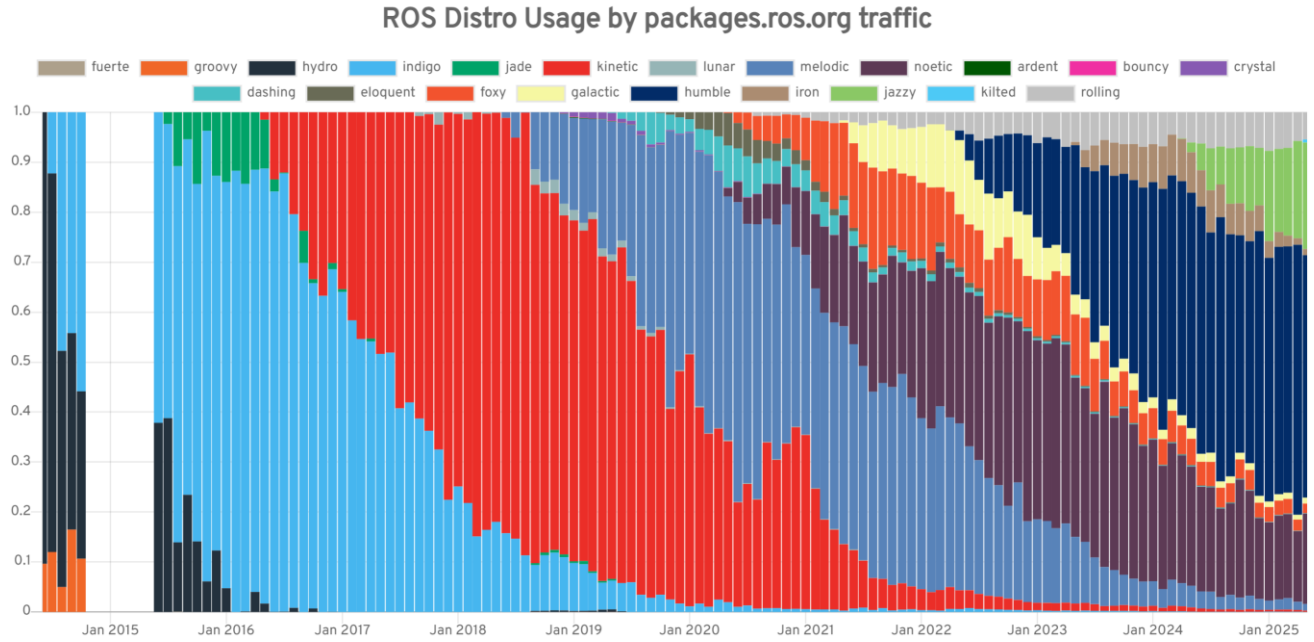
- 로봇 애플리케이션 구축을 위한 소프트웨어 라이브러리, 도구
  - 2007년 스탠포드 인공지능 연구소에서 시작
  - 로봇 개발에 필요한 좌표계, 통신, 센서 등을 패키지 형태로 쉽게 배포
    - 알고리즘 구현, 데이터 분석, 시각화 용이
  - 2025년 5월 31일 ROS1 지원 종료 -> ROS2 전환



Introduction

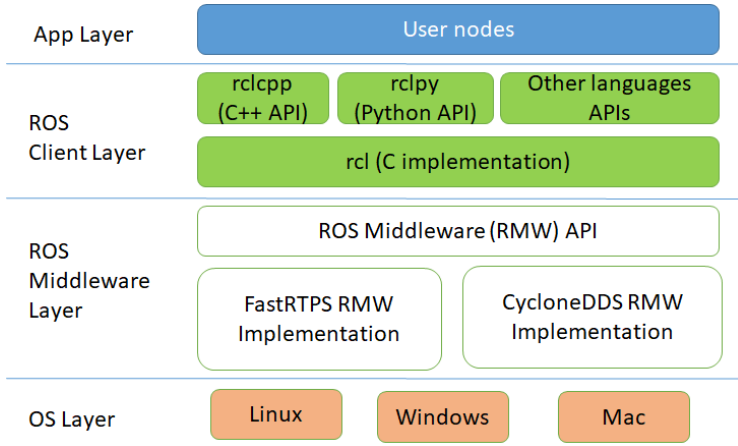
- ROS vs. ROS2
  - 2025년 5월 31일 ROS1 공식 지원 종료
    - 보안, 버그 등 문제 발생 시 대처 X
  - 통신, 지원 OS 등 여러 단점 보완

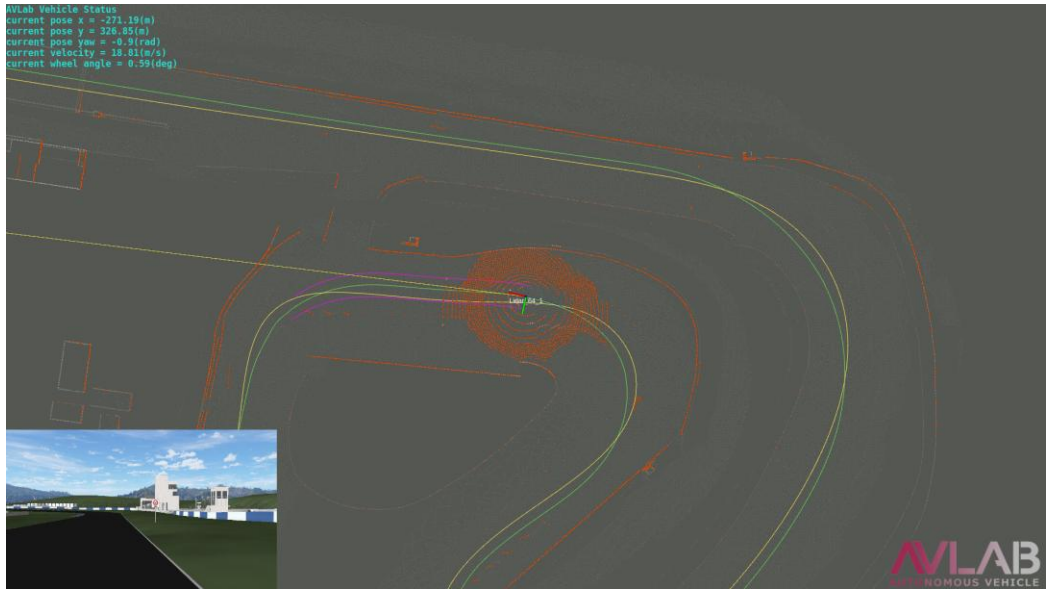




Introduction

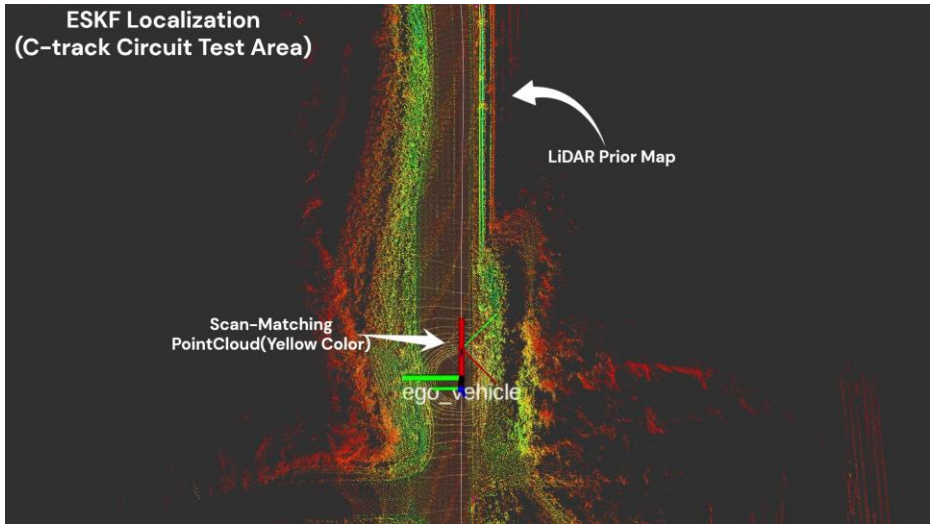
- Robot Operating System
  - 로봇 애플리케이션 구축을 위한 소프트웨어 라이브러리, 도구
  - Meta-OS로써 여러 OS와 호환
    - Linux, Windows, Mac 등
  - 다양한 센서를 쉽게 통합하여 사용 가능
    - LiDAR, Camera, IMU, GNSS/INS 등

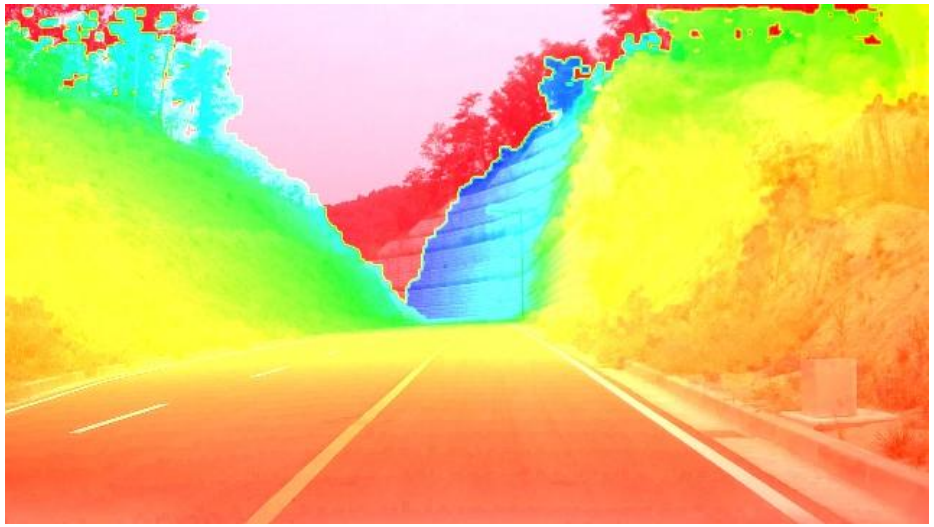


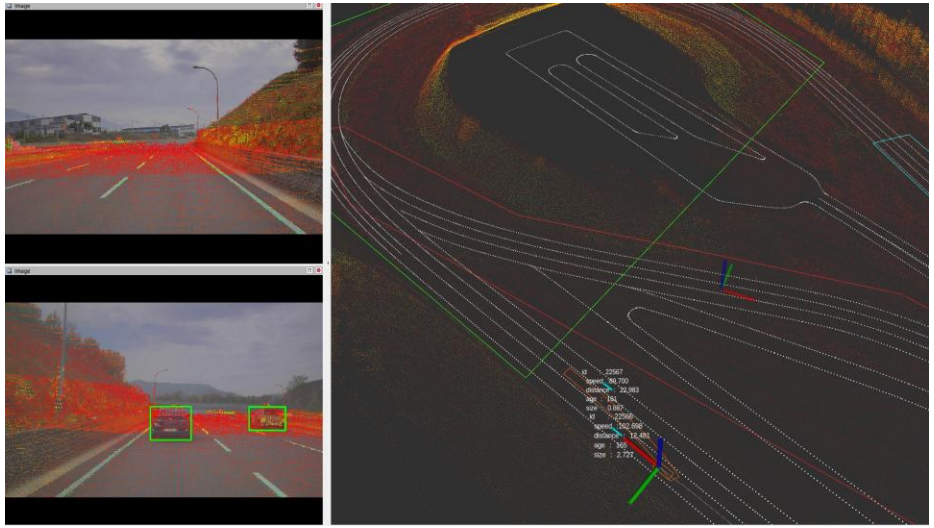


Introduction

- Some examples using ROS







Introduction

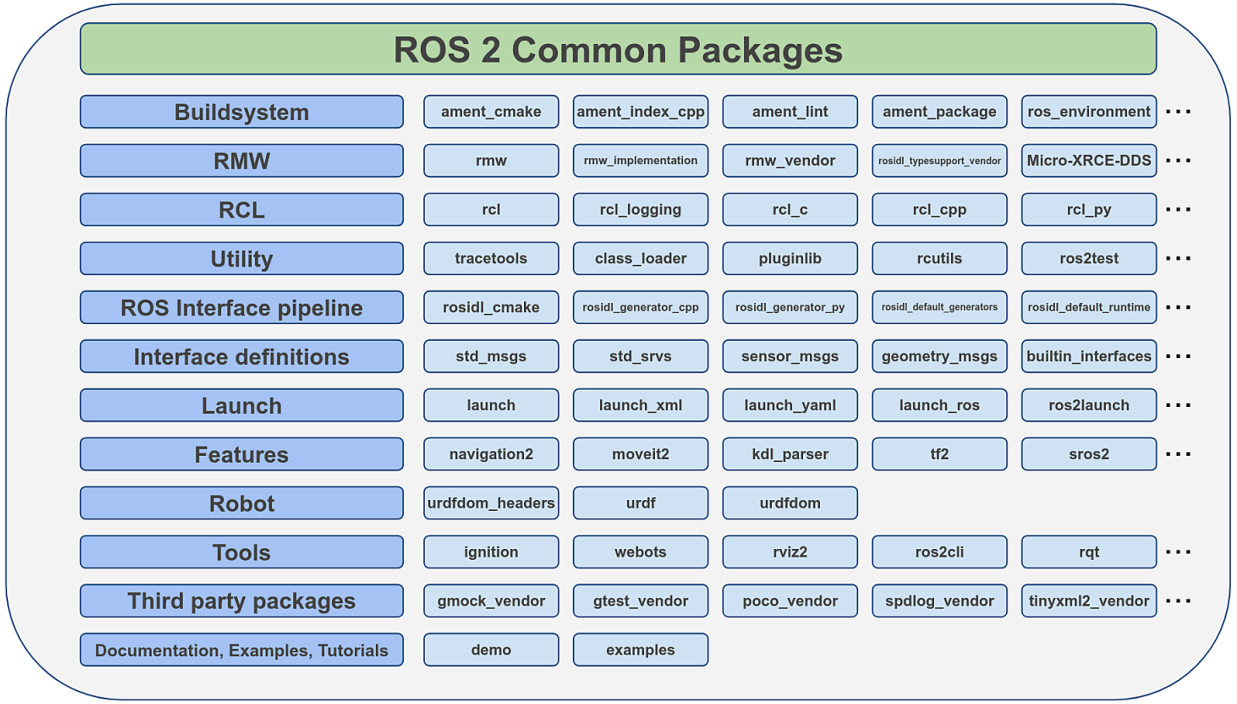
- Robot Operating System
  - 배포 버전, 공통 패키지

List of Distributions

Below is a list of current and historic ROS 2 distributions. Rows in the table marked in green are the currently supported distributions.

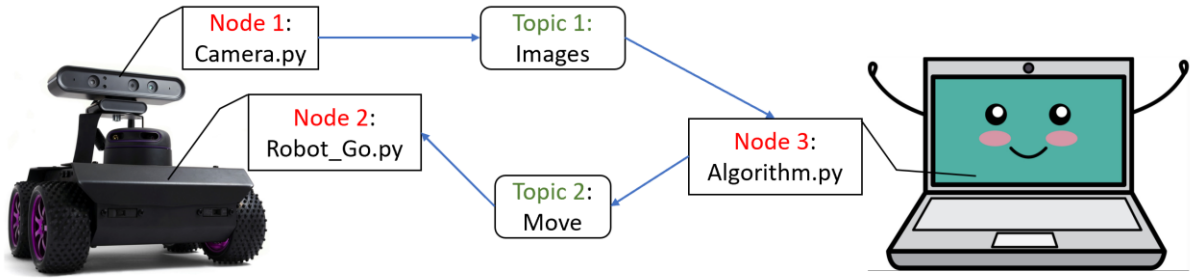
Distro	Release date	Logo	EOL date	ROS Boss
Kilted Kaiju	May 23, 2025		December 2026	Scott K Logan
Jazzy Jalisco	May 23, 2024		May 2029	Marco A. Gutiérrez
Iron Irwini	May 23, 2023		December 4, 2024	Yadunund Vijay
Humble Hawksbill	May 23, 2022		May 2027	Audrow Nash

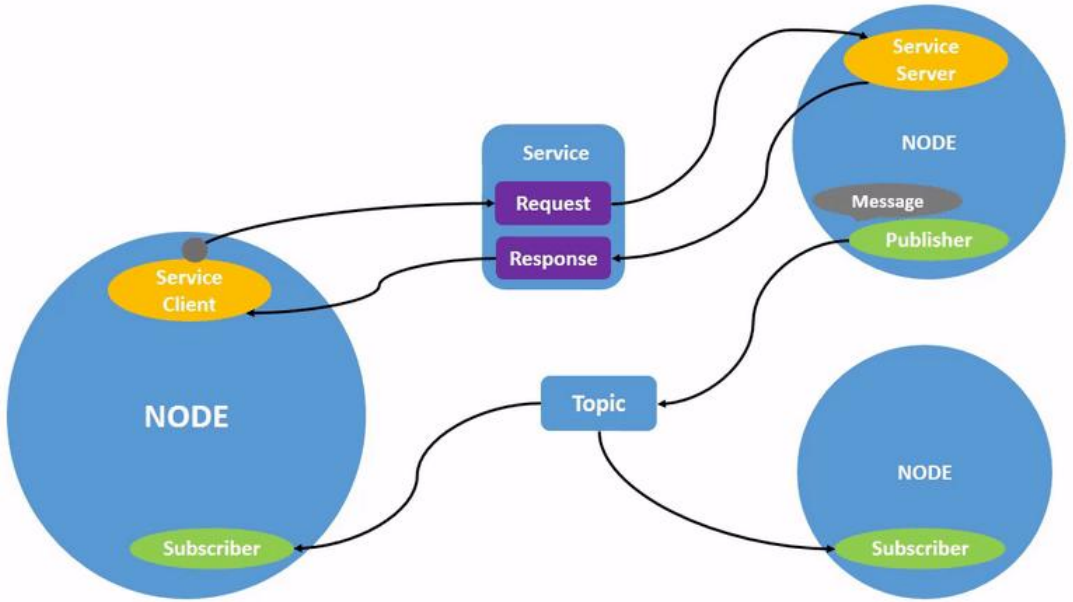
stable version



Node

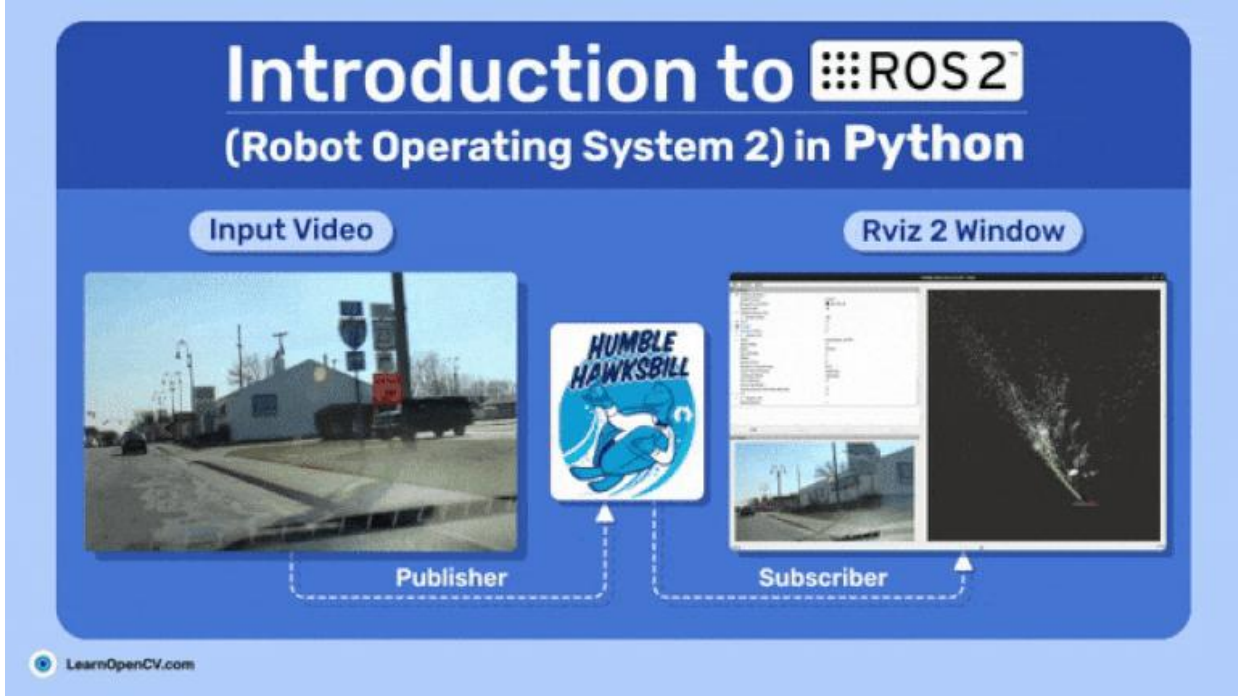
- ROS 통신을 수행하는 하나의 프로세스(실행 파일)
  - Topic, Service, Action, Parameter 등을 통해 Node 간 정보 교환
  - e.g., python scripts, C++ files 등
  - 노드 단위로 분리가 되어있으므로 시스템 충돌이 발생할 경우 다른 노드에 영향 X
    - 코드 복잡성 또한 감소

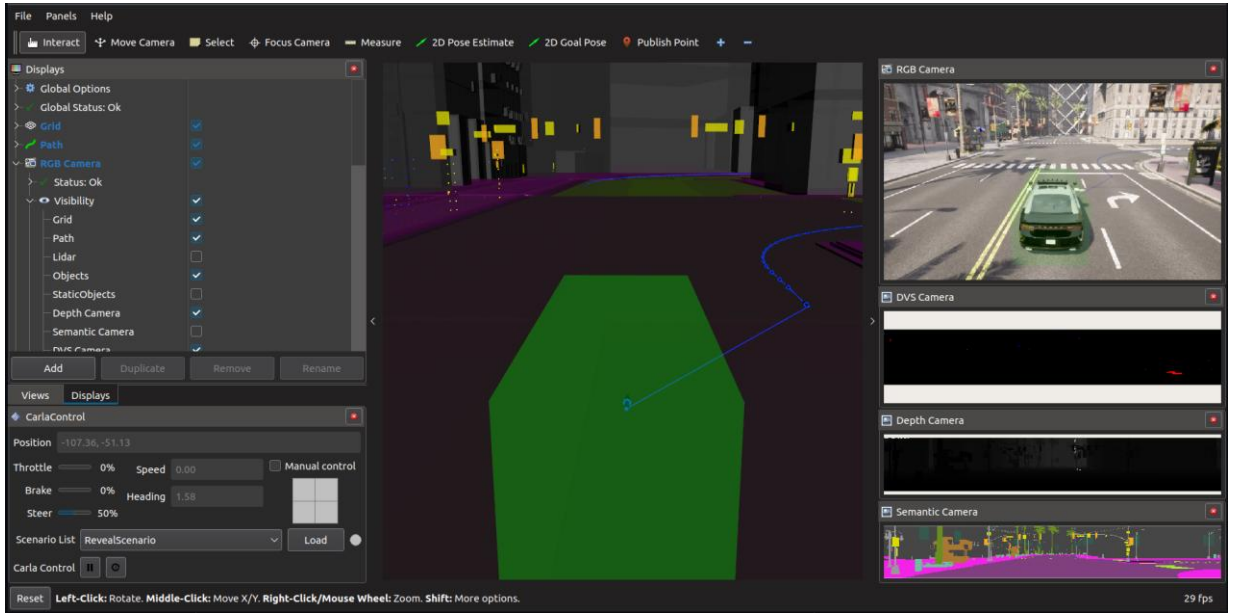




Topic, Service, Action

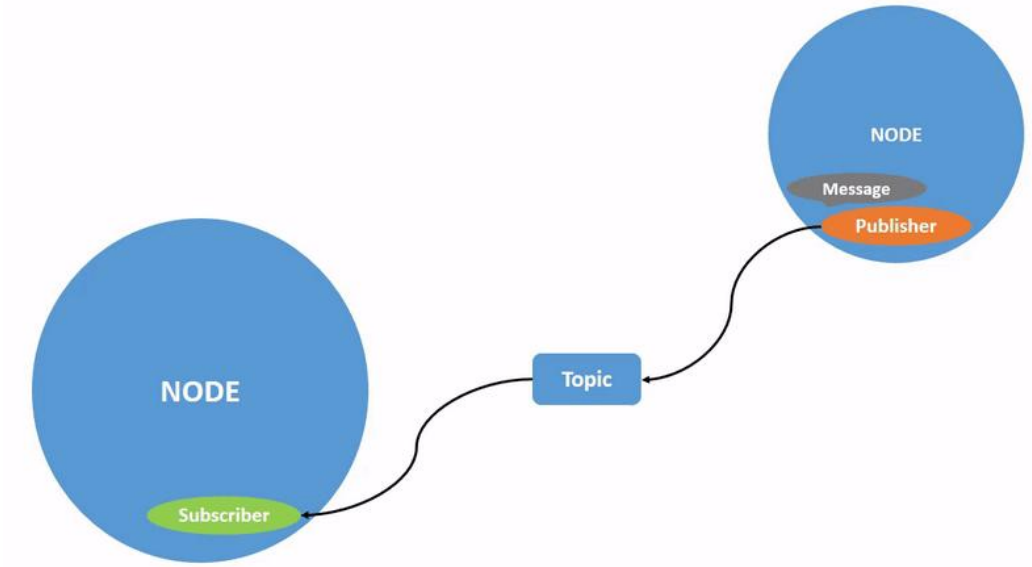
- Topic
  - Continuous data 처리에 사용
    - CAN, Serial 등 통신 및 Camera, LiDAR, GNSS, IMU와 같은 센서 데이터
  - Publisher-Subscriber 형태로 통신
  - Service, Action에 비해 많이 사용

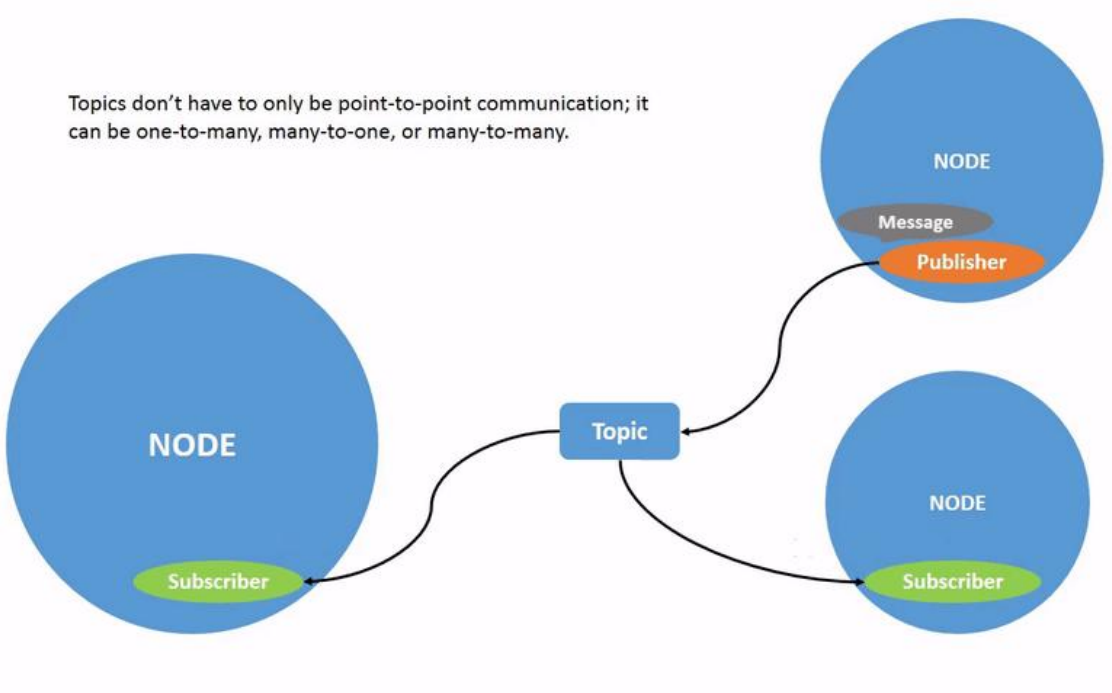




Topic, Service, Action

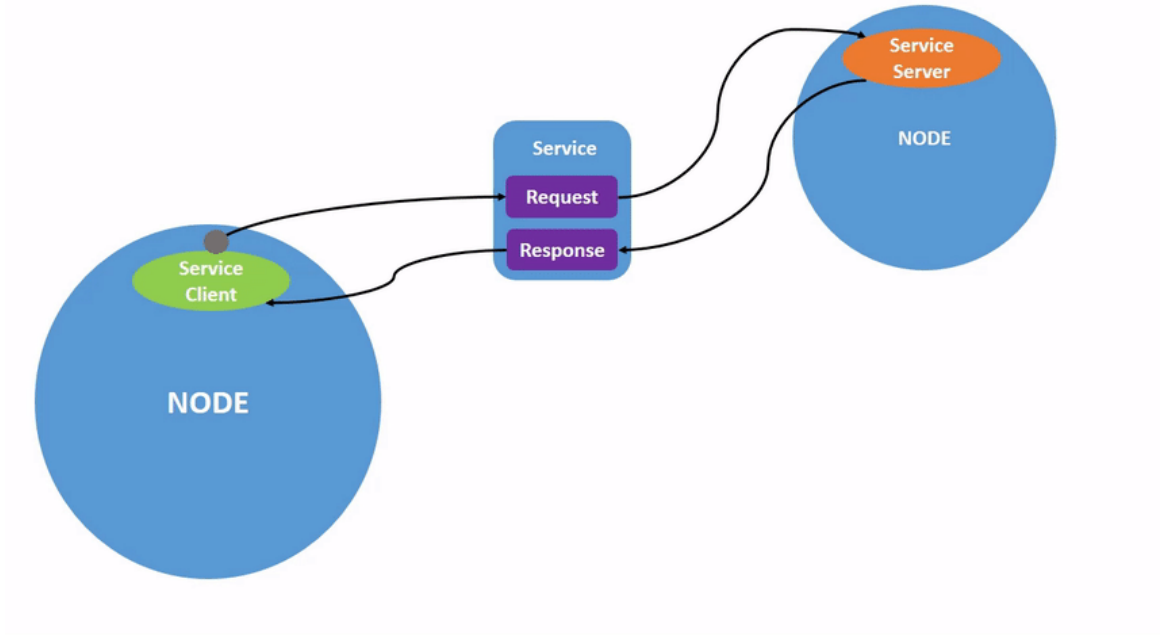
- Topic
  - Publisher-Subscriber는 1:1, 1:N, N:1, N:N 모두 통신 가능

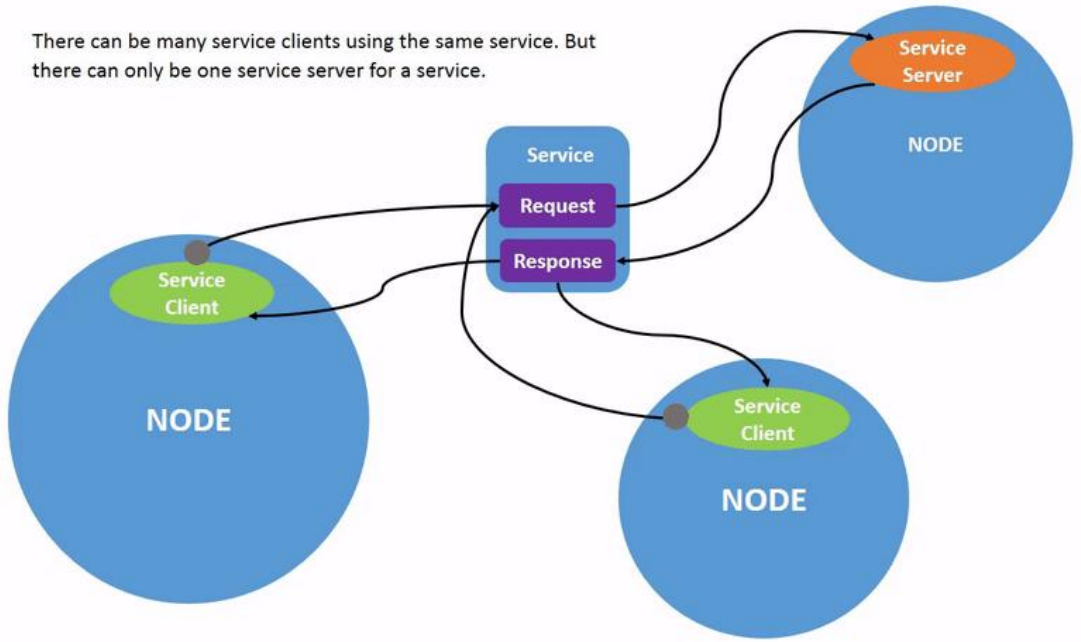




Topic, Service, Action

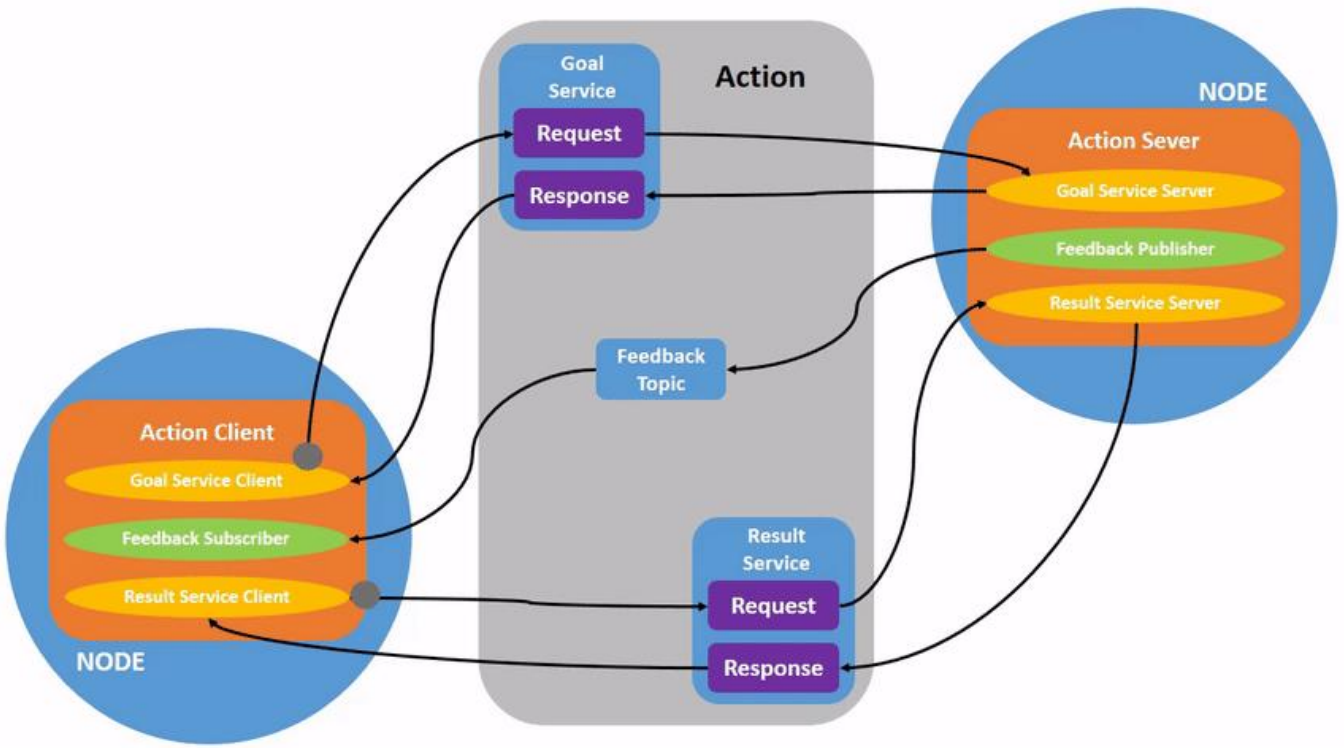
- Service
  - Call:Response 형태로 통신
  - Service Server은 1개, Service Client는 n개 사용 가능
  - Topic은 지속적으로 통신 가능, 서비스는 Client가 호출할 때만 통신 가능





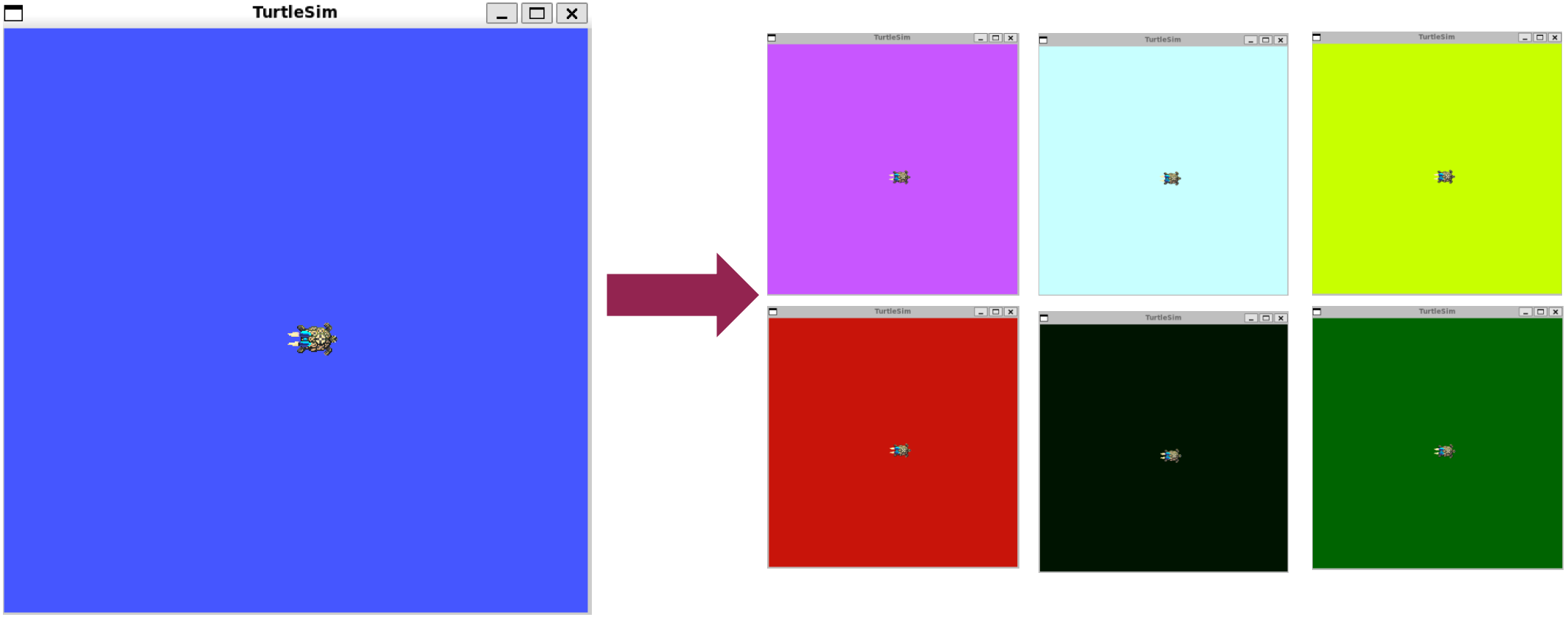
Topic, Service, Action

- Action
  - Topic과 Service 기반의 Long running tasks를 위한 통신
    - Goal, Feedback, Result로 구성
  - Service는 1번의 응답을 하지만, Action은 지속적인 피드백 제공



Parameter

- Node 설정 값으로 Integer, Float, Boolean, String, List 등의 변수를 저장
  - e.g., ROS2 기본 패키지인 turtlesim을 통해 background color parameter를 변경



ROS2 Installation

ROS2 Humble 공식 문서

- https://docs.ros.org/en/humble/index.html
- 2가지 설치 방법 존재 (*Ubuntu 22.04 기준)
  - **Debian Package(추천)**
    - **https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html**
  - Binary Archive

ROS2 설치

- 1. Locale 설정(UTF-8)

```
$ locale
$ sudo apt update && sudo apt install locales
$ sudo locale-gen en_US en_US.UTF-8
$ sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
$ export LANG=en_US.UTF-8
$ locale
```

ROS2 Installation

▶ ROS2 설치

- 2. ROS2 apt repository 추가

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
$ sudo apt update && sudo apt install curl -y
$ export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F\" '{print $4}')
$ curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"
$ sudo dpkg -i /tmp/ros2-apt-source.deb
```

- 3. ROS2 패키지 설치

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install ros-humble-desktop
$ gedit ~/.bashrc
   - source /opt/ros/humble/setup.bash ← 최하단 추가
$ source ~/.bashrc
```

ROS2 Installation

▶ ROS2 설치

- 4. 커스텀 작업 폴더 추가 & 예제 실행

```
$ cd ~ && mkdir -p lecture_ws/src
$ cd lecture_ws && colcon build
  ㄴ colcon command not found error
      - $ sudo apt install python3-colcon-common-extensions
$ gedit ~/.bashrc
  - source ~/lecture_ws/install/setup.bash ← 최하단 추가
$ source ~/.bashrc
$ cd src # cd ~/lecture_ws/src
```

```
(terminal 1) $ ros2 run demo_nodes_cpp talker
(terminal 2) $ ros2 run demo_nodes_py listener
(terminal 3) $ ros2 topic list # 토픽 리스트 확인(/chatter)
                $ ros2 topic echo /chatter

※Output
data: 'Hello World: (Number)'
---
```

▶ ROS2 기능 실습(Turtlesim Package)

- Turtlesim Package 실행

```
(terminal 1) $ ros2 pkg executables turtlesim # turtlesim 관련 패키지 목록 확인
(terminal 1) $ ros2 run turtlesim turtlesim_node # turtlesim 실행
(terminal 2) $ ros2 run turtlesim turtle_teleop_key # turtlesim 키보드 조작
```

- Node, Topic, Service, Action, Parameter 확인(list)

```
(terminal 3) $ ros2 node list
※Output
/teleop_turtle
/turtlesim
```

```
(terminal 3) $ ros2 topic list
※Output
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

```
(terminal 3) $ ros2 service list
※Output
/clear
/kill
/reset
/spawn
...
/turtlesim/set_parameters_atomically
```

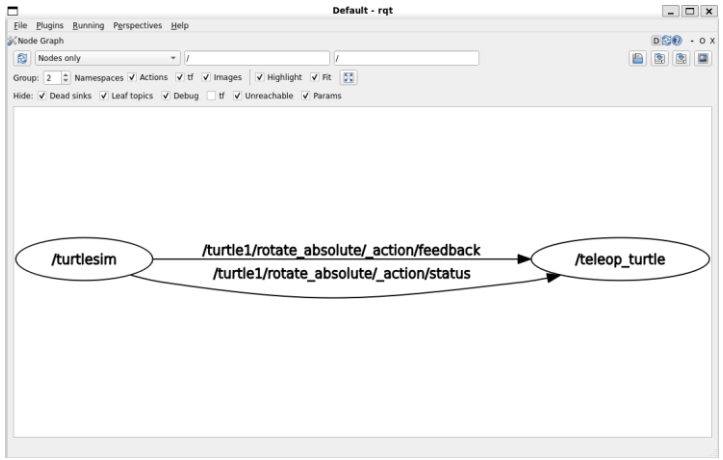
```
(terminal 3) $ ros2 action list
※Output
/turtle1/rotate_absolute
```

```
(terminal 3) $ ros2 param list
※Output
/teleop_turtle:
  qos_overrides
  ...
  use_sim_time
/turtlesim:
  background_b
  ...
  use_sim_time
```

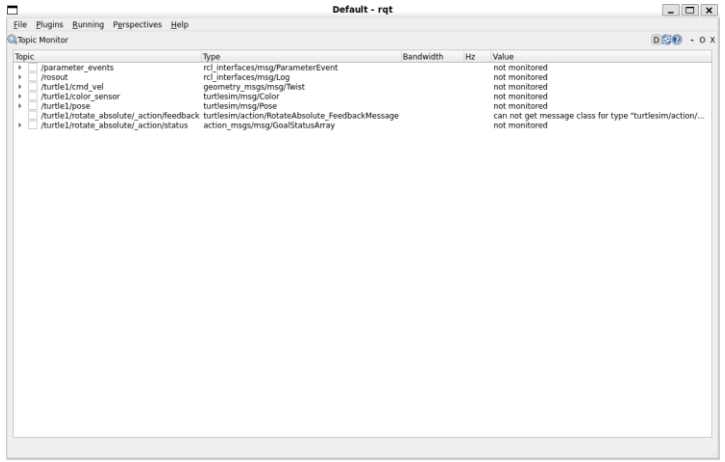
ROS2 기능 실습(Turtlesim Package)

- rqt

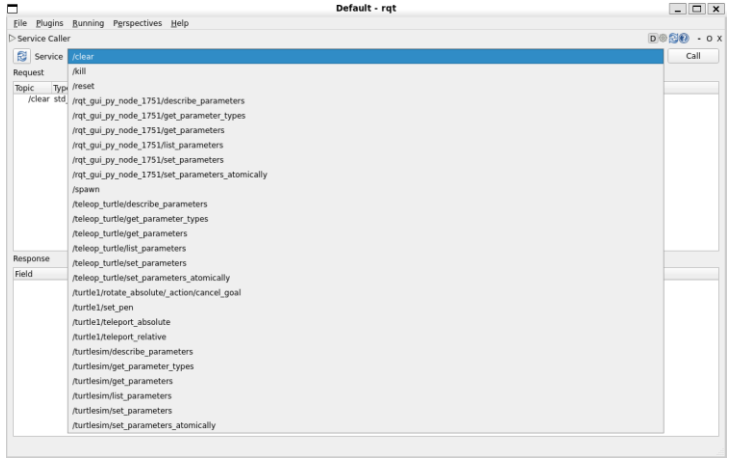
```
(terminal 3) $ rqt
```



Node Graph



Topic Monitor

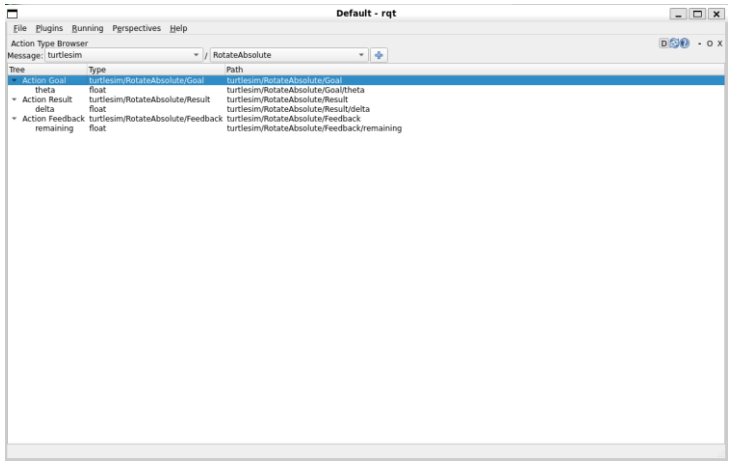


Service Caller

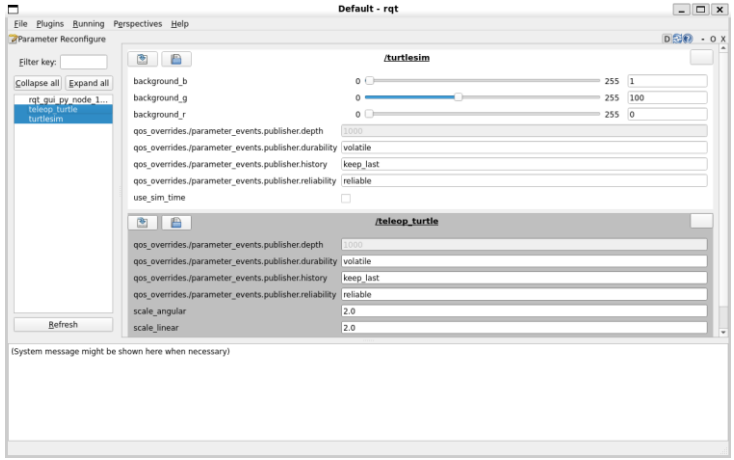
ROS2 기능 실습(Turtlesim Package)

- rqt

```
(terminal 3) $ rqt
```



Action Type Browser



Dynamic Reconfigure

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Topic

```
(terminal 3) $ ros2 topic echo /turtle1/pose
# Output
---
x: 6.5524444580078125
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
```

```
(terminal 3) $ ros2 topic info /turtle1/pose
# Output
Type: turtlesim/msg/Pose
Publisher count: 1
Subscription count: 0
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Topic

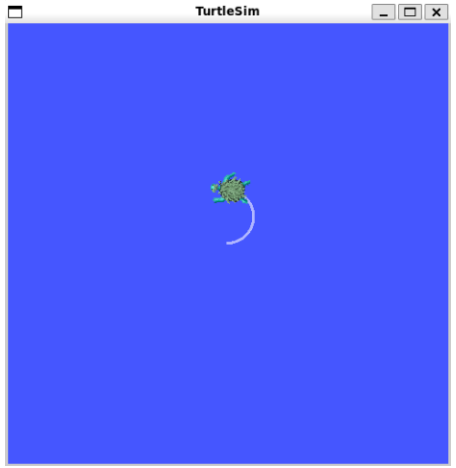
```
(terminal 3) $ ros2 interface show turtlesim/msg/Pose
# Output
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
```

```
(terminal 3) $ ros2 topic hz /turtle1/pose
# Output
average rate: 62.540
        min: 0.015s max: 0.016s std dev: 0.00045s window: 64
average rate: 62.499
        min: 0.015s max: 0.017s std dev: 0.00046s window: 127
average rate: 62.502
        min: 0.015s max: 0.017s std dev: 0.00045s window: 190
```

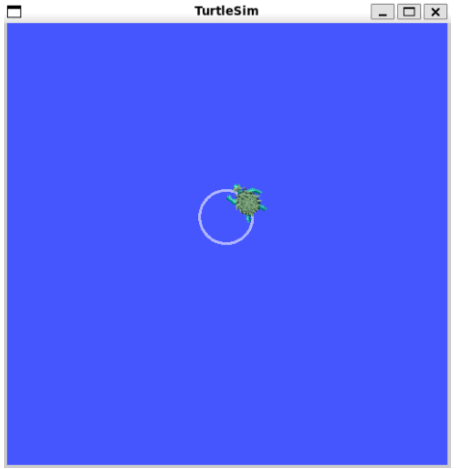
ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Topic

```
(terminal 3) $  ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist '{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 3.0}}' # 1 번 실행
# Output
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=3.0))
```




```
(terminal 3) $ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist '{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 3.0}}' # 1초당 1번 실행
# Output
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=3.0))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=3.0))
```



ROS2 기능 실습(Turtlesim Package)

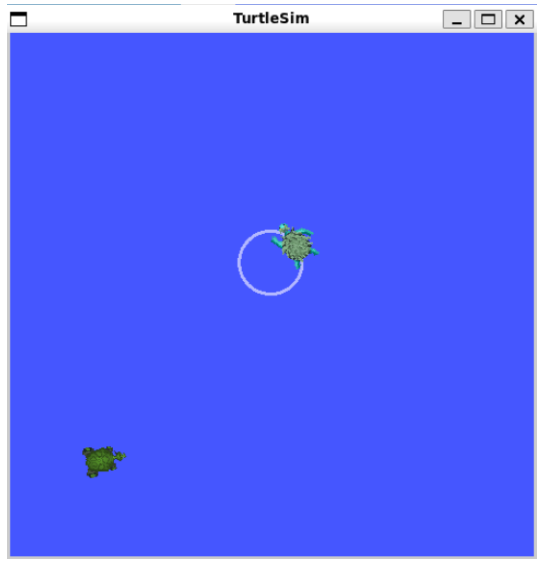
- CLI(Command Line Interface)
  - Service

```
(terminal 3) $ ros2 service type /spawn
# Output
turtlesim/srv/Spawn
(terminal 3) $ ros2 service list -t
# Output
/clear [std_srvs/srv/Empty]
/kill [turtlesim/srv/Kill]
/reset [std_srvs/srv/Empty]
/spawn [turtlesim/srv/Spawn]
...
(terminal 3) $ ros2 service find turtlesim/srv/Spawn
# Output
/spawn
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Service

```
(terminal 3) $ ros2 interface show turtlesim/srv/Spawn
# Output
float32 x
float32 y
float32 theta
string name # Optional.  A unique name will be created and returned if this is empty
---
string name
(terminal 3) $ ros2 service call /spawn turtlesim/srv/Spawn '{x: 2, y: 2, theta: 0.2, name: ''}'
# Output
requester: making request: turtlesim.srv.Spawn_Request(x=2.0, y=2.0, theta=0.2, name='None')

response:
turtlesim.srv.Spawn_Response(name='')
```



ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Action

```
(terminal 3) $ ros2 node info /turtlesim
# Output
/turtlesim
...
Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
 Action Clients:
```

```
(terminal 3) $ ros2 node info /teleop_turtle
# Output
 Action Servers:

 Action Clients:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Action

```
(terminal 3) $ ros2 action list
# Output
/turtle1/rotate_absolute
(terminal 3) $ ros2 action list -t
# Output
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

```
(terminal 3) $ ros2 action info /turtle1/rotate_absolute
# Output
Action: /turtle1/rotate_absolute
Action clients: 1
   /teleop_turtle
Action servers: 1
   /turtlesim
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Action

```
(terminal 3) $ ros2 interface show turtlesim/action/RotateAbsolute
# Output
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Action

```
(terminal 3) $ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute '{theta: 1.57}'
# Output
Waiting for an action server to become available...
Sending goal:
    theta: 1.57
Goal accepted with ID: 54c1d7bc5309471096924d0f904ee3f8
Result:
   delta: 0.0
Goal finished with status: SUCCEEDED
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Action

```
(terminal 3) $ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute '{theta: -1.57}' --feedback
# Output
Sending goal:
   theta: -1.57
Goal accepted with ID: e6092c831f994afda92f0086f220da27
Feedback:
  remaining: -3.1268222332000732
...
Feedback:
   remaining: -0.01644420623779297
Result:
   delta: 3.1040000915527344

Goal finished with status: SUCCEEDED
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Parameter

```
(terminal 3) $ ros2 param list
# Output
/teleop_turtle:
  scale_angular
  scale_linear
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  use_sim_time
```

```
(terminal 3) $ ros2 param get /turtlesim background_b
# Output
Integer value is: 255
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Parameter

```
(terminal 3) $ ros2 param set /turtlesim background_b 0
# Output
Set parameter successful
(terminal 3) $ ros2 param dump /turtlesim # Parameter save
# Output
Saving to:  ./turtlesim.yaml
(In turtlesim.yaml)
turtlesim:
  ros__parameters:
    background_b: 255
    background_g: 86
    background_r: 150
    use_sim_time: false
```

ROS2 기능 실습(Turtlesim Package)

- CLI(Command Line Interface)
  - Parameter

```
(terminal 3) $ ros2 param load /turtlesim ./turtlesim.yaml
# Output
Set parameter background_b successful
Set parameter background_g successful
Set parameter background_r successful
Set parameter use_sim_time successful
(optional) (terminal 3) $ ros2 run turtlesim turtlesim_node --ros-args --params-file ./turtlesim.yaml
```

ROS2 기능 실습(Ros2 bag)

- Bag
  - Topic 데이터를 저장하여 db에 기록하는 명령어
    - 특정 시점, 공간의 반복적 실험이 필요할 때 유용
      - record: 녹화, info: .bag 파일 정보 확인

```
(terminal 3) $ cd ~/ && mkdir logging_data && cd logging_data # Make logging folder
             $ ros2 bag record /turtle1/cmd_vel
# Output
[INFO] [1761487288.804753988] [rosbag2_recorder]: Press SPACE for pausing/resuming
[INFO] [1761487288.807635980] [rosbag2_storage]: Opened database 'rosbag2_2025_xx_xx-xx_xx_xx/rosbag2_2025_xx_xx-xx_xx_xx_0.db3' for READ_WRITE.
(optional) (terminal 3) $ ros2 bag record -o [name] /topic_data /topic_data2 # edit name of bag file
(terminal 3) $ ros2 bag info rosbag2_2025_xx_xx-xx_xx_xx/
# Output
Duration:       ...
Start:          ...
End:            ...
Messages:       197
Topic information: Topic: /turtle1/cmd_vel | Type: geometry_msgs/msg/Twist | Count: 197 |
```

ROS2 기능 실습(Ros2 bag)

- Bag
  - Topic 데이터를 저장하여 db에 기록하는 명령어
    - 특정 시점, 공간의 반복적 실험이 필요할 때 유용
      - play: .bag 파일 재생

```
(terminal 3) $ ros2 bag play rosbag2_2025_xx_xx-xx_xx_xx
# Output
[INFO] [1761488083.637960742] [rosbag2_storage]: Opened database 'rosbag2_2025_xx_xx-xx_xx_xx/rosbag2_2025_xx_xx-xx_xx_xx_0.db3' for READ_ONLY.
[INFO] [1761488083.638010783] [rosbag2_player]: Set rate to 1
[INFO] [1761488083.642512680] [rosbag2_player]: Adding keyboard callbacks.
[INFO] [1761488083.642546873] [rosbag2_player]: Press SPACE for Pause/Resume
[INFO] [1761488083.642551462] [rosbag2_player]: Press CURSOR_RIGHT for Play Next Message
[INFO] [1761488083.642565011] [rosbag2_player]: Press CURSOR_UP for Increase Rate 10%
[INFO] [1761488083.642579062] [rosbag2_player]: Press CURSOR_DOWN for Decrease Rate 10%
```

ROS2 기능 실습(Create a Custom Package)

- ROS2 Humble Custom Package Docs
  - https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html
- Colcon
  - ROS2의 Build command로, 멀티 패키지 빌드 및 테스트 통합 도구
    - Workspace 내 build, install, log 폴더 생성
    - 백엔드로 ament 호출(ament_cmake, ament_python)
      - ROS1 - catkin(CMake only) -> ROS2 - ament(CMake+Python setuptools)


```
(In workspace) $ colcon build --symlink-install # (Build packages (symlinked))
               $ source install install/setup.bash # (Load the Built Workspace)
```

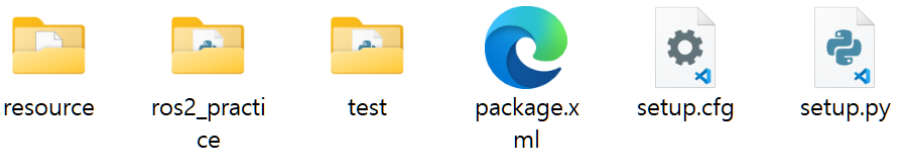
ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)

```
(In workspace/src) $ ros2 pkg create --build-type ament_python ros2_practice
# Output
going to create a new package
package name: ros2_practice
destination directory: {$HOME}/lecture_ws/src
...
MIT-0
$ cd ../ && colcon build --symlink-install
```

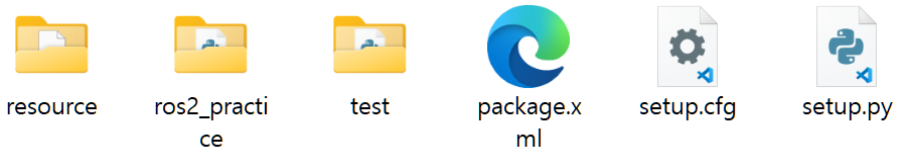
```
kys@kys:~/colcon_ws/src/ros2_practice$ tree
.
├── package.xml
├── resource
│   └── ros2_practice
├── ros2_practice
│   └── __init__.py
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py

3 directories, 8 files
```

– 폴더 구조



ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)
  - 폴더 구조



  - resource: 패키지 존재를 표시하는 marker 파일(share directory indexing)
  - ros2_practice(package_name): 실제 python 코드 경로
  - test: pytest or lint 테스트 폴더
  - package: ROS Metadata, dependency 파일
  - setup.cfg / setup.py: setuptools를 통한 패키지 빌드 설정 파일

ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)
  - Publisher(차량의 속도를 m/s로 발행하는 예제)

```python
#!/usr/bin/env python3
import random
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

class VehicleSpeedPublisher(Node):
    def __init__(self):
        super().__init__('vehicle_speed_publisher')
        self.pub = self.create_publisher(Float32, '/vehicle/speed_mps', 10)
        self.timer = self.create_timer(0.05, self.on_timer)
        self._v = 0.0
    def on_timer(self):
        self._v += random.uniform(-0.08, 0.1)
        self._v = max(0.0, min(self._v, 16.7))
        msg = Float32(data=self._v)
        self.pub.publish(msg)
def main():
    rclpy.init()
    node = VehicleSpeedPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)
  - Subscriber(들어오는 차량의 속도 데이터(m/s)를 km/h로 변환하여 표시하는 예제)

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

MPS_TO_KMPH = 3.6

class CarSpeedSubscriber(Node):
    def __init__(self):
        super().__init__('vehicle_speed_subscriber')
        self.sub = self.create_subscription(Float32, '/vehicle/speed_mps',
self.on_msg, 10)

    def on_msg(self, msg: Float32):
        kmh = msg.data * MPS_TO_KMPH
        print(f'Current Vehicle Speed: {kmh:.1f} km/h')
def main():
    rclpy.init()
    node = CarSpeedSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)
  - 패키지 설정
    - 1. setup.py 내 entry_points에 등록

```
$ cd ~/lecture_ws/src/ros2_practice/ && gedit setup.py (or code .)
# Publisher, Subscriber node 추가
entry_points={
    'console_scripts':
    [
    'vehicle_speed_publisher = ros2_practice.vehicle_speed_publisher:main',
    'vehicle_speed_subscriber  = ros2_practice.vehicle_speed_subscriber:main',
    ],
```

```
entry_points={
    'console_scripts':
    [
    'vehicle_speed_publisher = ros2_practice.vehicle_speed_publisher:main',
    'vehicle_speed_subscriber = ros2_practice.vehicle_speed_subscriber:main',
    ],
```

ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)
  - 패키지 설정
    - 2. package.xml 내 ros2 dependency 추가

```
$ cd ~/lecture_ws/src/ros2_practice/ && gedit setup.py (or code .)
# rclpy(ros2 python client library), std_msgs(int, float, Boolean ...) 추가
 <exec_depend>rclpy</exec_depend>
 <exec_depend>std_msgs</exec_depend>
```

```
  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>
```

ROS2 기능 실습(Create a Custom Package)

- Custom Package(Python)
  - 빌드 및 실행

```
$ cd ~/lecture_ws && colcon build --symlink-install
$ source install/setup.bash
(terminal 1) $ ros2 run ros2_practice vehicle_speed_publisher
(terminal 2) $ ros2 run ros2_practice vehicle_speed_subscriber
(terminal 3) $ ros2 topic echo /vehicle/speed_mps
```

```
kys@kys:~/colcon_ws/src/ros2_practice$ ros2 run ros2_practice vehicle_speed_subscriber
Current Vehicle Speed: 0.5 km/h
Current Vehicle Speed: 0.6 km/h
Current Vehicle Speed: 0.8 km/h
Current Vehicle Speed: 0.9 km/h
Current Vehicle Speed: 0.8 km/h
Current Vehicle Speed: 1.0 km/h
Current Vehicle Speed: 1.1 km/h
Current Vehicle Speed: 1.4 km/h
Current Vehicle Speed: 1.1 km/h
Current Vehicle Speed: 1.0 km/h
Current Vehicle Speed: 1.2 km/h
```

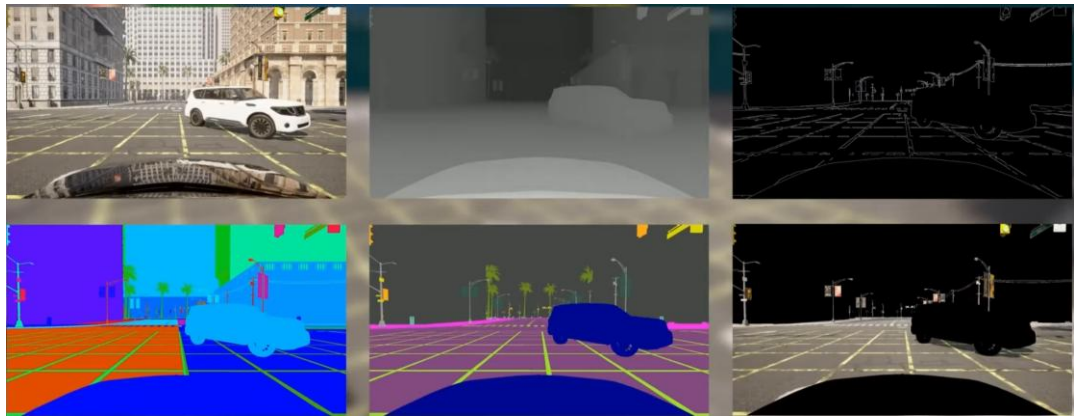
```
kys@kys:~/colcon_ws/src/ros2_practice$ ros2 topic echo /vehicle/speed_mps
data: 1.265470027923584
---
data: 1.2761770486831665
---
data: 1.3685742616653442
---
data: 1.3927221298217773
---
data: 1.367760181427002
---
data: 1.3335561752319336
---
data: 1.3934508562088013
---
data: 1.422240972518921
---
data: 1.3604389429092407
---
data: 1.3405848741531372
---
data: 1.3970544338226318
---
data: 1.355398416519165
---
data: 1.41675865650177
---
```

CARLA

CARLA

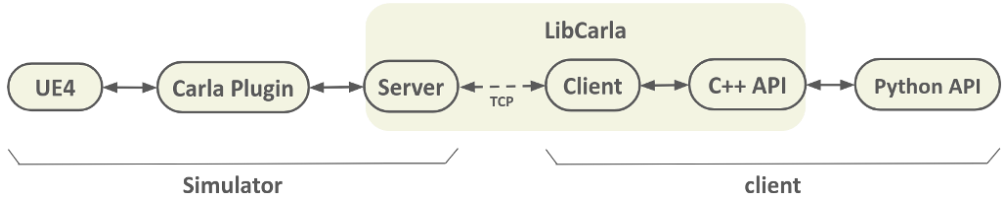
CARLA(Car Learning to Act) 개요

- Unreal Engine 기반 자율주행 오픈소스 시뮬레이터
  - 실습 버전 0.9.16 기준 UE4 (0.10.0 기준 UE5)
  - 실제 도로, 교통, 센서 등의 물리적 특성을 가상으로 재현
    - Camera, LiDAR, RADAR, GNSS, IMU 등
  - Python API를 통한 시나리오 제어 및 자동화
  - 대규모 맵, 차량, 보행자 assets 지원



CARLA 구조

- Server-Client Architecture
  - TCP 통신, Asynchronous 모드
  - 월드-맵-액터(Vehicle/Pedestrian/Sensor) 계층 구조
  - Traffic Manager/Scenario Runner 기반 시나리오 실행 구조
  - 좌표계, 타임스탬프, 날씨, 조명 파라미터화



▶ 권장 사양

항목	권장 사양
OS	Ubuntu 20.04, 22.04
GPU	RTX 2070 이상
CPU	i7 9~11세대, Ryzen 7 or 9
Disk Space	100GB 이상
RAM	32GB 이상
Python Version	3.8 이상

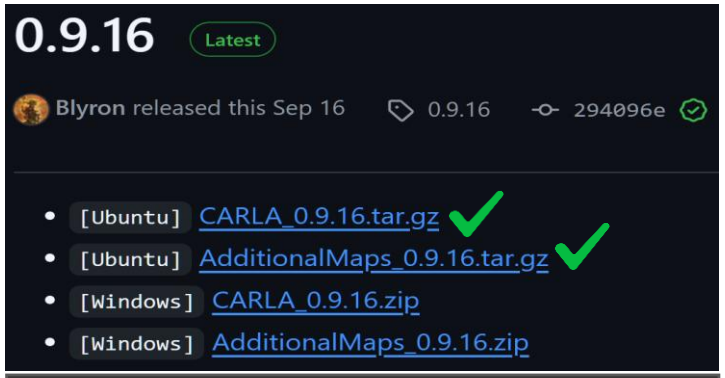
CARLA Installation

CARLA 공식 문서

- https://carla.readthedocs.io/en/0.9.16
- 2가지 설치 방법 존재
  - **Github 다운로드**
  - 소스 빌드**(Unreal Engine Editor 사용 가능)**

CARLA (Package Installation)

- Package 다운로드 및 실행(~22GB)
  - https://github.com/carla-simulator/carla/releases/tag/0.9.16/




```
$ mkdir carla
$ tar -xvzf CARLA_0.9.16.tar.gz -C ~/carla
$ cd carla && ./CarlaUE4.sh
```

CARLA Installation

▶ CARLA (Source Build) (해당 방법으로 설치 X)

- 1. S/W 업데이트 및 설치

```
$ sudo apt update
$ sudo apt install -y build-essential g++-12 cmake ninja-build libvulkan1 python3 python3-dev python3-pip python3-venv autoconf wget curl rsync unzip git git-lfs libpng-dev libtiff5-dev libjpeg-dev
```

- 2. EPIC GAMES-Github 계정 연동 & Unreal Engine 4.26 설치
  - https://www.unrealengine.com/en-US/ue-on-github

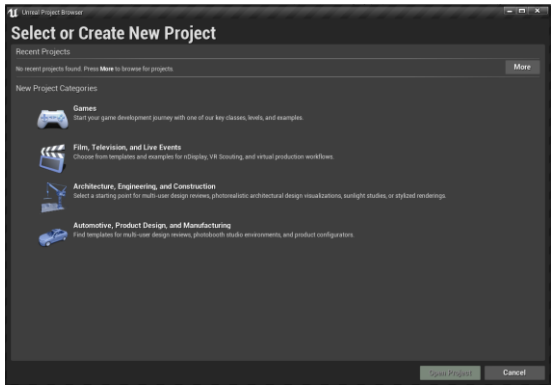
```
$ git clone --depth 1 -b carla https://oauth2:TOKEN@github.com/CarlaUnreal/UnrealEngine.git ~/UnrealEngine_4.26
$ cd ~/UnrealEngine_4.26
$ ./Setup.sh && ./GenerateProjectFiles.sh && make
```

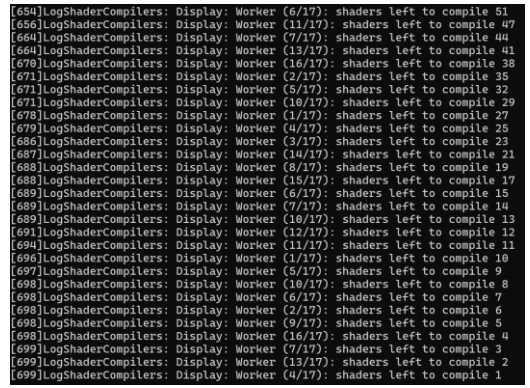
CARLA Installation

▶ CARLA (Source Build) (해당 방법으로 설치 X)

- 3. Unreal Engine 설치 확인 & 환경 변수 설정

```
$ cd ~/UnrealEngine_4.26/Engine/Binaries/Linux && ./UE4Editor
$ gedit ~/.bashrc
  - export UE4_ROOT=~/UnrealEngine_4.26
$ source ~/.bashrc
```





- 4. CARLA 빌드

```
$ sudo apt install aria2
$ cd ~ && git clone -b ue4/0.9.16 https://github.com/carla-simulator/carla
$ gedit ~/.bashrc
  - export CARLA_UE4_ROOT=~/carla
$ source ~/.bashrc
$ cd carla && ./Update.sh
$ pip install -U pip setuptools
$ python3 -m pip install -U -r PythonAPI/carla/requirements.txt
$ make PythonAPI
```

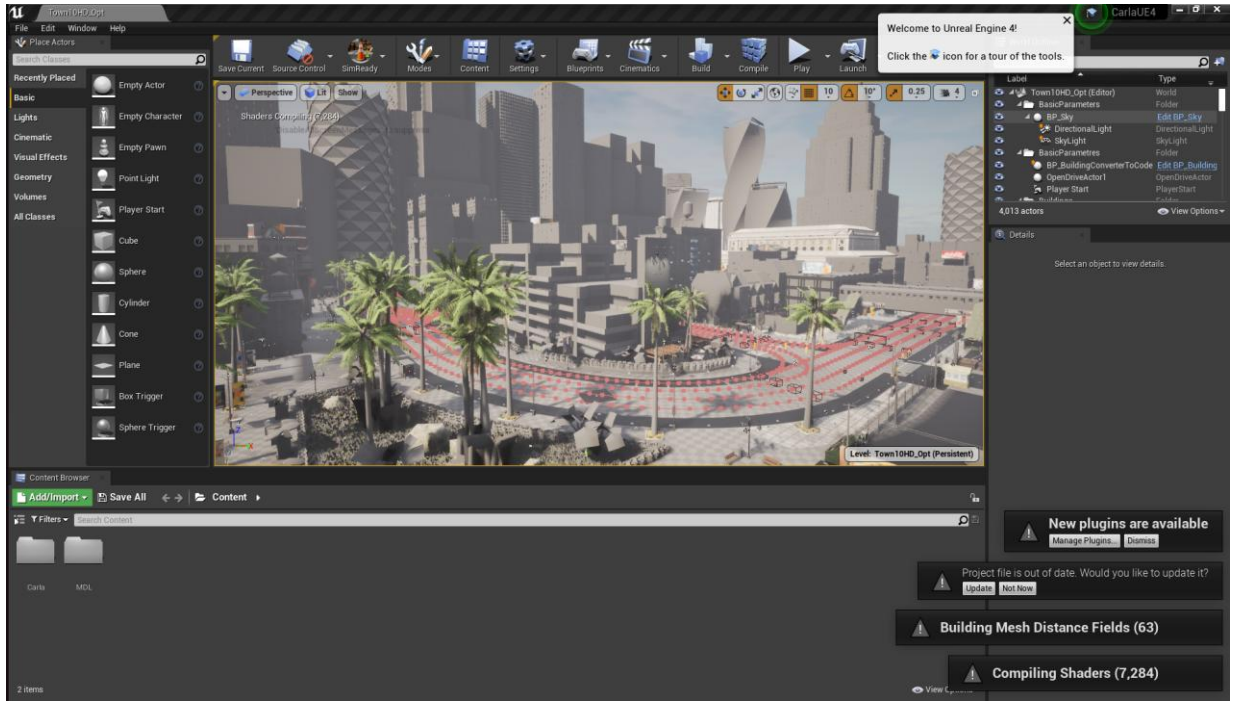
```
adding 'carla/__init__.py'
adding 'carla/command.pyi'
adding 'carla/libcarla.cpython-310-x86_64-linux-gnu.so'
adding 'carla/libcarla.pyi'
adding 'carla-0.9.16.dist-info/METADATA'
adding 'carla-0.9.16.dist-info/WHEEL'
adding 'carla-0.9.16.dist-info/top_level.txt'
adding 'carla-0.9.16.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built carla-0.9.16-cp310-cp310-linux_x86_64.whl
BuildPythonAPI.sh: Installing Python API for Python 3.
Defaulting to user installation because normal site-packages is not writeable
Processing ./dist/.tmp/carla-0.9.16-cp310-cp310-linux_x86_64.whl
Installing collected packages: carla
Successfully installed carla-0.9.16
BuildPythonAPI.sh: Success!
```

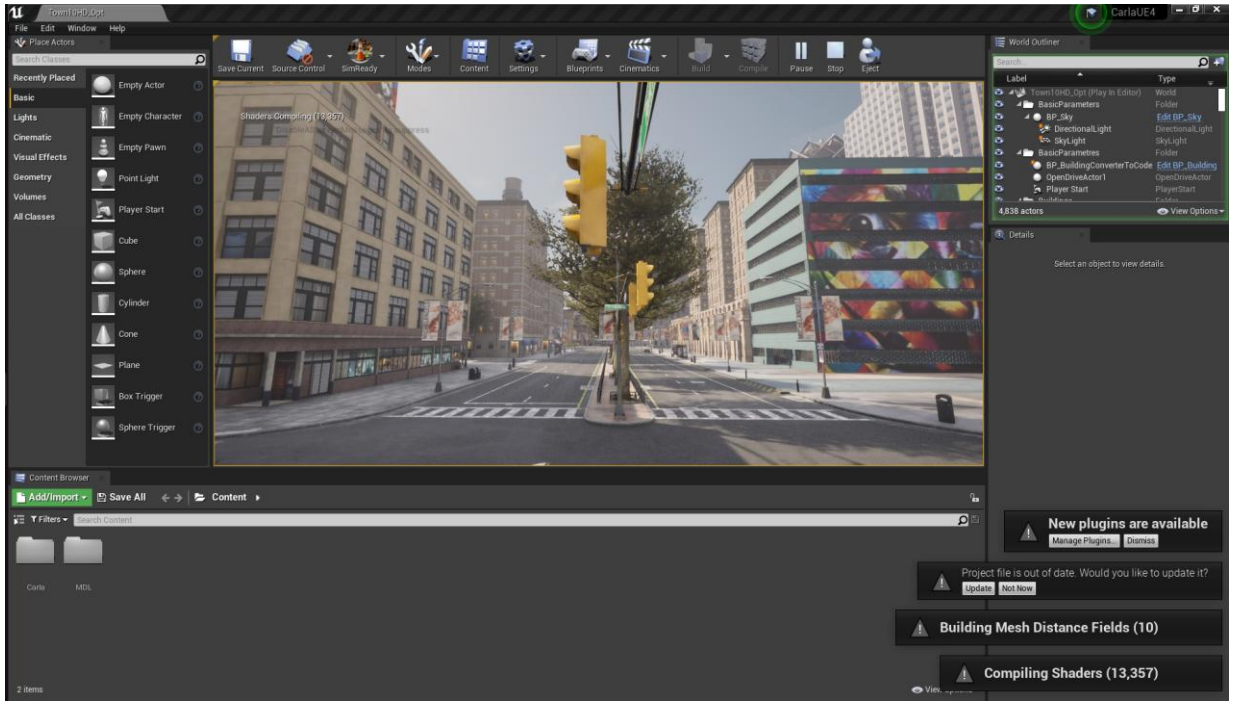
CARLA Installation

▶ CARLA (Source Build) (해당 방법으로 설치 X)

- 5. Unreal Engine Editor 컴파일

```
$ make launch
(In Unreal Engine) 'Play' 클릭
```





CARLA practice

▶ CARLA 기능 실습

- Map(Town01)

CARLA practice

▶ CARLA 기능 실습

- Map(Town02)

CARLA practice

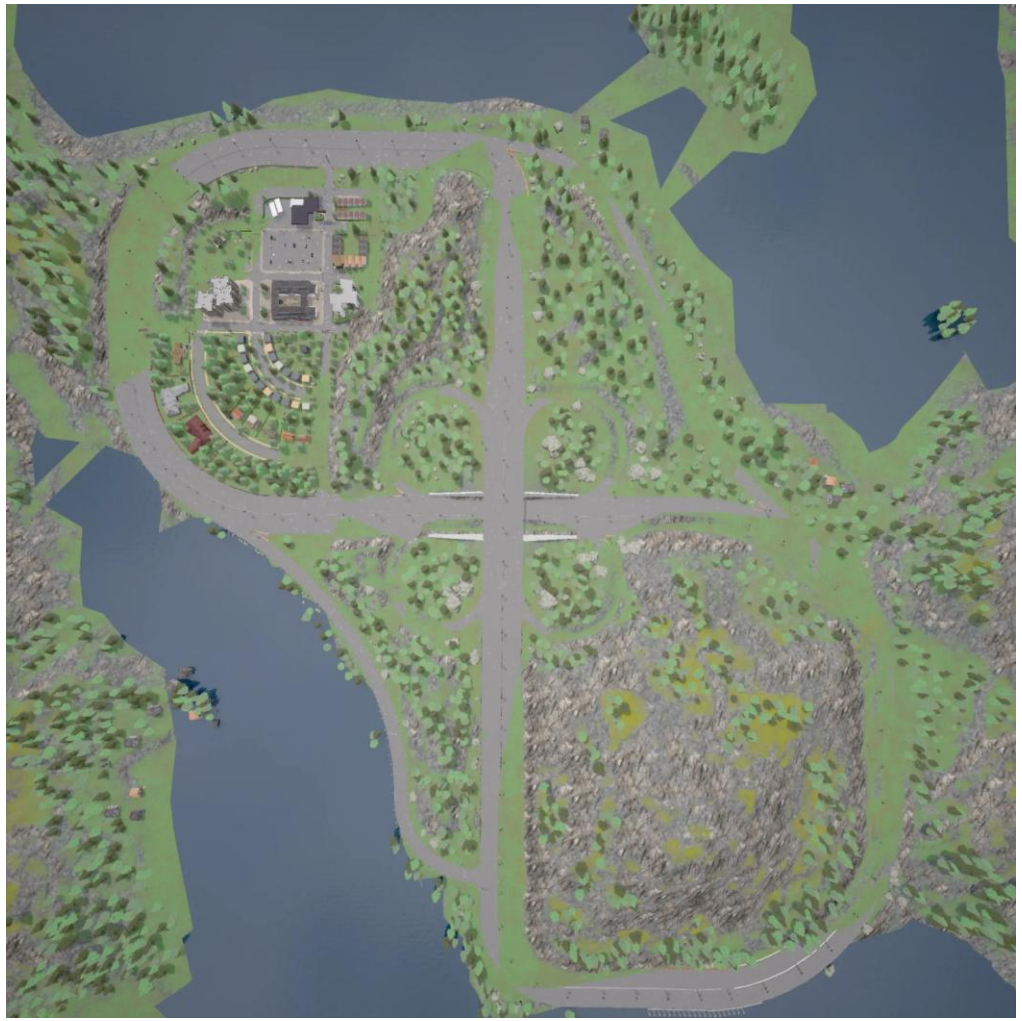
▶ CARLA 기능 실습

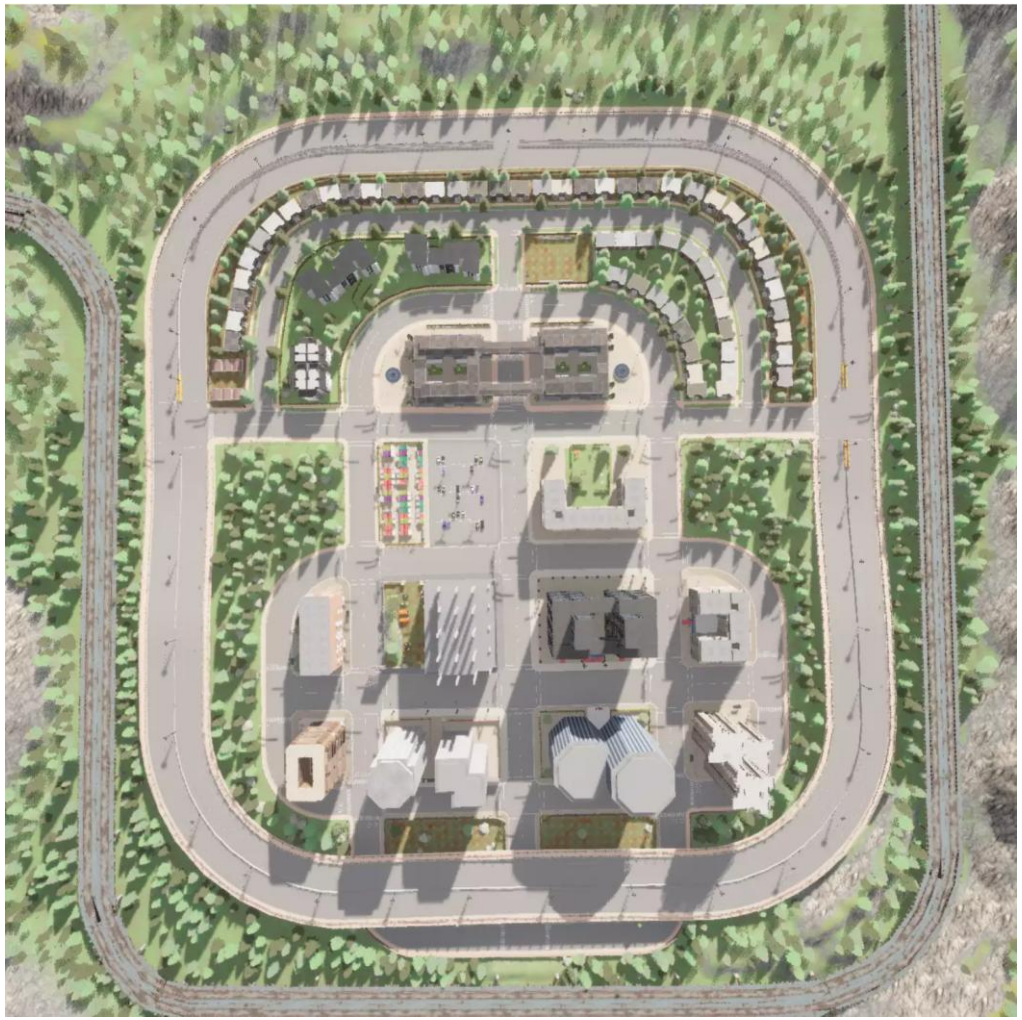
- Map(Town03)

CARLA practice

▶ CARLA 기능 실습

- Map(Town04, Town05)





CARLA 기능 실습

- Map(Town06, **Additional Map**)

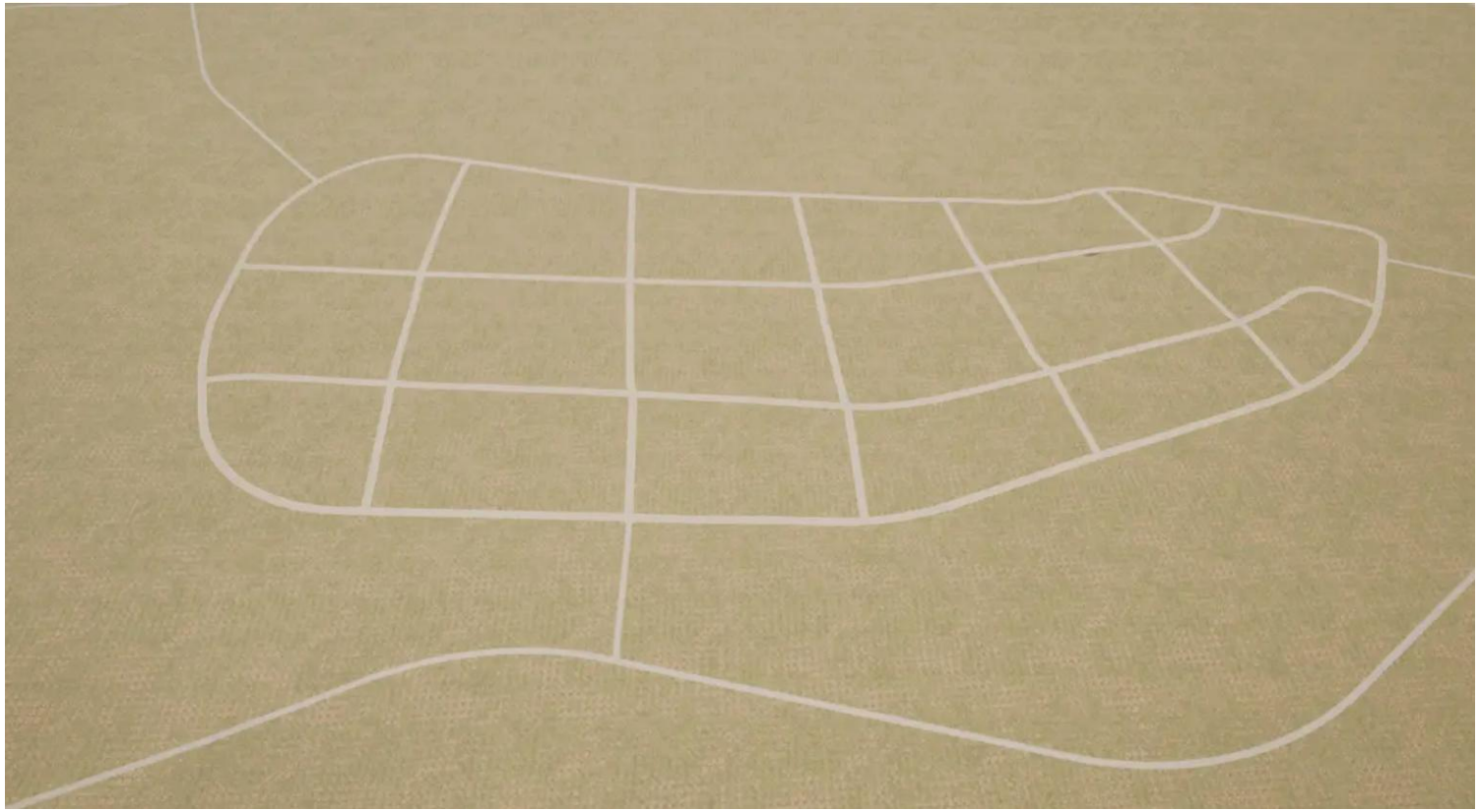
CARLA 기능 실습

- Map(Town07, **Additional Map**)

CARLA practice

▶ CARLA 기능 실습

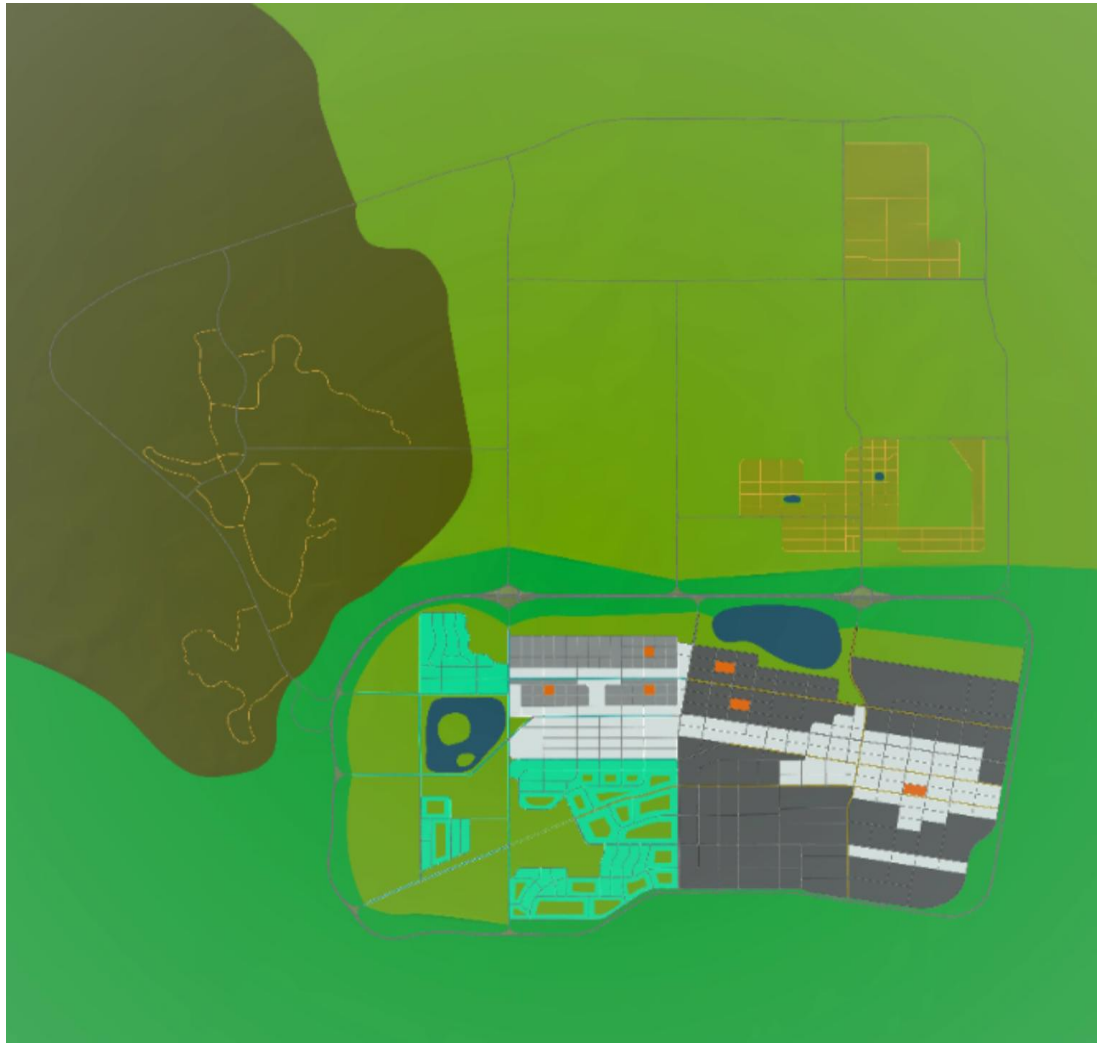
- Map(Town10(**Default Map**), Town11)



CARLA practice

▶ CARLA 기능 실습

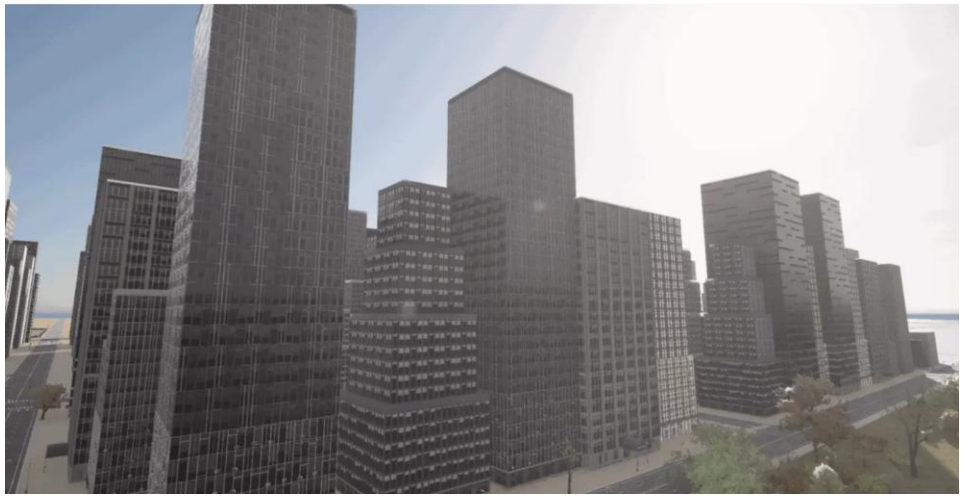
- Map(Town12)

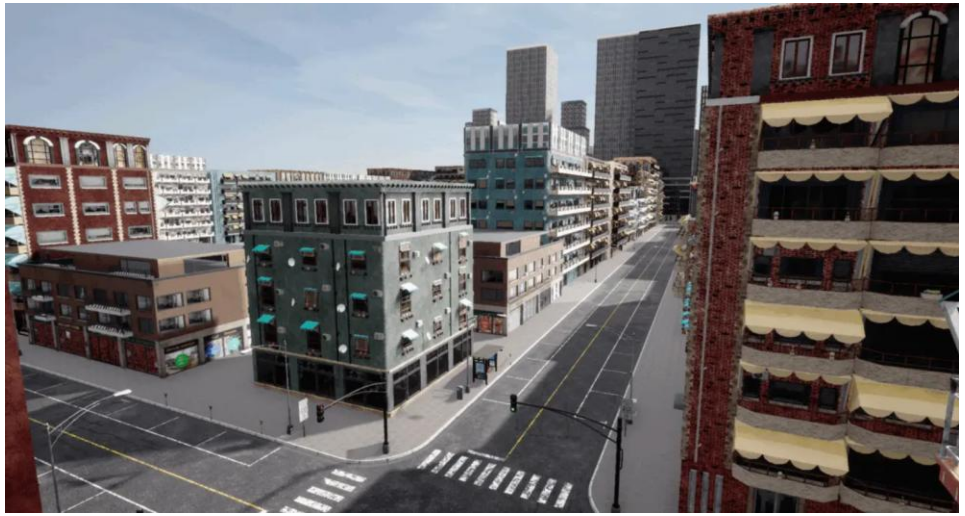


CARLA practice

▶ CARLA 기능 실습

- Map(Town13)





CARLA practice

▶ CARLA 기능 실습

- Map(Town15)

CARLA practice

▶ CARLA 기능 실습

- Map 변경(Town 01~10, 11, 12, 13, 15)

```
(terminal 2) $ ~/mkdir/carla_python_practice && cd carla_python_practice
             $ touch carla_map_change.py
```

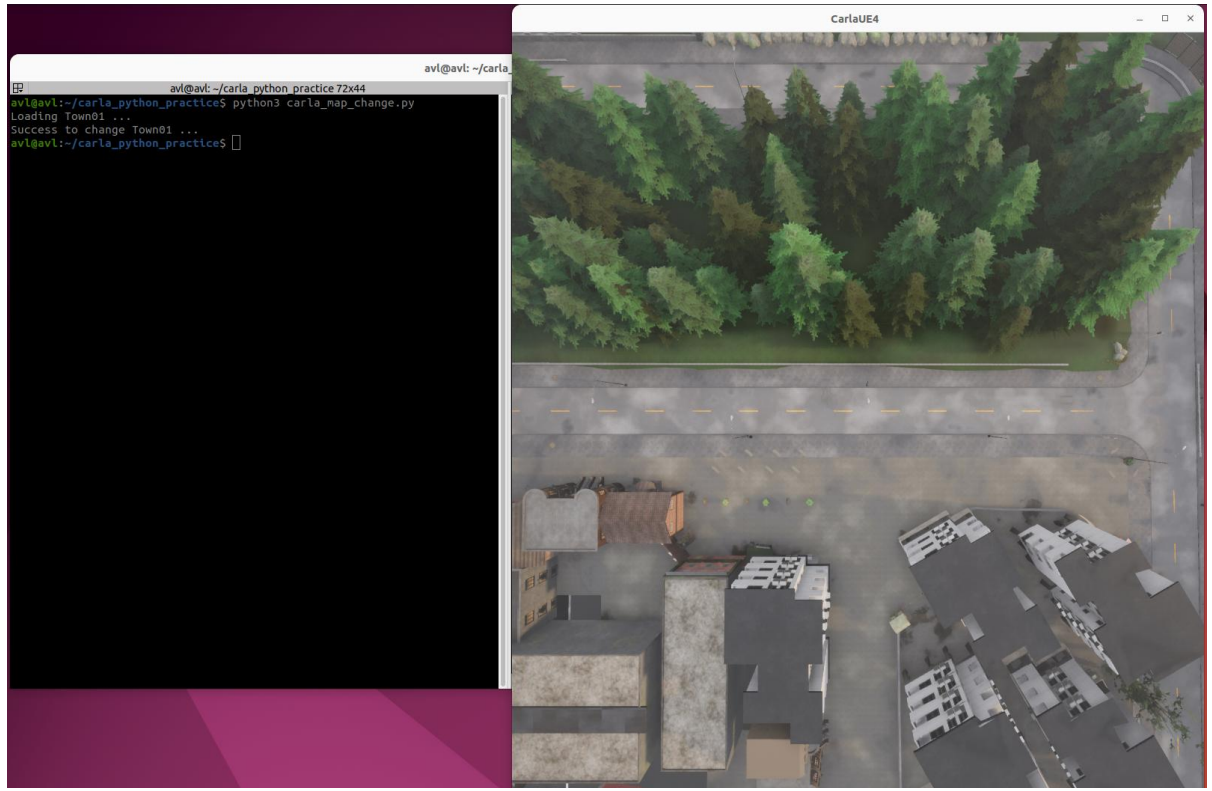
- 코드 작성
  - Loopback IP(127.0.0.1), 2000번 포트로 Carla 접속
  - TOWN 변수를 통해 변경할 맵 지정
  - get_spectator() 함수를 통해 시점 변경

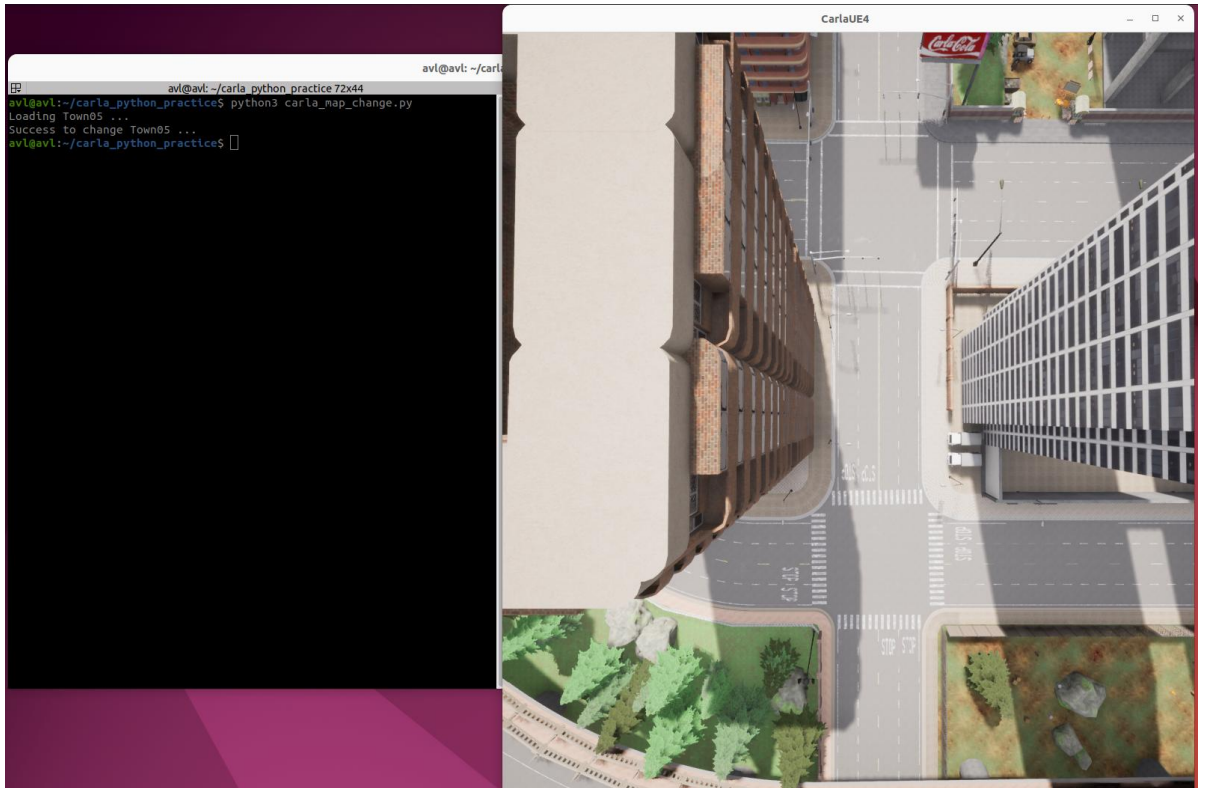
```python
#!/usr/bin/env python3
import carla
import time, sys
TOWN = "Town15" # Town01~10, Town11, Town12, Town13,
Town15
HOST, PORT = "127.0.0.1", 2000
def main():
    client = carla.Client(HOST, PORT)
    client.set_timeout(10.0)
    print(f"Loading {TOWN} ...")
    client.load_world(TOWN)
    while True:
        try:
            if
client.get_world().get_map().name.endswith(TOWN):
                break
        except RuntimeError:
            pass
        time.sleep(0.1)
    world = client.get_world()
    spawns = world.get_map().get_spawn_points()
    if spawns:
        sp = spawns[0]
        loc = sp.location; loc.z += 60.0
        rot = carla.Rotation(pitch=-90)
        world.get_spectator().set_transform(carla.Trans
form(loc, rot))
    print(f"Success to change {TOWN} ...")
if __name__ == "__main__":
    main()
```

CARLA practice

CARLA 기능 실습

- 맵 변경 소스 코드 실행 결과
  - Town01, Town05





CARLA 기능 실습

- 날씨 변경

```
(terminal 2) $ touch carla_weather_change.py
```

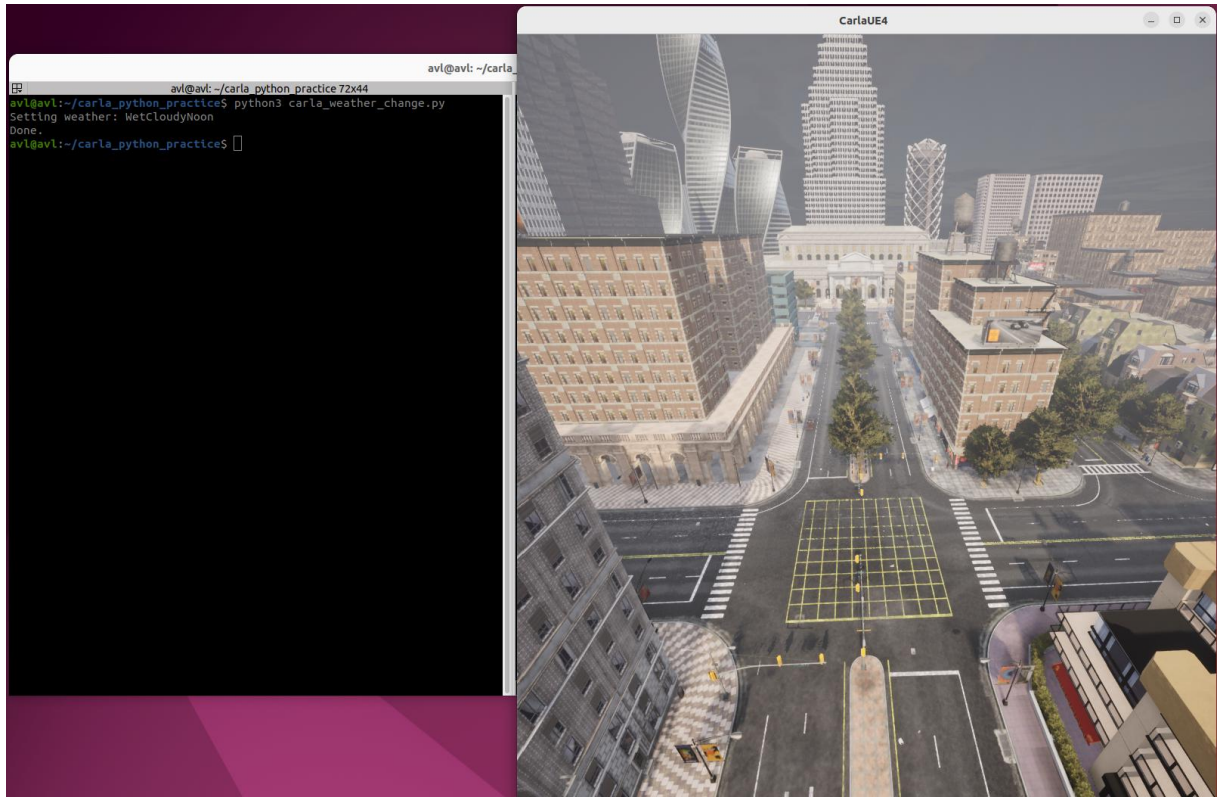
- 코드 작성
  - Loopback IP(127.0.0.1), 2000번 포트로 Carla 접속
  - WEATHER 변수를 통해 변경할 날씨 설정
  - set_weather() 함수를 통해 날씨 변경

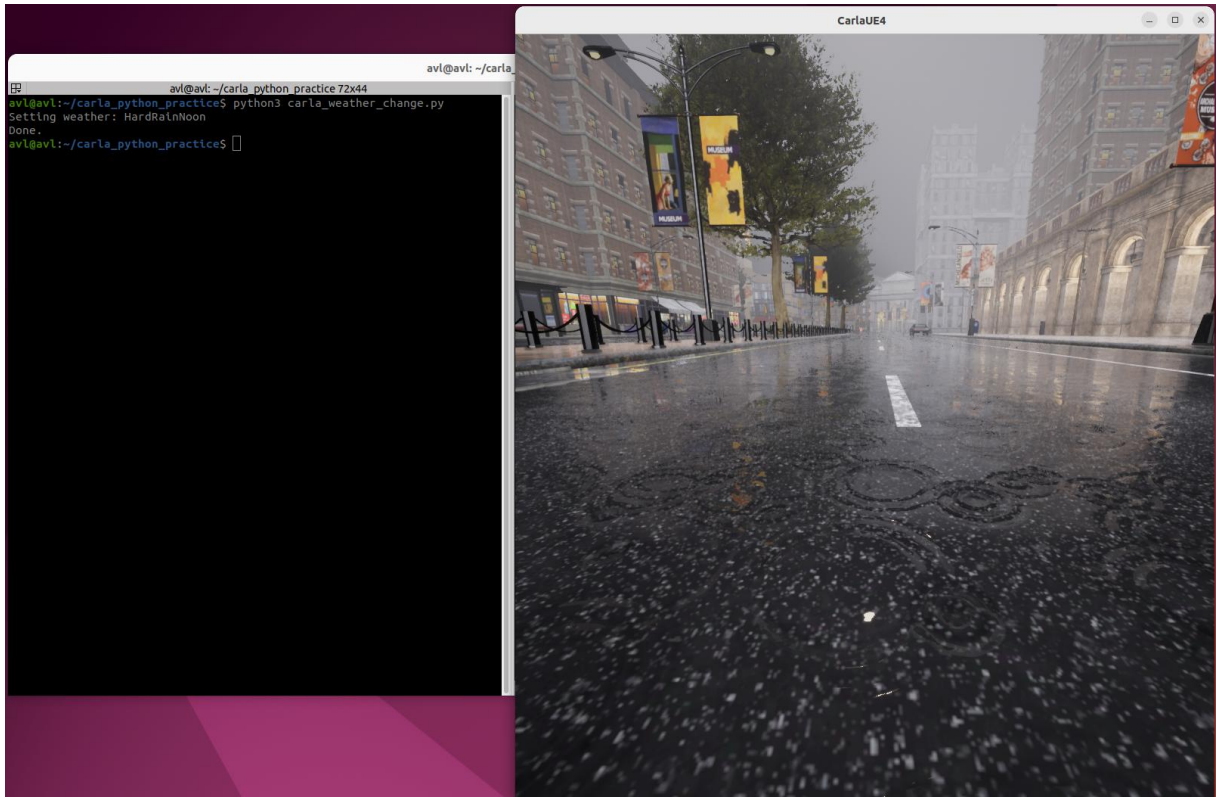
```python
#!/usr/bin/env python3
import carla
import time, sys

WEATHER = "HardRainNoon" # ClearNoon, CloudyNoon, WetCloudyNoon,
SoftRainNoon, HardRainNoon, ClearSunset
HOST, PORT = "127.0.0.1", 2000

def main():
    client = carla.Client(HOST, PORT)
    client.set_timeout(10.0)

    world = client.get_world()

    print(f"Setting weather: {WEATHER}")
    if hasattr(carla.WeatherParameters, WEATHER):
        wp = getattr(carla.WeatherParameters, WEATHER)
    else:
        wp = carla.WeatherParameters.Default
        print("'{WEATHER}' not found.")
    world.set_weather(wp)

    print("Done.")

if __name__ == "__main__":
    main()
```

CARLA practice

CARLA 기능 실습

- 날씨 변경 소스 코드 실행 결과
  - 실제 차량 시점으로 보면 자세한 날씨 변화 확인 가능





CARLA practice

▶ CARLA 기능 실습

- 차량 설정

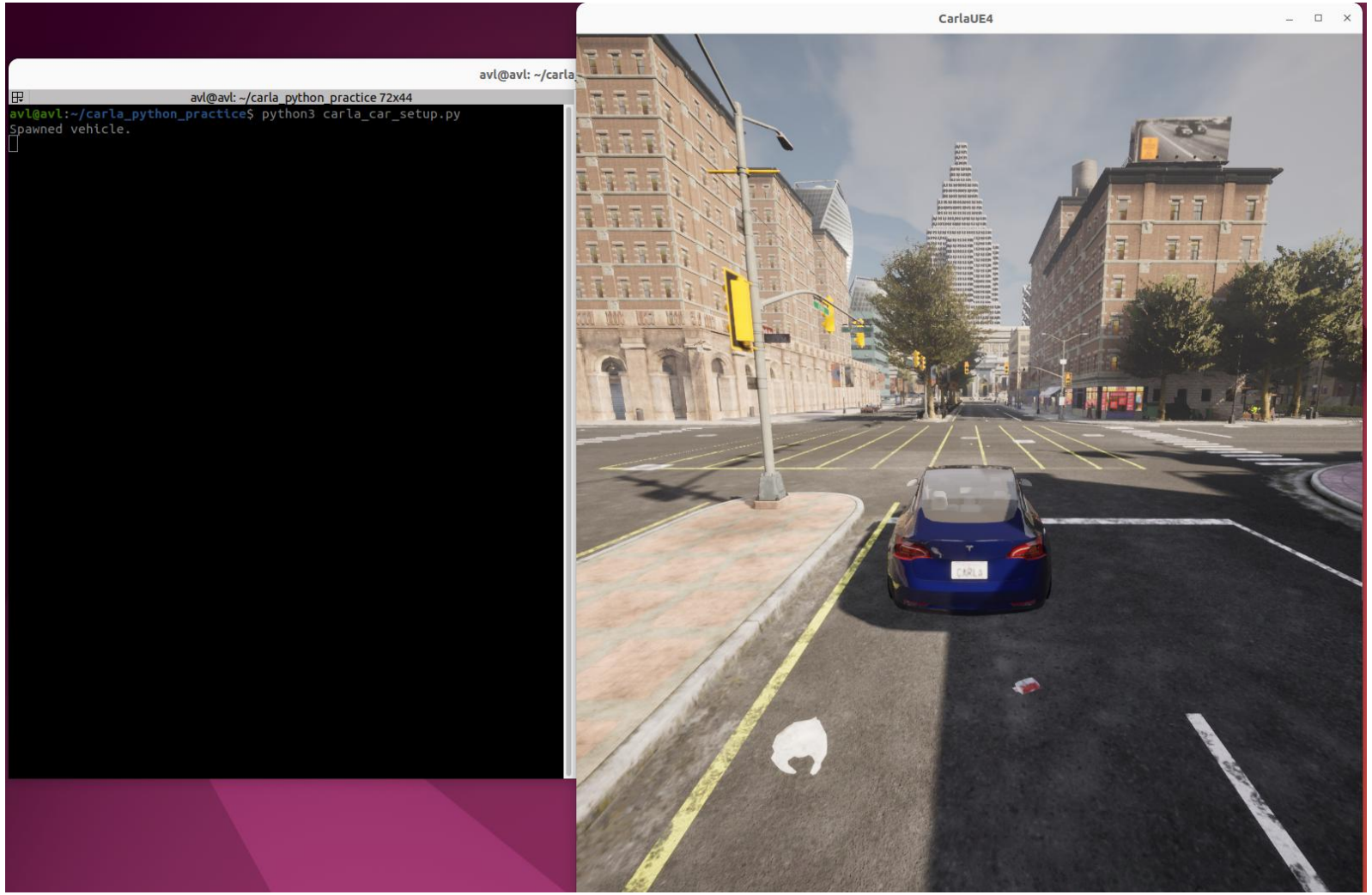
```
(terminal 2) $ touch carla_car_setup.py
```

- 코드 작성
  - vehicle.tesla.model3(테슬라 모델3) 차량 스폰
  - 차량을 3자 시점에서 보도록 설정
  - set_autopilot 함수를 통해 자율주행 On/Off 가능

```python
#!/usr/bin/env python3
import carla, time
HOST, PORT = "127.0.0.1", 2000
def third_person_follow(world, vehicle, dist=6.0, height=2.5,
pitch=-10.0):
    spec = world.get_spectator()
    while True:
        tf = vehicle.get_transform()
        f = tf.get_forward_vector()
        loc = tf.location - f * dist
        loc.z += height
        rot = carla.Rotation(pitch=pitch, yaw=tf.rotation.yaw)
        spec.set_transform(carla.Transform(loc, rot))
        time.sleep(0.03)
def main():
    client = carla.Client(HOST, PORT)
    client.set_timeout(10.0)
    world = client.get_world()
    bp_lib = world.get_blueprint_library()
    bp = bp_lib.find("vehicle.tesla.model3")
    spawn = world.get_map().get_spawn_points()[0]
    vehicle = world.spawn_actor(bp, spawn)
    vehicle.set_autopilot(False)

    print("Spawned vehicle")
    third_person_follow(world, vehicle)

if __name__ == "__main__":
    main()
```

CARLA practice

- CARLA 기능 실습
  - 차량 설정 소스 코드 실행 결과



CARLA practice

▶ CARLA 데이터

- 차량 정보(Ego Vehicle)
  - actor.get_transform().location: 위치(x, y, z)(단위: m)
  - actor.get_transform().rotation: 자세(Pitch, Yaw, Roll)(단위: deg)
  - actor.get_velocity(): 선속도(단위: m/s)
  - actor.get_acceleration(): 선가속도(단위: m/s^2)
  - actor.get_angular_velocity(): 각속도(단위: rad/s)
  - actor.bounding_box.location: 바운딩 박스 중심(단위: m)
  - waypoint.lane_width: 도로 폭(단위: m)
- 차량 제어
  - throttle(0.0-1.0)
  - brake(0.0-1.0)
  - steer(-1.0-1.0)(마이너스가 왼쪽)
  - manual_gear_shift(Boolean)(오토, 수동 주행 설정)
  - hand_brake(Boolean)(오토로 주행 시)
  - reverse(Boolean)(오토로 주행 시)
  - gear(int)(수동 주행 시, -1:R, 0:N, 1:D)

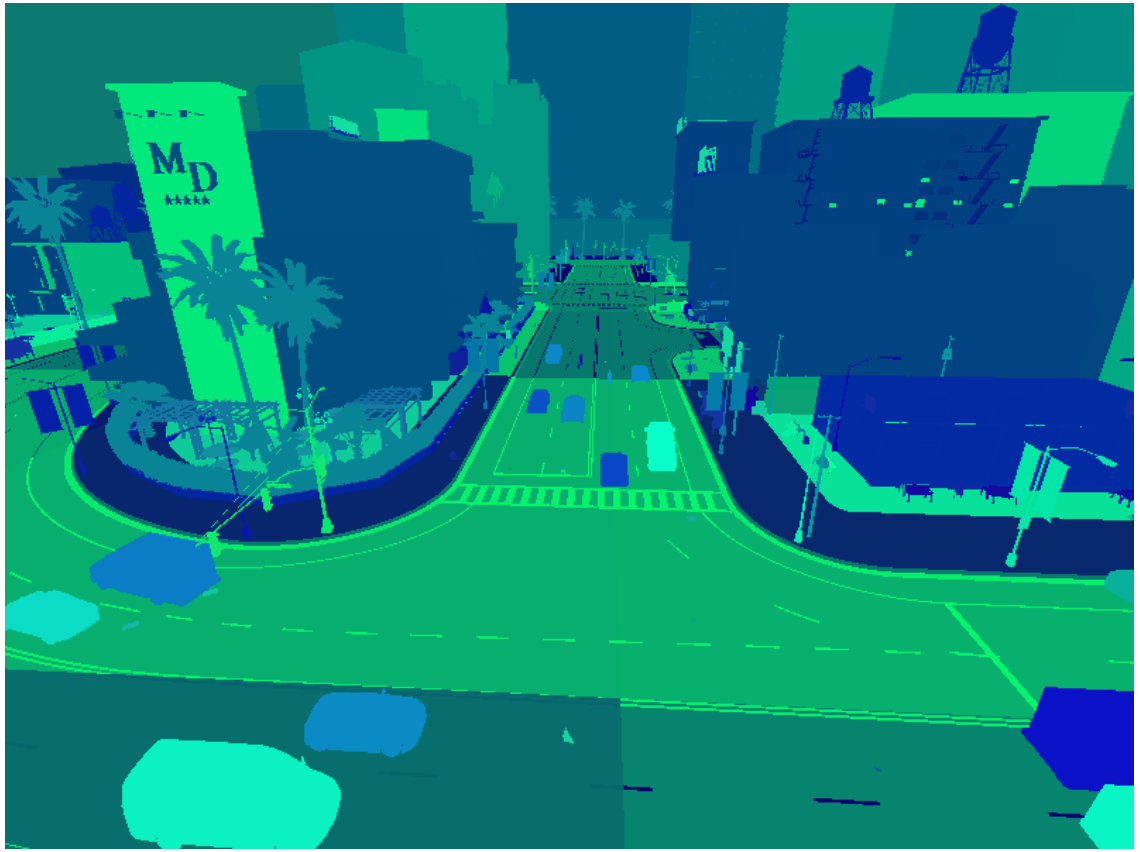
- CARLA 데이터
  - 센서 파라미터
    - 공통
      - role_name: 제어하고자 하는 대상의 고유한 이름
      - sensor_tick: 센서 동작 (단위: s/frame)
      - fixed_delta_seconds: 월드 동작 시간 (fixed_delta_seconds == sensor_tick 추천)
    - Camera
      - image_size_x, y: 해상도
      - fov: 수평 시야각
      - exposure_mode, shutter_speed, iso: 셔터, 노출 모드 옵션
      - lens_circle_falloff, lens_k, lens_kcube, lens_x_size, lens_y_size: 왜곡 및 렌즈 옵션
      - motion_blur_*, dof_*: 모션블러, DOF(심도) 옵션
      - image.raw_data: 출력 타입

CARLA practice

- CARLA 데이터
  - 센서 파라미터
    - LiDAR(x, y, z, intensity)
      - channels: 수직 채널 수(resolution)
      - rotation_frequency: 출력 주기
      - points_per_second: 포인트 속도
      - range: 레이저 거리
      - upper_fov, lower_fov: 수직 FOV
      - dropoff_general_rate, dropoff_intensity_limit, dropoff_zero_intensity: intensity 조절
      - noise_stddev: 레이저 노이즈 표준 편차
    - GNSS(WGS84, latitude, longitude, altitude)
      - noise_lat_stddev, noise_lon_stddev, noise_alt_stddev: 노이즈 표준 편차
      - noise_seed: 노이즈 시드
    - IMU(accelerometer(x,y,z), gyroscope(x,y,z), compass(yaw))
      - noise_accel_stddev_x/y/z: 가속도 노이즈 표준편차
      - noise_gyro_bias_x/y/z: 가속도 바이어스 표준편차
      - noise_gyro_stddev_x/y/z: 자이로 노이즈 표준편차

CARLA practice

- CARLA 데이터
  - 센서 파라미터
    - 언급한 센서들을 포함하여 여러 센서에 대한 reference documentation 존재
    - https://carla.readthedocs.io/en/0.9.16/ref_sensors/



CARLA 기능 실습

- 센서 설정(Camera, LiDAR, GNSS, IMU)

```
(terminal 2) $ touch carla_car_sensor_setup.py
```

- 코드 작성
  - vehicle.tesla.model3(테슬라 모델3) 차량 스폰
  - 차량을 3자 시점에서 보도록 설정
  - API를 통해 Camera, LiDAR, GNSS, IMU 파라미터 설정
    - Camera, LiDAR는 시각화
    - GNSS, IMU는 Terminal 출력

```python
#!/usr/bin/env python3
import carla, time, numpy as np, cv2


HOST, PORT = "127.0.0.1", 2000
CAM_W, CAM_H, CAM_FOV = 320, 240, 90
LIDAR_IMG_SIZE, LIDAR_SCALE = 150, 4.0


last_cam = None
last_lidar = np.zeros((LIDAR_IMG_SIZE, LIDAR_IMG_SIZE, 3),
np.uint8)
last_gnss = None
last_imu = None


def third_person_follow(world, vehicle, dist=6.0, height=2.5,
pitch=-10.0):
    spec = world.get_spectator()
    tf = vehicle.get_transform()
    f = tf.get_forward_vector()
    loc = tf.location - f * dist
    loc.z += height
    rot = carla.Rotation(pitch=pitch, yaw=tf.rotation.yaw)
    spec.set_transform(carla.Transform(loc, rot))
```

CARLA practice

▶ CARLA 기능 실습

- 센서 설정(Camera, LiDAR, GNSS, IMU)

```
def on_cam(img):
    global last_cam
    a = np.frombuffer(img.raw_data,
dtype=np.uint8).reshape((img.height, img.width, 4))[:, :, :3]
    last_cam = a

def lidar_to_bev(points):
    img = np.zeros((LIDAR_IMG_SIZE, LIDAR_IMG_SIZE, 3), np.uint8)
    if points.size == 0:
        return img
    x = points[:, 0]; y = points[:, 1]
    cx = LIDAR_IMG_SIZE // 2; cy = LIDAR_IMG_SIZE - 50
    u = (cx + x * LIDAR_SCALE).astype(np.int32)
    v = (cy - y * LIDAR_SCALE).astype(np.int32)
    m = (u >= 0) & (u < LIDAR_IMG_SIZE) & (v >= 0) & (v
< LIDAR_IMG_SIZE)
    img[v[m], u[m]] = (255, 255, 255)
    cv2.line(img, (cx, cy), (cx, cy - 100), (128, 128, 128), 1)
    return img

def on_lidar(meas):
    global last_lidar
    a = np.frombuffer(meas.raw_data, dtype=np.float32).reshape(-1,
4)
    last_lidar = lidar_to_bev(a)
```

```
def on_gnss(meas):
    global last_gnss
    last_gnss = (meas.latitude, meas.longitude, meas.altitude)

def on_imu(meas):
    global last_imu
    last_imu = (meas.accelerometer.x, meas.accelerometer.y,
meas.accelerometer.z,
                meas.gyroscope.x, meas.gyroscope.y, meas.gyroscope.z,
meas.compass)

def main():
    client = carla.Client(HOST, PORT)
    client.set_timeout(10.0)
    world = client.get_world()

    # sync world to avoid LiDAR sweep
    settings = world.get_settings()
    settings.synchronous_mode = True
    settings.fixed_delta_seconds = 0.1
    world.apply_settings(settings)

    bp = world.get_blueprint_library().find("vehicle.tesla.model3")
    vehicle = world.spawn_actor(bp, world.get_map().get_spawn_points()[0])
    vehicle.set_autopilot(False)
    print("Spawned vehicle")
```

▶ CARLA 기능 실습

- 센서 설정(Camera, LiDAR, GNSS, IMU)

```
    cam_bp = world.get_blueprint_library().find("sensor.camera.rgb")
    cam_bp.set_attribute("image_size_x", str(CAM_W))
    cam_bp.set_attribute("image_size_y", str(CAM_H))
    cam_bp.set_attribute("fov", str(CAM_FOV))
    cam_bp.set_attribute("sensor_tick", "0.1")
    cam = world.spawn_actor(cam_bp,
carla.Transform(carla.Location(x=0.6, z=1.6)), attach_to=vehicle)
    cam.listen(on_cam)

    lidar_bp =
world.get_blueprint_library().find("sensor.lidar.ray_cast")
    lidar_bp.set_attribute("rotation_frequency", "10.0")
    lidar_bp.set_attribute("sensor_tick", "0.1")
    lidar_bp.set_attribute("channels", "32")
    lidar_bp.set_attribute("range", "80.0")
    lidar_bp.set_attribute("points_per_second", "300000")
    lidar_bp.set_attribute("upper_fov", "10.0")
    lidar_bp.set_attribute("lower_fov", "-30.0")
    lidar_bp.set_attribute("dropoff_general_rate", "0.0")
    lidar_bp.set_attribute("dropoff_intensity_limit", "1.0")
    lidar_bp.set_attribute("dropoff_zero_intensity", "0.0")
    lidar_bp.set_attribute("noise_stddev", "0.0")
    lidar = world.spawn_actor(lidar_bp,
carla.Transform(carla.Location(x=0.0, z=2.0)), attach_to=vehicle)
    lidar.listen(on_lidar)
```

```
    gnss = world.spawn_actor(world.get_blueprint_library().find("sensor.other.gnss"),
                             carla.Transform(), attach_to=vehicle)
    gnss.listen(on_gnss)

    imu = world.spawn_actor(world.get_blueprint_library().find("sensor.other.imu"),
                            carla.Transform(), attach_to=vehicle)
    imu.listen(on_imu)

    cv2.namedWindow("Camera", cv2.WINDOW_AUTOSIZE)
    cv2.namedWindow("LiDAR BEV", cv2.WINDOW_AUTOSIZE)

    log_ts = 0.0
    try:
        while True:
            world.tick()
            third_person_follow(world, vehicle)


            if last_cam is not None:
                cv2.imshow("Camera", last_cam)
            if last_lidar is not None:
                cv2.imshow("LiDAR BEV", last_lidar)
```

CARLA practice

▶ CARLA 기능 실습

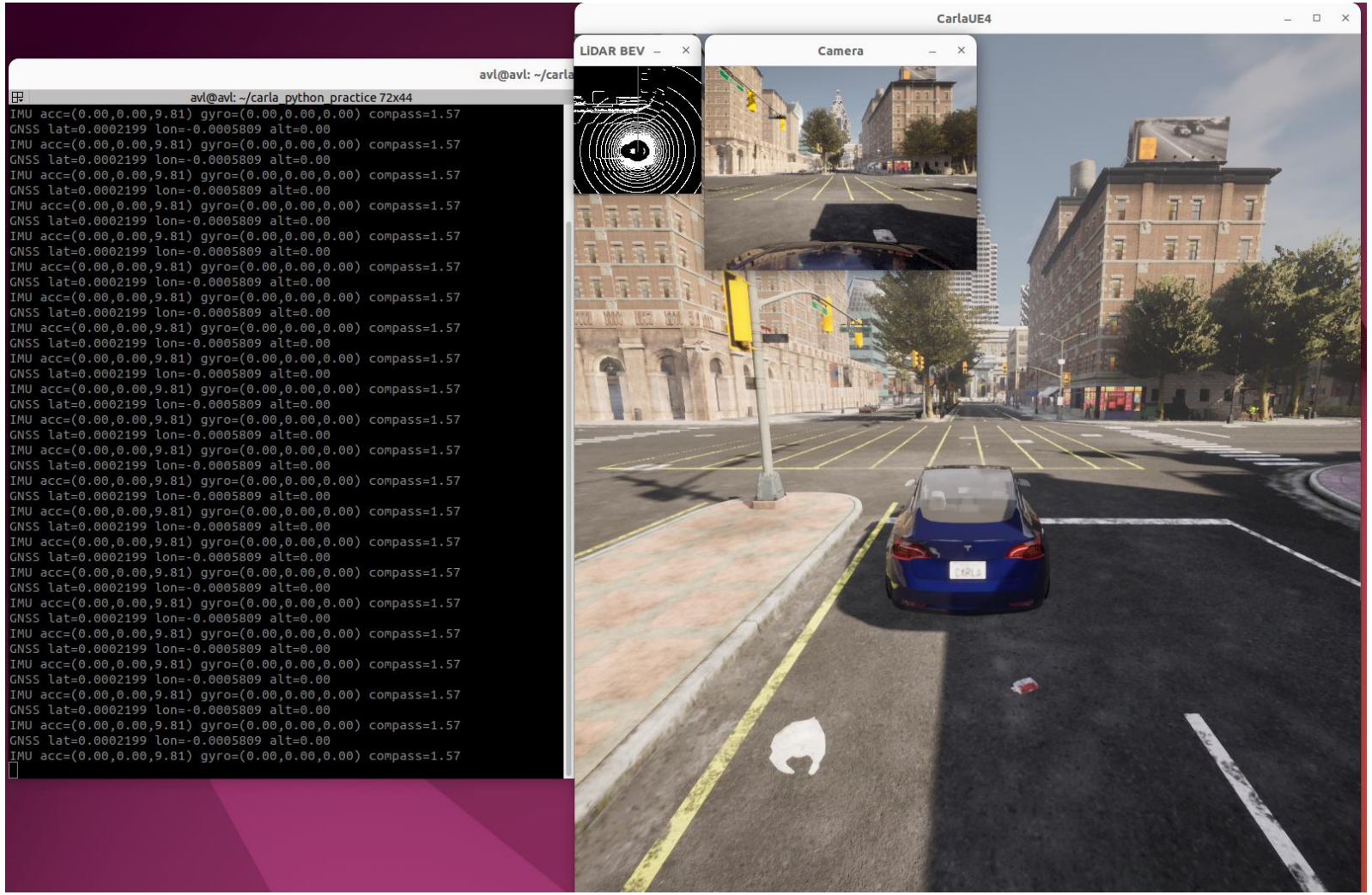
- 센서 설정(Camera, LiDAR, GNSS, IMU)

```
            t = time.time()
            if t - log_ts > 1.0:
                log_ts = t
                if last_gnss is not None:
                    lat, lon, alt = last_gnss
                    print(f"GNSS lat={lat:.7f} lon={lon:.7f} alt={alt:.2f}")
                if last_imu is not None:
                    ax, ay, az, gx, gy, gz, c = last_imu
                    print(f"IMU acc=({ax:.2f},{ay:.2f},{az:.2f}) gyro=({gx:.2f},{gy:.2f},{gz:.2f}) compass={c:.2f}")


            if cv2.waitKey(1) & 0xFF == ord('q'):
                break
    finally:
        cam.stop(); lidar.stop(); gnss.stop(); imu.stop()
        vehicle.destroy(); cam.destroy(); lidar.destroy(); gnss.destroy(); imu.destroy()
        cv2.destroyAllWindows()
        settings.synchronous_mode = False
        settings.fixed_delta_seconds = None
        world.apply_settings(settings)

if __name__ == "__main__":
    main()
```

CARLA practice

CARLA 기능 실습

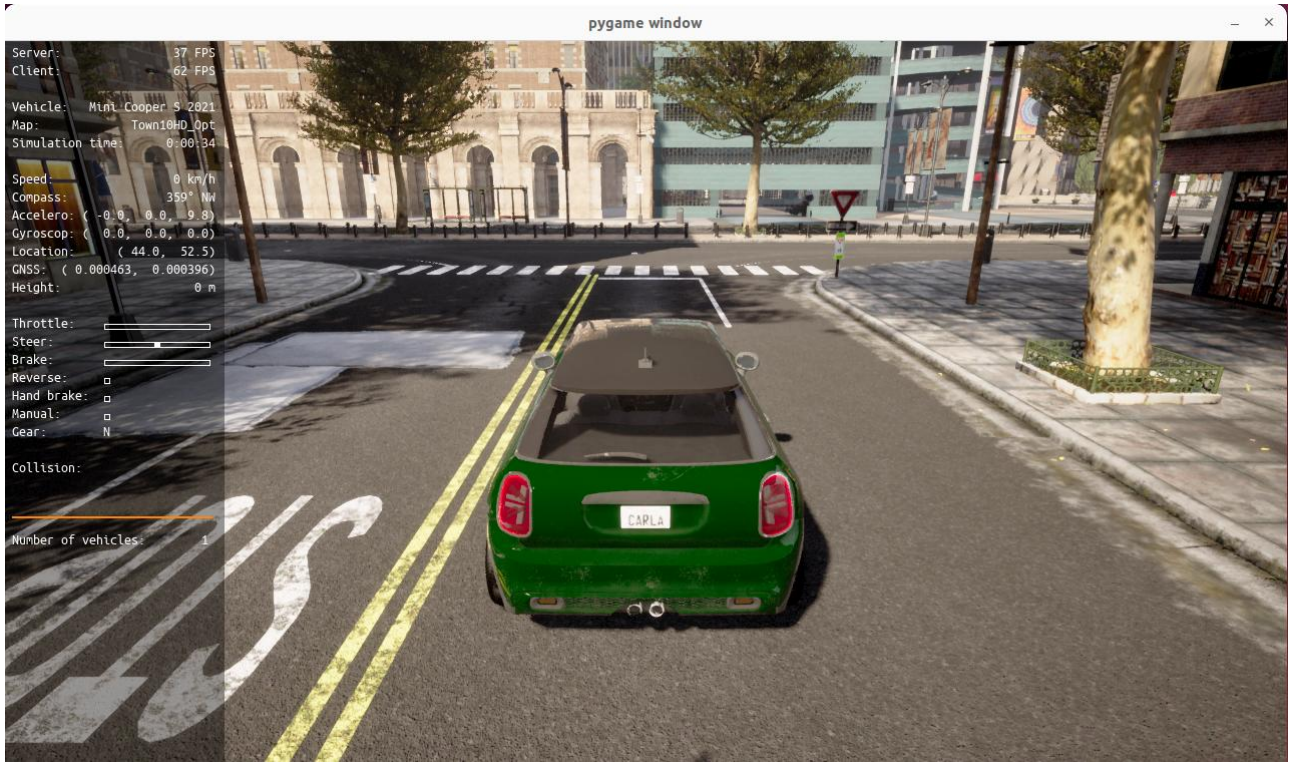
- 센서 설정 소스 코드 실행 결과



▶ CARLA 기능 실습

- 종/횡 방향 제어(예제 코드 활용)

```
(terminal 2) $ cd ~/carla/PythonAPI/examples
             $ python3 manual_control.py
```



❖ 자주 사용하는 명령어

- W: 가속
- S: 브레이크
- A/D: 좌/우 조향
- Q: 기어 변경(D, R)
- Space: 사이드 브레이크
- P: CARLA 내장 자율주행 토글
- Ctrl+W: 60km/h 크루즈 컨트롤
- TAP: 카메라 시점 변경
- 숫자 [1~9]: 센서 변경 (시각화)
- G: RADAR 시각화 토글
- C: 날씨 변경 (Shift+C: 역순 변경)
- Backspace: 차량 모델 변경

▶ CARLA 기능 실습

- 장애물 설치

```
(terminal 2) $ cd ~/carla_python_practice
             $ touch carla_create_collision.py
```

- 코드 작성
  - Waypoint 기반으로 앞쪽 차선 중심에 장애물 설치
    - DIST_LIST로 앞 거리, LAT_FACTORS로 좌우 오프셋 설정
  - 0.5m를 띄워 지면 충돌 오류 방지

```python
#!/usr/bin/env python3
import carla, time

HOST, PORT = "127.0.0.1", 2000
DIST_LIST = [10.0, 14.0, 18.0, 22.0]   # meters
LAT_FACTORS = [0.0, 0.3, -0.3]          # road lane lateral offset
MODEL = "vehicle.audi.tt"

def get_hero(world):
    for v in world.get_actors().filter("vehicle.*"):
        if v.attributes.get("role_name") == "hero":
            return v
    vs = world.get_actors().filter("vehicle.*")
    return vs[0] if len(vs) else None

def main():
    client = carla.Client(HOST, PORT)
    client.set_timeout(10.0)
    world = client.get_world()
    m = world.get_map()
    bp_lib = world.get_blueprint_library()

    hero = get_hero(world)
    if not hero:
        print("No vehicle found."); return

    bp = bp_lib.find(MODEL) if bp_lib.find(MODEL) else
bp_lib.filter("vehicle.*")[0]
    bp.set_attribute("role_name", "obstacle")
```

▶ CARLA 기능 실습

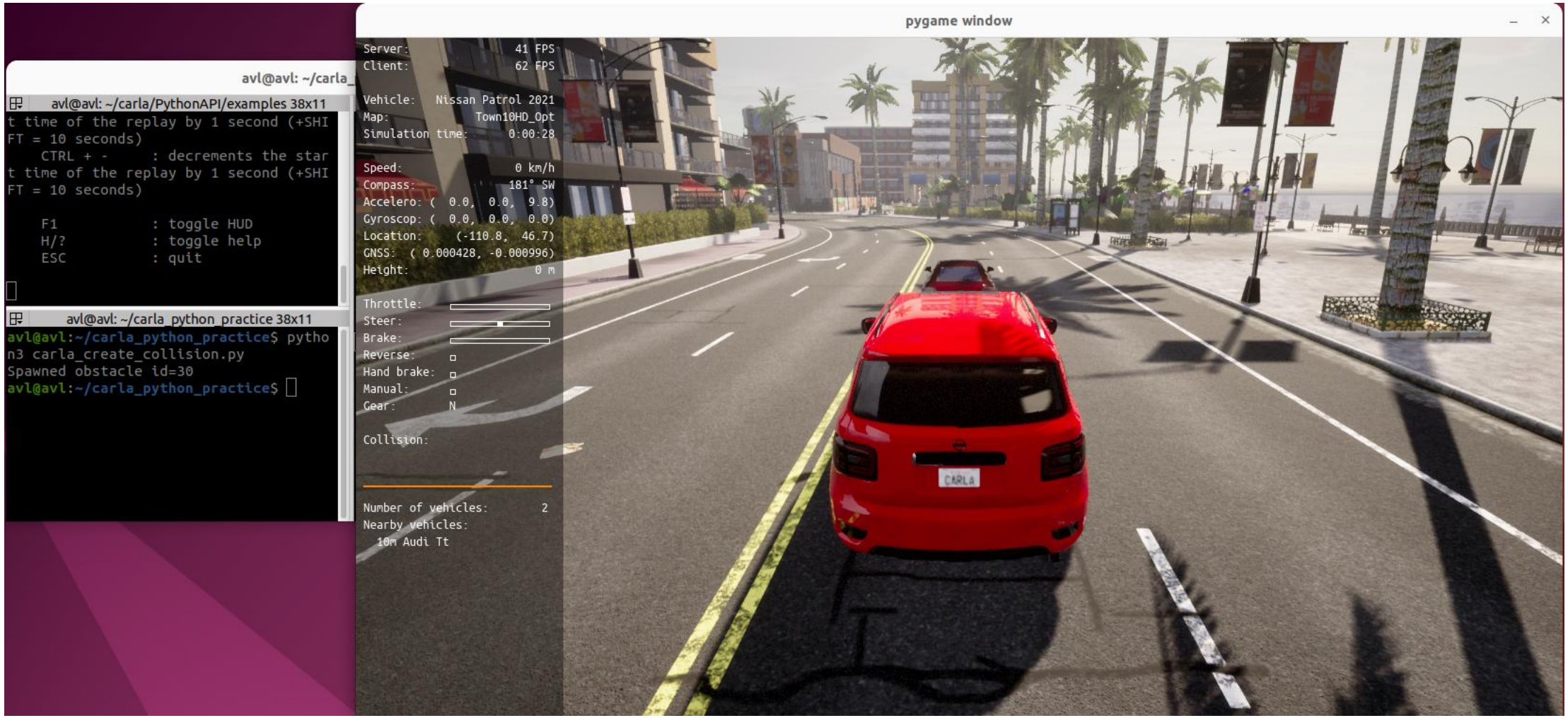
- 장애물 설치

```
    # compute candidate spawn transforms on the driving lane ahead of hero
    h_loc = hero.get_location()
    wp0 = m.get_waypoint(h_loc, project_to_road=True,
lane_type=carla.LaneType.Driving)
    lane_w = wp0.lane_width if wp0 else 3.5
    cands = []
    for d in DIST_LIST:
        wps = wp0.next(d) if wp0 else []
        if not wps: continue
        base = wps[0].transform
        base.location.z += 0.5
        right = base.get_right_vector()
        for lf in LAT_FACTORS:
            t = carla.Transform(
                carla.Location(
                    base.location.x + right.x * (lane_w*lf),
                    base.location.y + right.y * (lane_w*lf),
                    base.location.z
                ),
                carla.Rotation(pitch=0.0, yaw=base.rotation.yaw, roll=0.0)
            )
            cands.append(t)
```

```
    obstacle = None
    for tf in cands:
        obstacle = world.try_spawn_actor(bp, tf)
        if obstacle: break

    if not obstacle:
        print("Spawn failed."); return

    obstacle.set_autopilot(False)
    obstacle.apply_control(carla.VehicleControl(throttle=0.0, brake=1.0,
hand_brake=True))
    print(f"Spawned obstacle id={obstacle.id}")

    time.sleep(2.0)

if __name__ == "__main__":
    main()
```

CARLA 기능 실습

- 장애물 설치 소스 코드 실행 결과

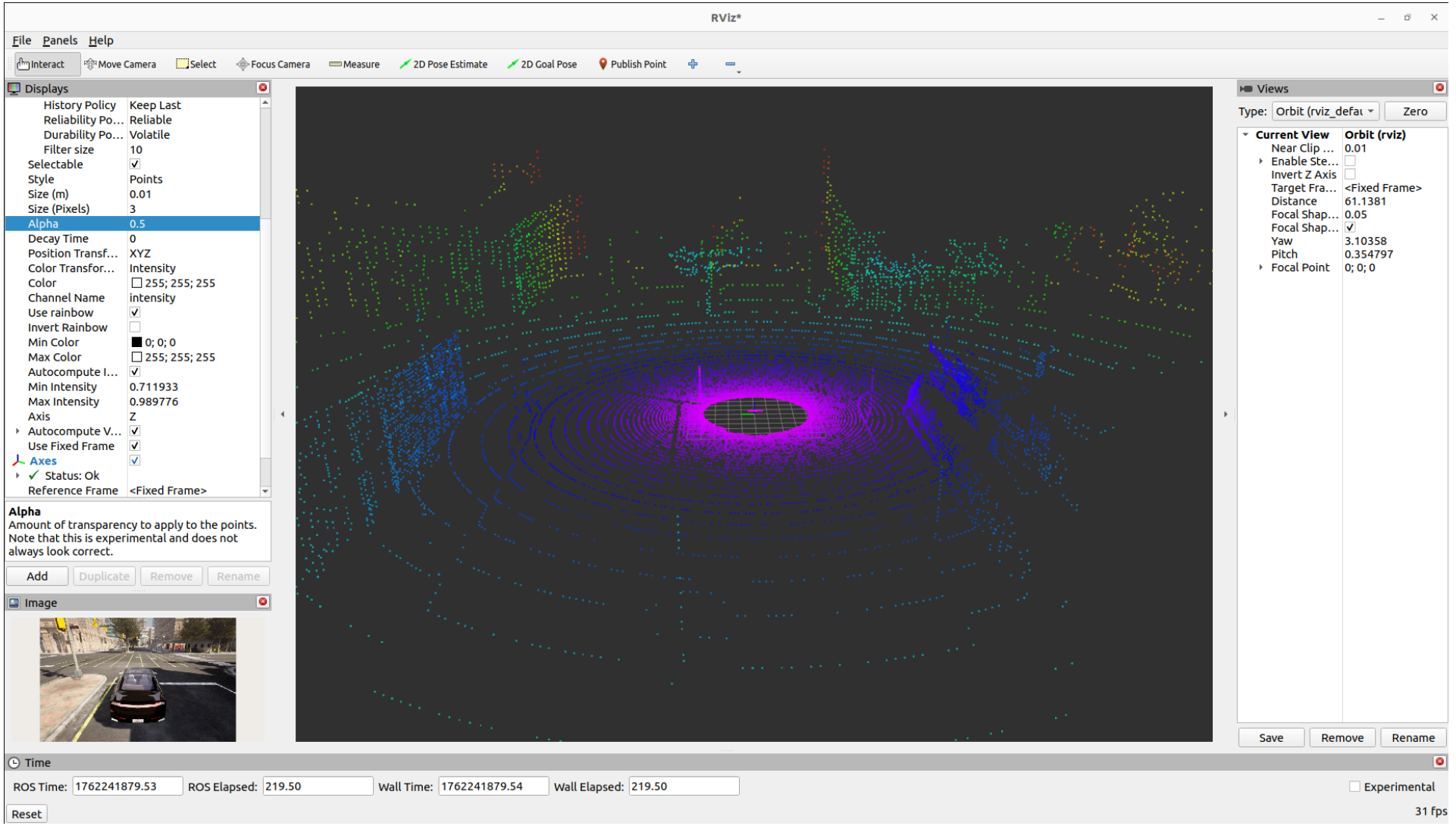


ROS2-CARLA

ROS2 CARLA 실행 및 시각화

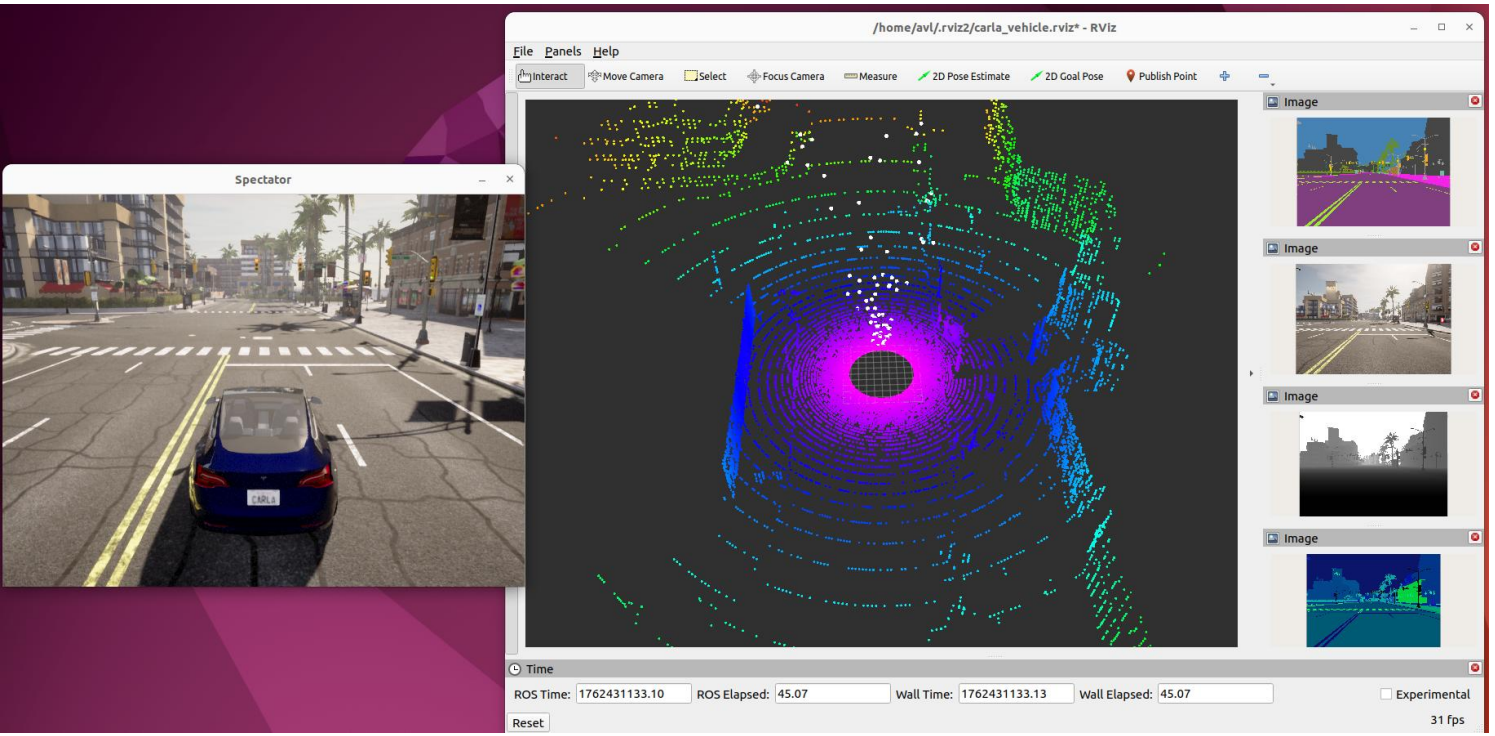
```
(terminal 1) $ cd ~/carla && ./CarlaUE4.sh --ros2
(terminal 2) $ cd ~/carla/PythonAPI/examples/ros2 && python3 ros2_native.py --file stack.json
(terminal 3) $ rviz2
```

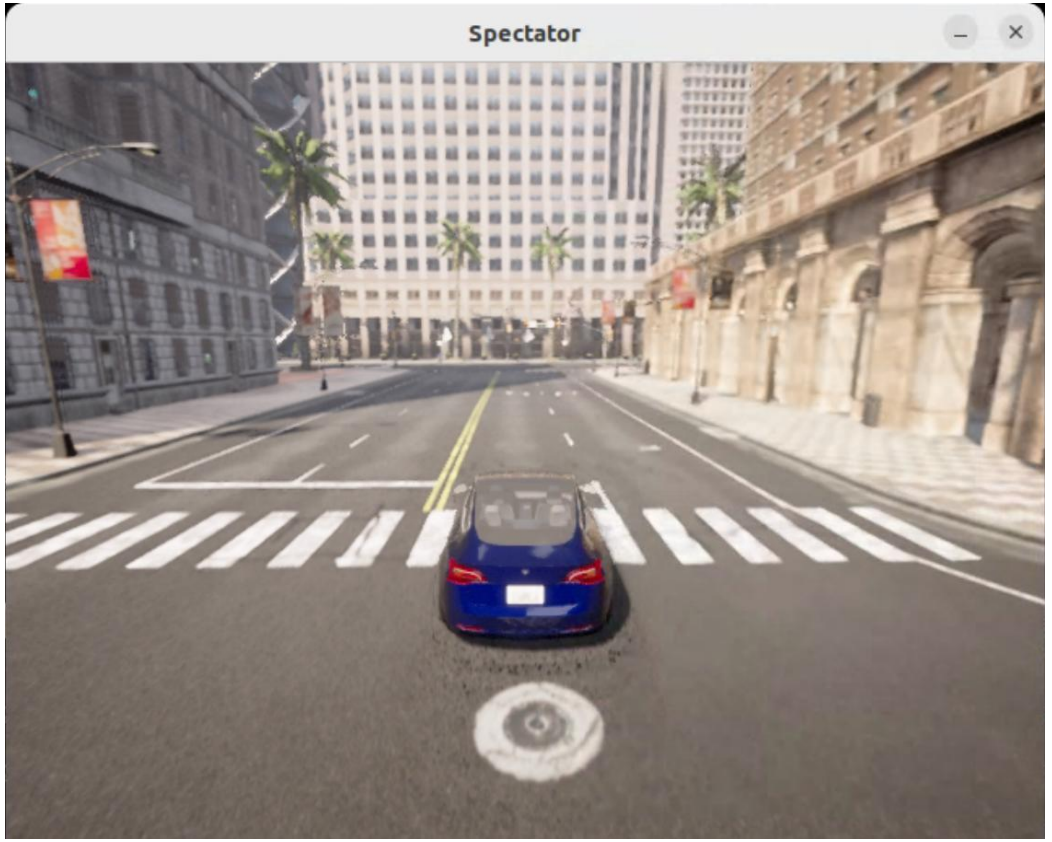
ROS2 데이터 확인(Rviz2)



ROS2-CARLA

- ROS2-CARLA 연동 데이터 취득(Github)
  - 센서 설정 파라미터 수정(tesla.json)
  - 센서 데이터 처리 파이프라인 추가
  - 3자 시점 추가

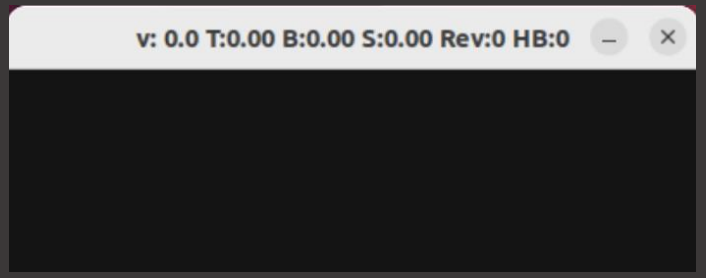




ROS2-CARLA 연동 데이터 취득

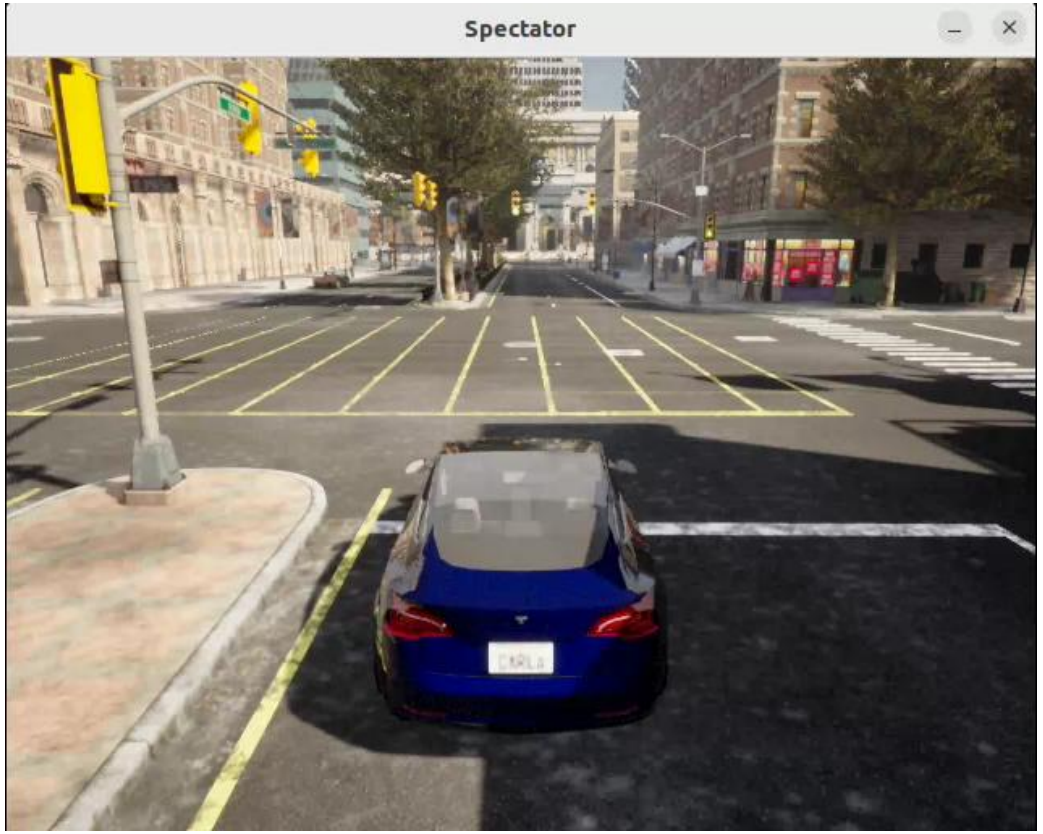
- 차량 제어(키보드, Autopilot 해제)
  - W/A/S/D, Q: 후진 기어, D: 전진 기어, Space: 브레이크
  - rosbag record로 주행 데이터 logging 가능

```
(terminal 1) $ cd carla
             $ ./CarlaUE4.sh --ros2 -RenderOffScreen
(terminal 2) $ cd ~/carla/PythonAPI/examples/ros2
             $ python3 ros2_native.py --file tesla.json
(terminal 3) $ cd ~/carla/PythonAPI/examples/ros2
             $ python3 teleop_key.py
(terminal 4)(optional) $ mkdir ~/carla_logging && cd ~/carla_logging
                       $ rosbag record -a
```



ROS2-CARLA 연동 데이터 취득

- 차량 제어(ROS2 Publish)
  - 코드 작성
    - 1. 속도, 조향, 브레이크 값이 담긴 cmd_vel 토픽 전송
    - 2. ros2_native.py의 subscribe 부분에서 차량 제어




```python
#!/usr/bin/env python3
import time, rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

V_MPS    = 0.0    # m/s
STEER_D = 0.0    # Tire angle
BRAKE    = 0.0    # [0~1]
carla_vehicle_control_topic = "/carla/hero/cmd_vel"

class VehicleControl(Node):
    def __init__(self):
        super().__init__("ros2_vehicle_control")
        while True:
            pub = self.create_publisher(Twist,
carla_vehicle_control_topic, 10)
            msg = Twist()
            msg.linear.x  = V_MPS
            msg.linear.y  = BRAKE
            msg.angular.z = STEER_D
            pub.publish(msg)
            self.get_logger().info(f"publish -> v={V_MPS:.2f} m/s,
brake={BRAKE:.2f}, steer={STEER_D:.1f} deg")
            time.sleep(0.05) # 20hz
def main():
    rclpy.init()
    rclpy.spin(VehicleControl())

if __name__ == "__main__":
    main()
```

ROS2-CARLA 연동 데이터 취득(실습 코드 다운로드)

- C_track 맵 변경(**C_track_CARLA_Map 압축 해제**)

```
(terminal 1) $ cd carla
             $ ./CarlaUE4.sh --ros2 -RenderOffScreen
(terminal 2) $ cd ~/carla/PythonAPI/util
             $ python3 config.py --map C_track
(terminal 3) $ cd ~/carla/PythonAPI/examples/ros2
             $ python3 ros2_native.py --file tesla.json
```

Autonomous Driving Application Project

Lane Detection

- Classical Lane Detection Methods
  - 1. Edge-based Feature Extraction
    - Canny Edge Detector, Sobel Operator, Laplacian Filter 등을 이용해 도로 경계(edge) 강조
    - 차선은 일반적으로 밝기·색 대비가 뚜렷하므로, gradient magnitude 기반 검출에 적합
    - ROI(Region of Interest) 설정을 통해 하부 도로 영역만 탐색 -> 연산량 감소 및 오검출 방지
  - 2. Hough Transformation
    - edge points를 (ρ, θ) 공간으로 투영 후 직선의 누적 투표(voting) 기반 검출
      - ρ: 원점에서 직선까지의 수직선
      - θ: 수직선과 직선 사이 각도
    - Probabilistic Hough Transform 또는 Progressive Probabilistic Hough Transform (PPHT) 으로 계산 최적화
    - 직선 차선(urban/highway)에서는 우수한 성능, 그러나 곡선 구간 및 차선 단절 구간에서 취약
  - 3. Polynomial Fitting with Inverse Perspective Mapping (IPM)
    - 카메라 영상을 Bird's-Eye View (BEV) 로 투영
      - 차선이 평면상 직선 혹은 완만한 곡선 형태로 변환
    - lane 영역을 color mask로 필터링 -> Sliding Window Search + 2nd/3rd Order Polynomial Fitting 수행
      - 허프 변환의 직선 제약을 극복, 곡선 차선 검출 가능

Lane Detection

- Deep Learning-based Lane Detection
  - Convolutional Neural Networks (CNN), Transformer 기반 모델이 등장하며 급격히 발전
  - 차선이 불연속적이거나 가려진 경우에도 contextual feature learning을 통해 연속성 유지
  - 대표 모델
    - SCNN (Spatial CNN, CVPR 2018)
      - Convolution propagation across rows/columns -> lane structure 유지
    - LaneNet (VPGNet, ECCV 2018)
      - Instance segmentation 기반 다중 차선 분리
    - **Ultra Fast Lane Detection (UFLD, ECCV 2020) ← 실습**
      - Anchor-based classification 방식으로 real-time (300 FPS) 구현
    - LLama or Transformer-based Lane Detection (2022~)
      - Global attention으로 lane continuity, topology 인식

Ultra Fast Lane Detection

- Deep Learning 분야는 기본적으로 High Computational cost를 갖고 있음
  - 하지만 자율주행 분야의 차선 인식은 Low Computational cost를 요구
    - UFLD는 정확도를 유지하며 속도를 높이기 위해 단순한 수식을 제안
      - **2020년 기준 정확도, 속도 모두 SOTA 달성, 약 300+ FPS로 기존 방법 대비 4배의 효과를 보임**

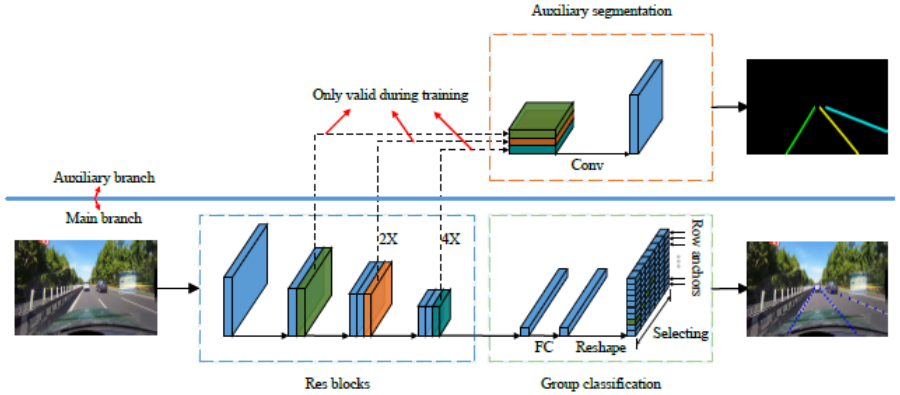


Fig. 4. Overall architecture. The auxiliary branch is shown in the upper part, which is only valid when training. The feature extractor is shown in the blue box. The classification-based prediction and auxiliary segmentation task are illustrated in the green and orange boxes, respectively. The group classification is conducted on each row anchor.

Category	Res50-Seg[3]	SCNN[22]	FD-50[24]	Res34-SAD	SAD[9]	Res18-Ours	Res34-Ours
Normal	87.4	90.6	85.9	89.9	90.1	87.7	**90.7**
Crowded	64.1	69.7	63.6	68.5	68.8	66.0	**70.2**
Night	60.6	66.1	57.8	64.6	66.0	62.1	**66.7**
No-line	38.1	43.4	40.6	42.2	41.6	40.2	**44.4**
Shadow	60.7	66.9	59.9	67.7	65.9	62.8	**69.3**
Arrow	79.0	84.1	79.4	83.8	84.0	81.0	**85.7**
Dazzlelight	54.1	58.5	57.0	59.9	**60.2**	58.4	59.5
Curve	59.8	64.4	65.2	66.0	65.7	57.9	**69.5**
Crossroad	2505	1990	7013	1960	1998	**1743**	2037
Total	66.7	71.6	-	70.7	70.8	68.4	**72.3**
Runtime(ms)	-	133.5	-	50.5	13.4	3.1	5.7
Multiple	-	1.0x	-	2.6x	10.0x	43.0x	23.4x
FPS	-	7.5	-	19.8	74.6	322.5	175.4

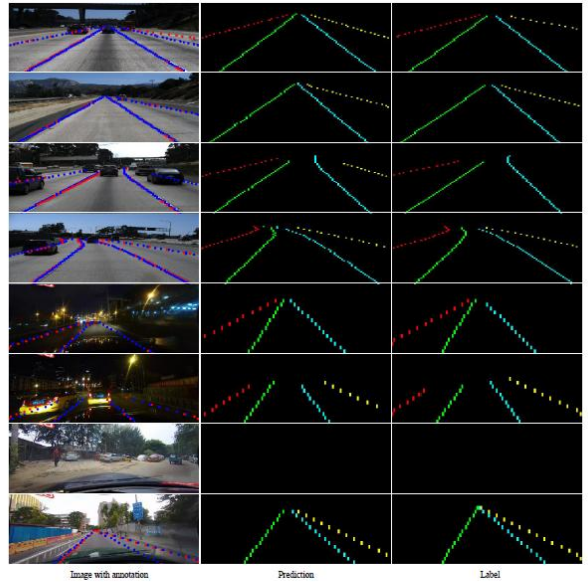


Fig. 8. Visualization on the Tusimple lane detection benchmark and the CULane dataset. The first four rows are results on the Tusimple dataset. The rest rows are results on the CULane dataset. From left to right, the results are image, prediction and ground truth, respectively. In the image, predicted lane points are marked in blue and ground truth annotations are marked in red. Because our classification-based formulation only predicts on the predefined row anchors, the scales of images and labels in the vertical direction are not identical.

Lane Detection Summary

방법	핵심 기술	성능	취약성
Edge-based	Canny/Sobel Edge	Simple, Fast	Noise sensitive
Hough Transform	Line Voting in (ρ,θ)	Robust to partial gaps	Limited to straight lines
Polynomial(IPM)	BEV + Sliding Window	Works with curves, easy to tune	Requires camera calibration
Deep Learning	CNN/Transformer	Context-aware, Robust	High computational cost

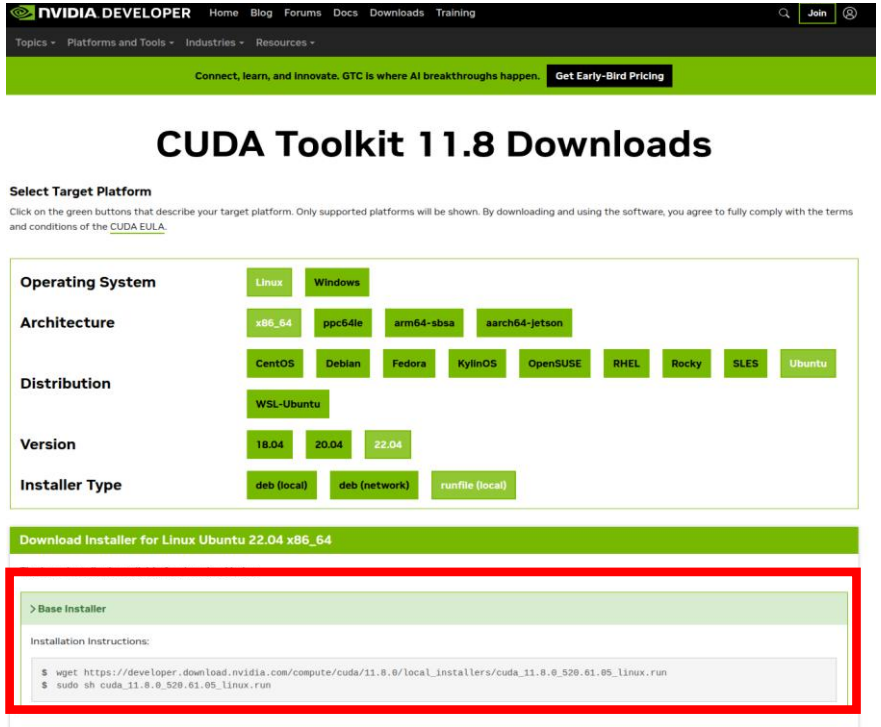
ROS2-CARLA Lane Detection

- Ultra Fast Lane Detection 설치
  - 1. CUDA 11.8 설치

(terminal 1) $ https://developer.nvidia.com/cuda-12-8-1-download-archive

- 설치 후 .bashrc 최하단에 하단 문구 추가(gedit ~/.bashrc)

1. export PATH="/usr/local/cuda-11.8/bin:$PATH"
2. export LD_LIBRARY_PATH="/usr/local/cuda-11.8/lib64:$LD_LIBRARY_PATH"




```
avl@avl:~/Downloads$ wget https://developer.download.nvidia.com/compute/cuda/11.
8.0/local_installers/cuda_11.8.0_520.61.05_linux.run
--2026-01-04 22:34:31--  https://developer.download.nvidia.com/compute/cuda/11.8
.0/local_installers/cuda_11.8.0_520.61.05_linux.run
Resolving developer.download.nvidia.com (developer.download.nvidia.com)... 23.59
.252.129, 23.59.252.145, 23.59.252.137, ...
Connecting to developer.download.nvidia.com (developer.download.nvidia.com)|23.5
9.252.129|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4336730777 (4.0G) [application/octet-stream]
Saving to: ‘cuda_11.8.0_520.61.05_linux.run’

1.05_linux.run       33%[=====>              ]   1.35G  13.9MB/s    eta 3m 43s
```

ROS2-CARLA Lane Detection

- Ultra Fast Lane Detection 설치
  - 2. Dependency 설치

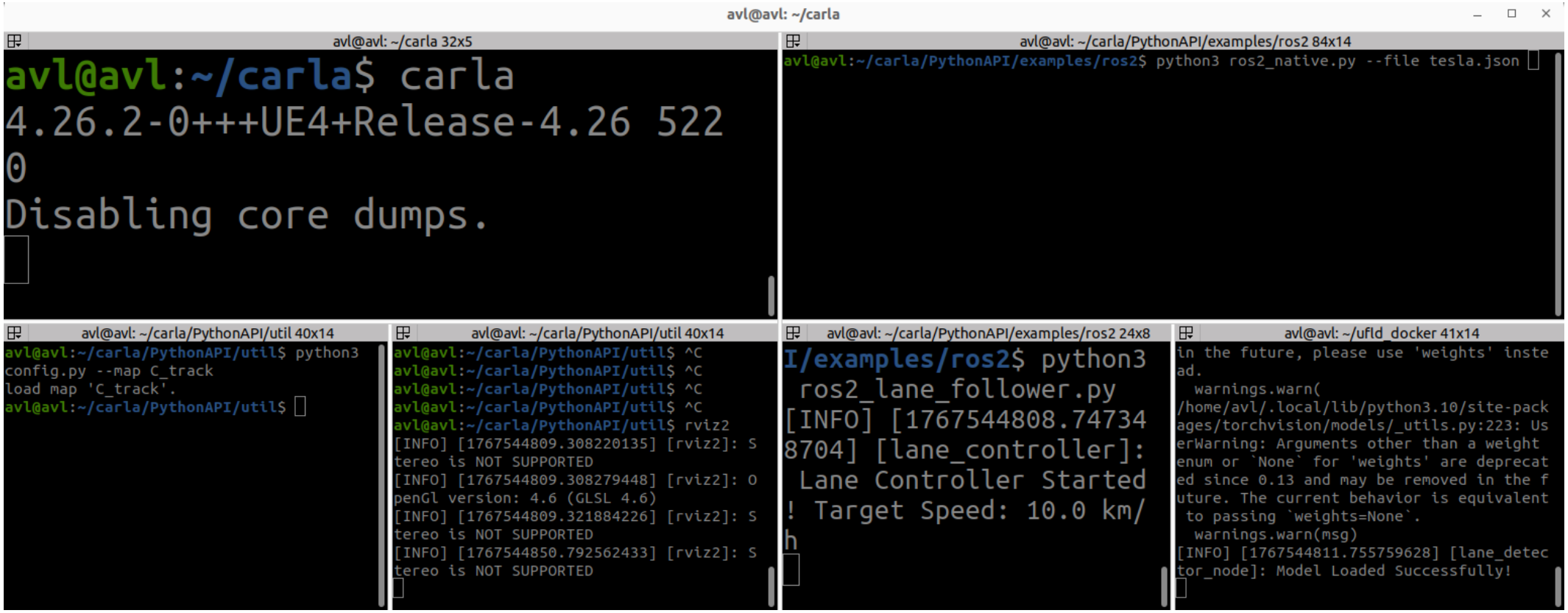
```
(terminal 1) $ pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
             $ pip3 install scipy numpy==1.21.4 opencv-python==4.5.4.60 opencv-contrib-python==4.5.4.60 matplotlib
```

  - 3. UFLD 소스 빌드

```
(terminal 1) $ cd ~/ufld_docker
             $ colcon build --symlink-install --packages-select ufld_ros
             $ source install/setup.bash
             $ ros2 run ufld_ros lane_detector # Model Loaded Successfully! 출력 확인
```

ROS2-CARLA Lane Detection

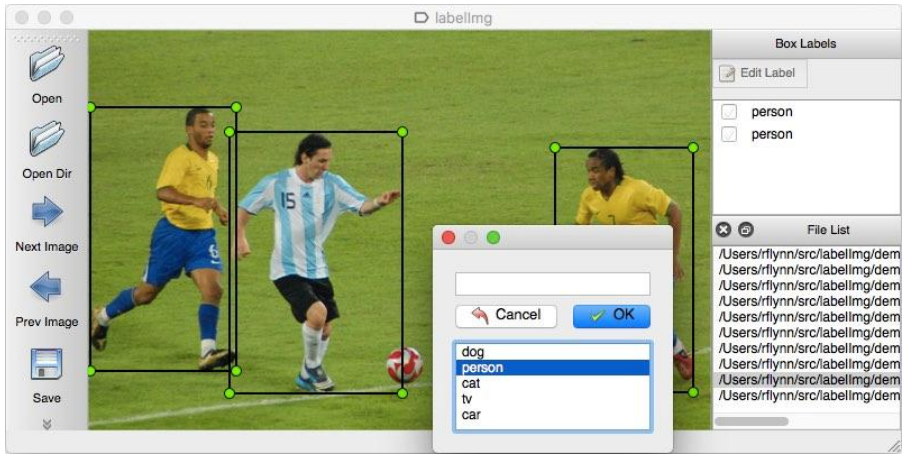
- 코드 실행



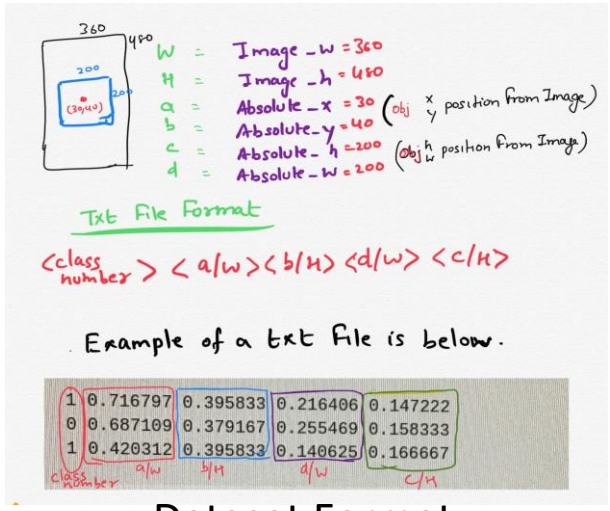
ROS2-CARLA Lane Detection

2D Object Detection

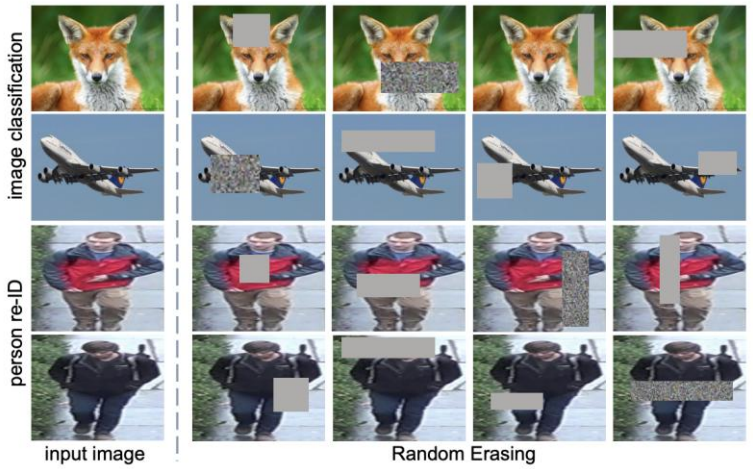
- YOLO(You Only Look Once)
  - 검출해야 할 대상을 학습 데이터로써 취득 및 가공 절차가 필요
    - 1. Teleop_key를 이용하여 Carla 차량 이미지 데이터 취득 및 저장
    - 2. 저장한 이미지를 가공
      - Annotation(Labeling)
      - Dataset format
      - Augmentation
    - 3. 학습 시 데이터는 Train/Validation/Test를 분리
      - 데이터 규모, 분포, 증강 등에 따라 분배 비율이 다름(8:1:1, 7:2:1 등)



Data Annotation



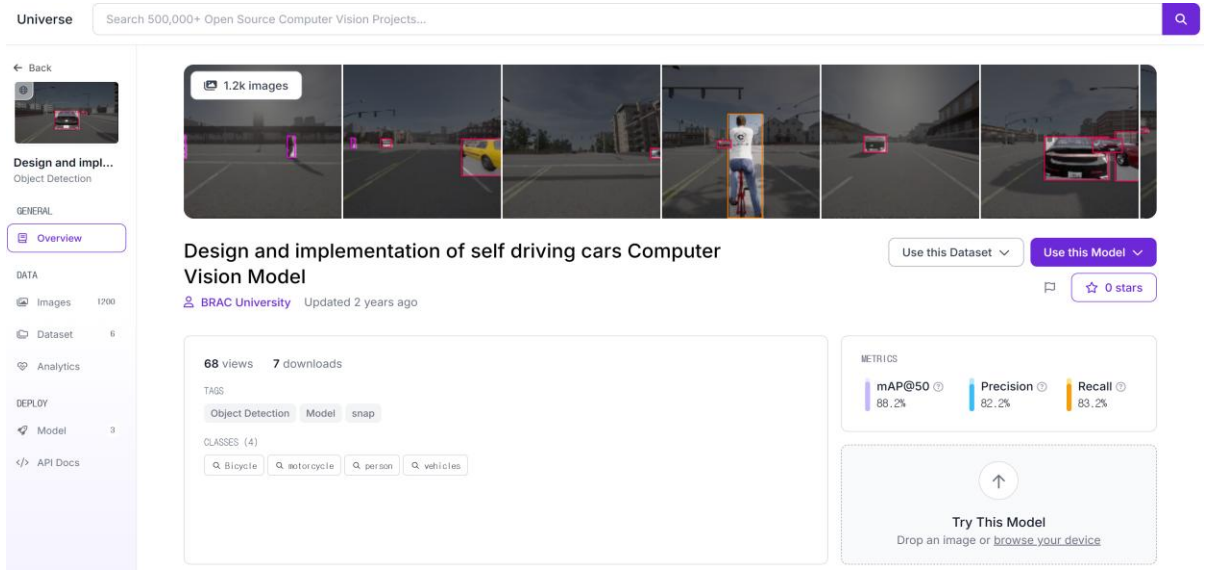
Dataset Format

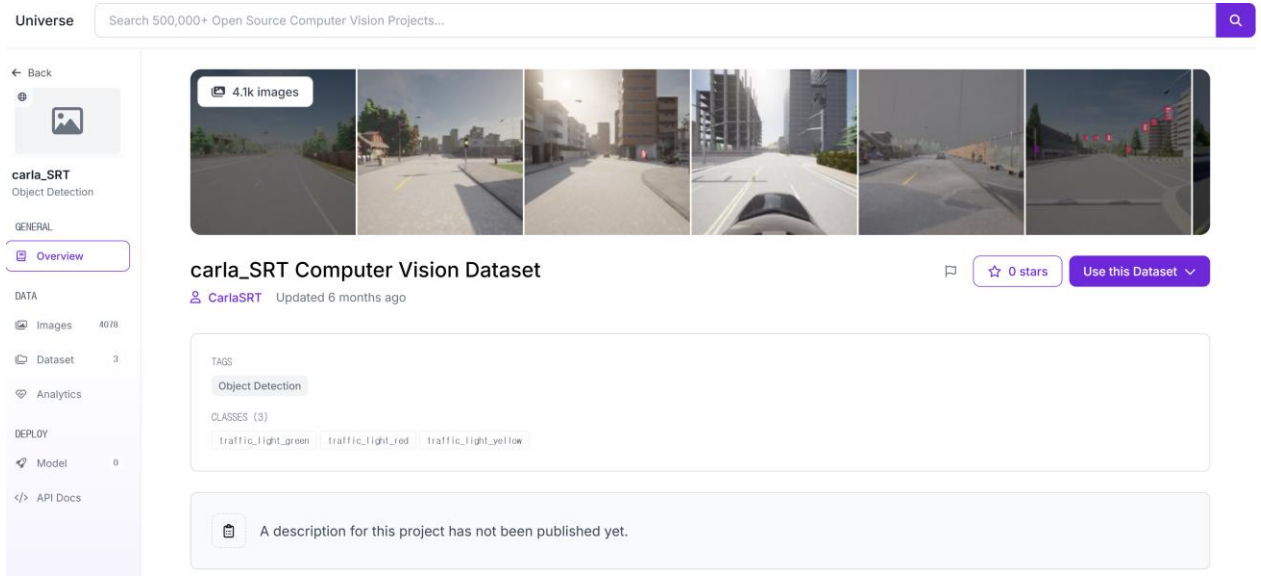


Data Augmentation

▶ 학습 데이터 취득(Roboflow)

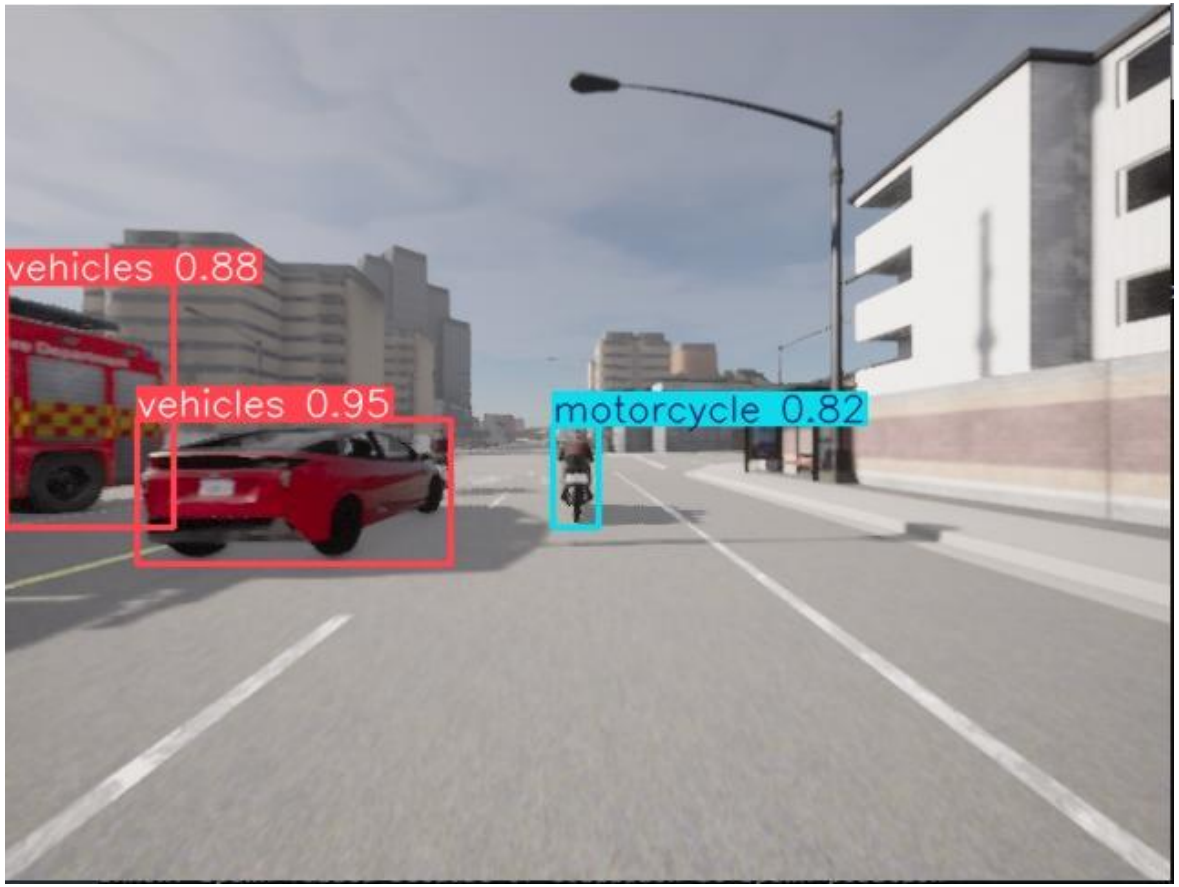
- Roboflow
  - **Object Detection**, **Image Classification**에 특화된 모델/데이터셋 관련 온라인 플랫폼
    - 차량/자전거/오토바이/보행자 dataset: https://universe.roboflow.com/carlasrt/carla_srt
    - 신호등 dataset: https://universe.roboflow.com/brac-university-1bpok/design-and-implementation-of-self-driving-cars

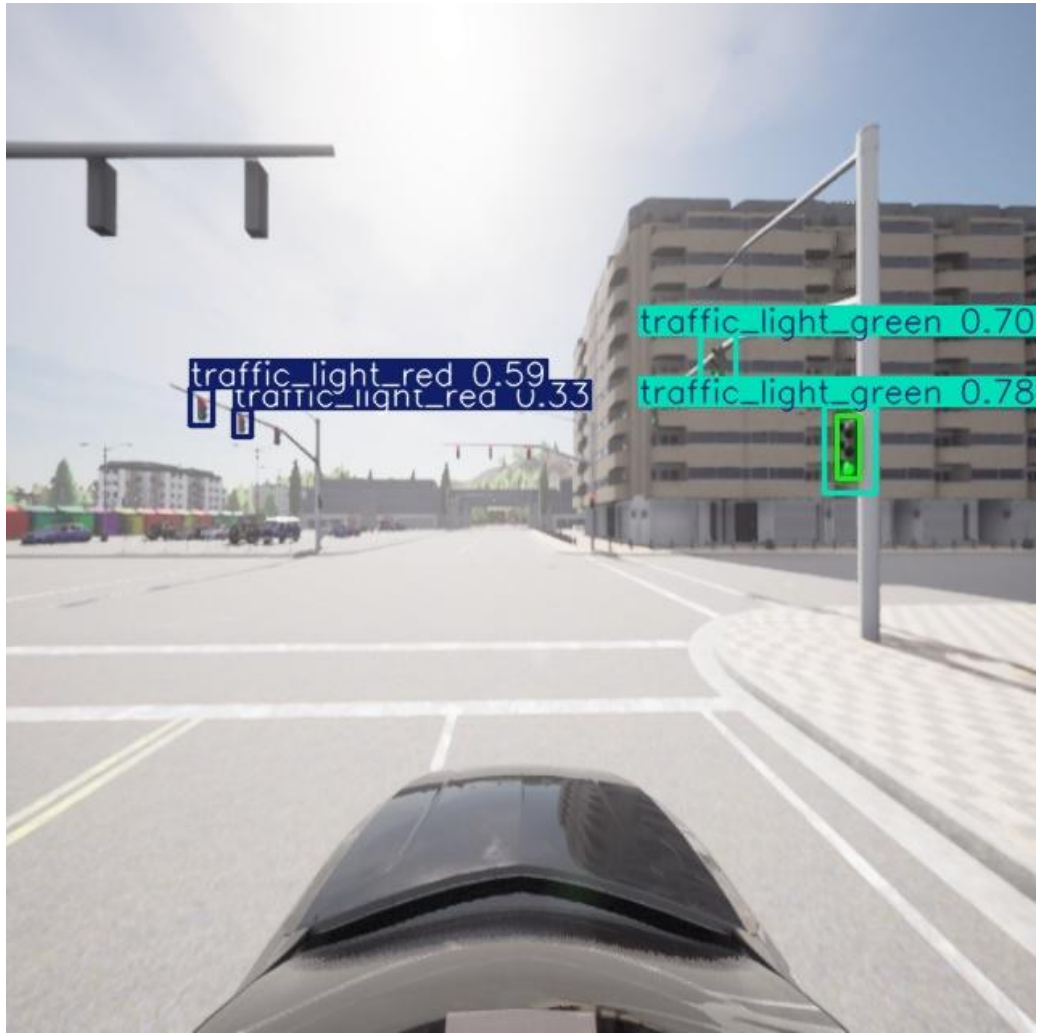




ROS2-CARLA 2D Object Detection

- Pretrained Model 이용
  - carla_yolov8n.pt
  - ros2_yolo_detection.py





ROS2-CARLA 2D Object Detection

- ros2_yolo_detection.py
  - 1. 입력: CARLA 차량 전방 카메라(/carla/hero/camera_front/image)
  - 2. ROS2 image -> OpenCV nD-array 변환
  - 3. width, height resizing(512x320)
  - 4. carla_yolov8n.pt 가중치 파일 로드(FP16)
  - 5. Bounding Box 추출
    - Center_x, y, size_x, y
    - Class id, name, Confidence score
  - 6. 출력: bounding box(/carla/object_detection_2d/bounding_box), visualization image(/carla/object_detection_2d/debug_image)
  - Parameters
    - model_path
    - device
    - conf_threshold, iou_threshold
    - img_width, img_height
    - publish_debug_image

ROS2-CARLA 2D Object Detection

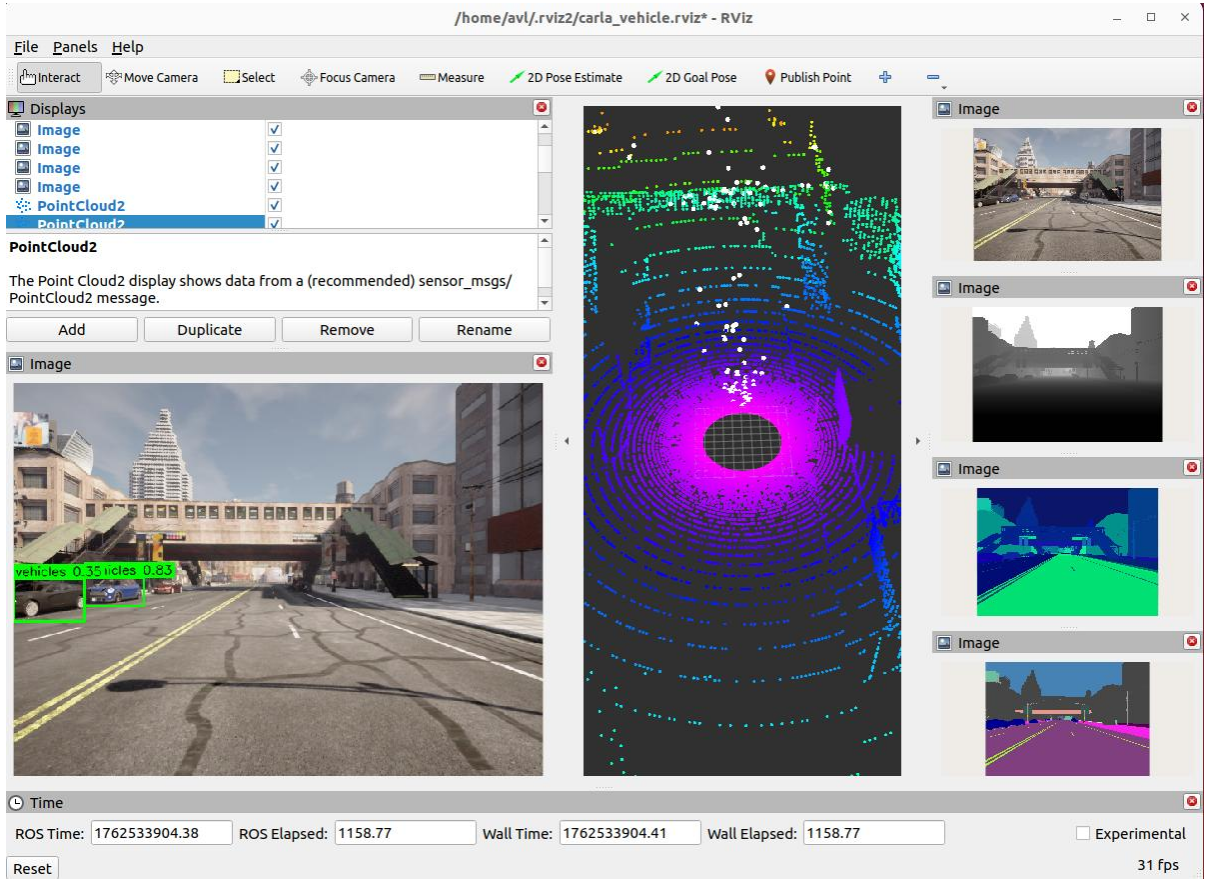
- Dependency 설치(anaconda)
  - https://www.anaconda.com/download/success

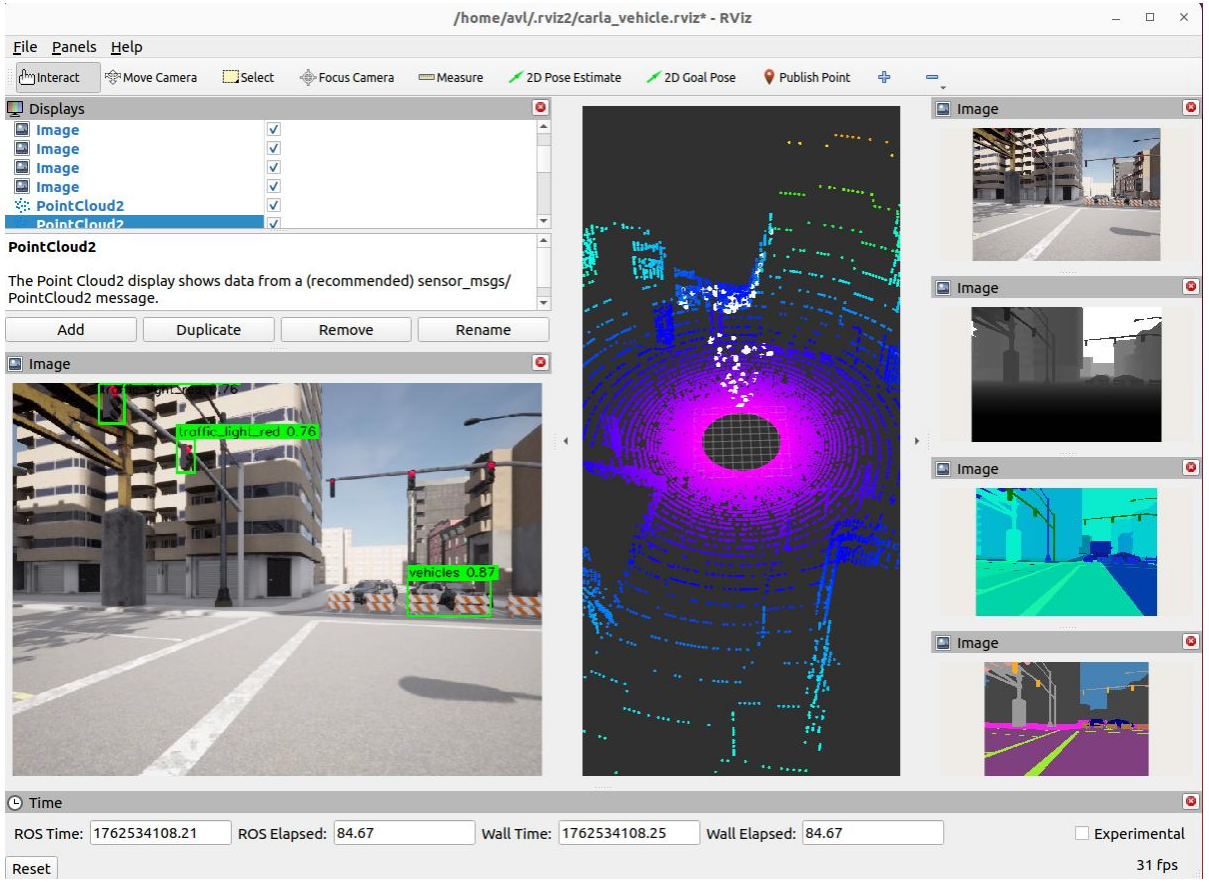
```
(terminal 1) $ cd Downloads && sh Anaconda3-2025.xx-x-Linux-x86_64.sh
             $ export PATH=~/anaconda3/bin:$PATH
             $ conda config --set auto_activate_base false
             $ anaconda create -n yolo python=3.10
             $ conda activate yolo
             $ conda install -c conda-forge "libstdcxx-ng>=12"
             $ pip install --no-cache-dir "numpy==1.24.4" "opencv-python==4.8.1.78"
```

- YOLO Library(Ultralytics) 설치 및 코드 실행

```
(terminal 1) (yolo) $ pip install ultralytics
              (yolo) $ python3 ros2_yolo_detection.py
(terminal 2) $ cd ~/carla/PythonAPI/examples/ros2
             $ python3 ros2_native.py
(terminal 3) $ cd ~/carla/PythonAPI/examples/ros2
              $ python3 teleop_key.py
(terminal 4) $ rviz2 (Add - By topic - /carla/object_detection_2d/debug_image)
```

ROS2-CARLA 2D Object Detection 실행 결과





LiDAR-based 3D Object Detection

- Point Cloud Processing Pipeline
  - 1. Preprocessing
    - ROI filtering (Region of Interest)
      - 전방 일정 거리 이내만 사용
    - Ground removal
      - 지면 평면 추정 (RANSAC 등) 후 제거
  - 2. Feature Extraction
    - 점들의 density, height difference, reflectivity 등
    - Local voxel 또는 BEV grid 상으로 투영
  - 3. Object Clustering
    - Euclidean Distance Clustering (EDC)
      - 포인트 간 거리 기반, 일정 반경 내 점들을 그룹화
    - DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
      - 밀도 기반 자동 클러스터링
    - Voxel-based Grouping
      - 공간을 균등한 voxel로 나누고 인접 voxel과 병합
  - 4. Bounding Box Estimation
    - 각 클러스터에 대해 최소 외접 박스(Minimum Bounding Box, MBB) 계산
    - 중심 좌표, 크기, 방향 추정

LiDAR-based 3D Object Detection

- Learning-based 3D Detection
  - 전통적인 클러스터링 대신 point-wise feature learning 사용
    - 객체 클래스, 방향, 크기까지 end-to-end 예측 가능
    - 단점으로는 고가 GPU, 대용량 데이터셋(KITTI, nuScenes 등) 필요 및 시스템 통합 시 실시간성 확보가 어려움
- Camera-LiDAR Fusion
  - 라이다의 정확한 geometry 정보, 카메라의 풍부한 semantic 정보 결합
    - 결합 시 3D localization + semantic labeling 정밀도 향상
  - Early Fusion, Late Fusion, Intermediate Fusion으로 구분
    - Early Fusion
      - LiDAR point cloud를 카메라 이미지 좌표계로 투영 -> RGB feature와 병합
    - Late Fusion
      - 두 센서의 탐지 결과(e.g., Bounding Box)를 후단에서 병합 (tracking or decision level)
    - Intermediate Fusion
      - Feature-level에서 attention 또는 cross-modal transformer로 융합

LiDAR-based 3D Object Detection Summary

분류	입력값	방법	성능	취약성
Classical	PointCloud	Euclidean / DBSCAN	Simple, Fast	Weak classification
Voxel-based DL	Discretized 3D grid	3D CNN	Structure-aware	Heavy computation
Fusion	LiDAR+Camera	Feature/Decision fu sion	High accuracy	Complex calibration

ROS2-CARLA 3D Object Detection

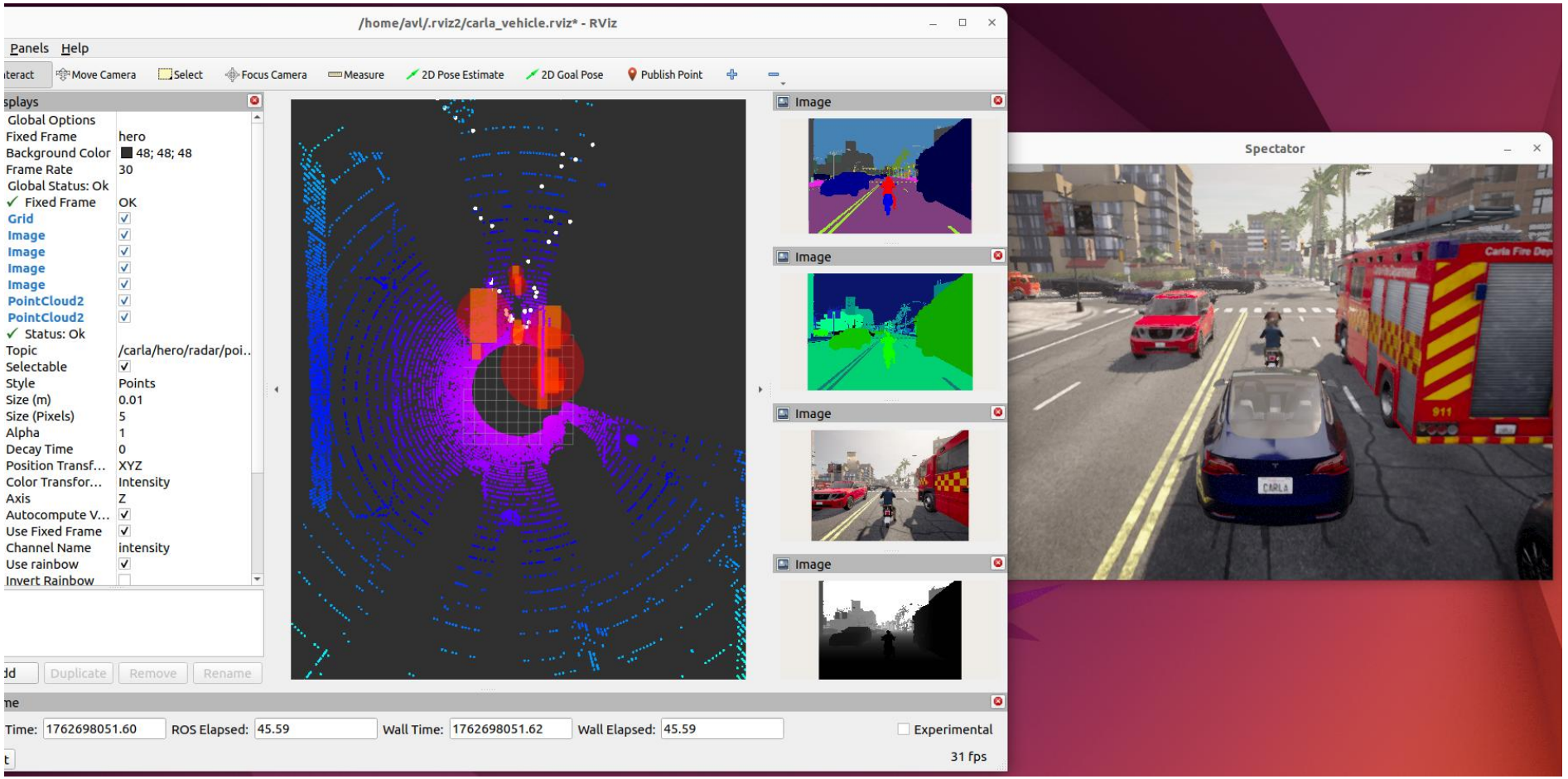
- ros2_lidar_clustering.py
  - 1. 입력: LiDAR 포인트클라우드(/carla/hero/lidar/point_cloud)
  - 2. ROI 자르기(전방 0~35 m, 좌우 ±6 m, z -3~3 m)
  - 3. 근거리 z 백분위 기반 지면 자동 추정 -> 지면 제거
  - 4. 2D 격자화(voxel=0.25 m) -> 8-connected grid 클러스터링
  - 5. 클러스터 중심/크기 산출(마진(inflate) 고려)
- 출력
  - 장애물 중심들(/carla/obstacles_2d, geometry_msgs/PoseArray)
  - 시각화 마커(/carla/obstacles_markers, visualization_msgs/MarkerArray)
- Parameters
  - X_MAX=35.0, Y_ABS=6.0
  - VOX=0.25, MIN_PTS=8
  - GROUND_PCT=12.0, GROUND_EPS=0.18
  - inflate_r=0.7

ROS2-CARLA 3D Object Detection

- 코드 실행 설치

```
(terminal 1) (yolo) $ pip install ultralytics
                    (yolo) $ python3 ros2_yolo_detection.py
(terminal 2) $ cd ~/carla/PythonAPI/examples/ros2
                   $ python3 ros2_native.py
(terminal 3) $ cd ~/carla/PythonAPI/examples/ros2
                    $ python3 teleop_key.py
(terminal 4) $ rviz2 (Add - By topic - /carla/object_detection_2d/debug_image)
```

ROS2-CARLA 3D Object Detection 실행 결과



Global Path Planning

- Global Path Planning은 차량의 현재 위치에서 목적지까지의 optimal global route를 생성하는 단계
  - 일반적으로 고정된 map 또는 global coordinates 상에서 수행
  - 입력 및 출력
    - $Input: (x_{start}, y_{start}), (x_{goal}, y_{goal}), Map\ Graph$
    - $Output: Sequence\ of\ Waypoints\ \{w_0, w_1, \dots, w_n\}$
  - Map Representation
    - Graph-based Representation
      - 도로를 node, 교차점을 edge로 모델링
      - 각 edge는 거리, 제한속도, 차선 수 등을 weight로 가짐
    - Grid-based Representation
      - 지도 영역을 일정 크기의 occupancy grid로 표현(1: 점유, 0: 비점유)

Global Path Planning Algorithms

알고리즘	구조	특성
Dijkstra's Algorithm	Uniform cost search	Shortest path guarantee, slow
A* (A-star)	Heuristic function $f(n) = g(n) + h(n)$	speed ↑, accuracy ↑(real-time X)
D* (Dynamic A-star)	Online version of A*	Dynamic environment adaptation
Hybrid A*	A* + Vehicle Dynamics (steering angle limit)	Real vehicle driving routes can be created
RRT / RRT*	Randomized sampling	Can be explored on complex maps

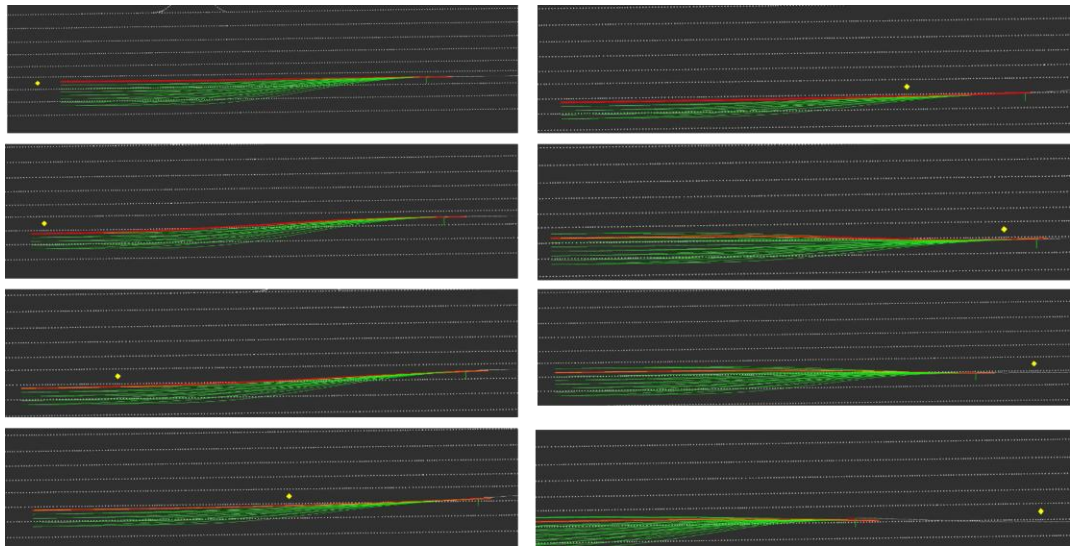
- $g(n)$: 시작점 -> 현재 노드까지의 cost
- $h(n)$: heuristic(목표까지의 추정 거리(Euclidean distance 등))

Global Path Planning Algorithms

- Coordinate Systems
  - GNSS 좌표계 (WGS84 → UTM 변환)
    – 위도·경도 데이터를 미터 단위 평면 좌표로 변환해야 경로 계획이 가능함
  - Local Map Frame
    – 시작 위치를 원점(0,0)으로 정규화하여 상대적 이동량 계산
- 실제 차량 또는 CARLA에서의 활용
  - 차량의 RTK-GNSS 센서 데이터를 기반으로 waypoint 시퀀스 생성
    - 혹은 차선, 연석, 노면 표시, 정지선 등 도로 정보 등이 담긴 지도(HD-Map 등) 사용
  - CARLA의 경우 /carla/hero/gnss 토픽 또는 .osm 맵 파일을 기반으로 waypoint 추출

Local Path Planning

- Global Path를 기반으로 차량의 현재 주행 상황을 반영하여 단기적으로 따라갈 안전한 Local Trajectory를 생성하는 단계
  - 차량 주변 수 m ~ 수십 m 범위에서 실시간 생성
  - 주로 10Hz~50Hz의 높은 주기로 업데이트
  - 입력 및 출력
    - $Input$
      - Global Path (Waypoints)
      - Vehicle State (x, y, yaw, v)
      - Perception Results (Obstacle positions, Free space 등)
    - $Output$
      - $\{p_0, p_1, \dots, p_k\}$



Local Path Planning Algorithms

알고리즘	구조	특성
Graph-based Local Planning	Node–edge graph	reliable, slow
Optimization-based Planning	Cost optimization	smooth path, heavy computation
Sampling-based Planning	Multiple samples	collision check, robust
Rule-based Planning	Simple rules	very fast, limited performance

Vehicle Control

- Global/Local Path를 실제 차량의 조향·가속·감속 명령으로 변환하여 원하는 경로를 정확히 따라가도록 하는 단계
  - Lateral Control, Longitudinal Control로 구분
  - 입력 및 출력
    - $Input$
      - $Global\ /\ Local\ Path\ (target\ trajectory)$
    - $Output$
      - $Steering\ Angle$
      - $Throttle\ /\ Brake$

Control Algorithms

알고리즘	구조	특성
PID Speed Control(Longitudinal)	Feedback control	simple tuning, widely used
Model Predictive Control(Longitudinal)	Model-based optimization	constraint handling, high performance
Pure Pursuit(Lateral)	Geometric look-ahead	stable, easy to implement
Stanley Controller(Lateral)	Cross-track error control	accurate at low speed

System Integration(example)

- 입력 토픽
  - 카메라 전방 RGB/Semantic, LiDAR PointCloud2, Ego Pose(/carla/ego_pose)
- Perception
  - 차선 중심·요각 추정(/lane/steering_angle)
  - 장애물 클러스터(/carla/obstacles_2d, /carla/obstacles_markers)
- Planning
  - 글로벌 경로 생성(/carla/path/global) -> 로컬 경로 생성(/carla/path/local)
- Control
  - 입력: 로컬 경로 + 차선 오프셋/요각(옵션) + 장애물
  - 출력: /carla/hero/cmd_vel (linear.x: 목표속도, angular.z: 조향각[deg])

Thank You